

ỦY BAN NHÂN DÂN TP . HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC SÀI GÒN
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO BÀI TẬP TUẦN

Học phần: Seminar Chuyên đề

Đề tài: Xây dựng sản phẩm phần mềm dùng GenAI

Giảng viên hướng dẫn: TS. Đỗ Như Tài

Lớp: DCT122C3

Nhóm sinh viên thực hiện:

STT	Họ và tên	MSSV
1	Nguyễn Hữu Anh Khoa	3122411098
2	Võ Thành Danh	3122411024
3	Trang Gia Huy	3122411068
4	Nguyễn Lê Quỳnh Hương	3122411078

MỤC LỤC

LỜI MỞ ĐẦU	12
CHƯƠNG 12: Documenting Code with GenAI	13
12.1. Giới thiệu	13
12.2. Tổng quan về Software Documentation	13
12.2.1. Khái niệm	13
12.2.2. Vai trò của documentation	14
12.3. Docstrings trong Python	14
12.3.1. Định nghĩa	14
12.3.2. Vai trò của docstrings	15
12.3.3. Các định dạng docstring phổ biến	16
12.4. Sử dụng GenAI để tạo docstrings	16
12.4.1. Với GitHub Copilot	16
12.4.2. Tạo docstring cho toàn bộ file	18
12.4.3. Hạn chế của GenAI	20
12.5. Mối quan hệ giữa Code và Docstring	20
12.5.1. Vấn đề: Docstring lỗi thời (Outdated)	20
12.5.2. Hai hướng tiếp cận	21
12.6. Giải pháp với GenAI	22
12.6.1. Tự động cập nhật docstring	22
12.6.2. Phát hiện docstring không khớp	22
12.6.3. Sử dụng Github Copilot	23
12.6.4. Sử dụng OpenAI API	25
12.7. Refactoring và Docstrings	25
12.8. Kết luận	26
CHƯƠNG 13: Writing and Maintaining Unit Tests	27
13.1. Giới thiệu	27
13.2. Unit Testing và vai trò trong phát triển phần mềm	27
13.2.1. Khái niệm và Đặc điểm của Unit Test	27
13.2.2. Vai trò của Unit Test	28

13.3. Ứng dụng GenAI trong Unit Testing.....	28
13.3.1. Khái niệm về n-grams.....	28
13.3.2. Lợi ích của GenAI.....	29
13.3.3. Hạn chế của GenAI.....	30
13.4. Sử dụng GitHub Copilot để tạo Unit Test.....	31
13.4.1. Quy trình thực thi chuẩn (Step-by-Step Workflow).....	31
13.4.2. Các đặc điểm hỗ trợ linh hoạt.....	32
13.4.3. Những lưu ý quan trọng khi vận hành.....	33
13.5. Sử dụng ChatGPT để viết Unit Test.....	33
13.5.1. Ưu điểm và Khả năng hỗ trợ phân tích.....	34
13.5.2. Những thách thức và hạn chế.....	35
13.6. Data-Driven Testing.....	35
13.6.1. Khái niệm cốt lõi của Data-Driven Testing.....	35
13.6.2. Ưu điểm.....	35
13.6.3. Vai trò của GenAI.....	36
13.7. Test-Driven Development (TDD).....	36
13.7.1. Khái niệm và Chu trình Red-Green-Refactor.....	36
13.7.2. Lợi ích.....	37
13.7.3. Phương pháp thực hiện.....	37
13.8. Kết hợp GenAI với TDD.....	38
13.8.1. Mô tả bài toán.....	38
13.8.2. Các trường hợp cần xử lý.....	39
13.8.4. Sử dụng GenAI cho chu trình TDD.....	41
13.8.5. Quy trình phối hợp thực tế.....	43
13.9. Tạo dữ liệu kiểm thử bằng GenAI.....	43
13.9.1. Vai trò chiến lược của dữ liệu kiểm thử.....	43
13.9.2. Ứng dụng GenAI.....	43
13.9.3. Khả năng thực thi của GenAI trong khởi tạo dữ liệu.....	44
13.10. Kết luận.....	45
CHƯƠNG 14: GenAI for Runtime and Memory Management.....	47
14.1. Giới thiệu.....	47
14.2. Phân tích Runtime và Space Complexity.....	47

14.2.1 Runtime của chương trình	47
14.2.2 Bộ nhớ của chương trình	48
14.2.3 Trade-off giữa Runtime và Memory	48
14.3. <i>Profiling (Đo hiệu năng)</i>	49
14.3.1. Profiling Runtime (Đo thời gian thực thi)	49
14.3.2 Profiling Memory	51
14.4. <i>Phân tích Max Capacity bằng ChatGPT</i>	52
14.4.1. Khái niệm Max Capacity (Giới hạn xử lý tối đa)	52
14.4.2 Phương pháp	53
14.4.3 Kết quả thực nghiệm	54
14.5. <i>Tối ưu code bằng Chained Prompts</i>	55
14.5.1. Khái niệm Chained Prompt	56
14.5.2 Tối ưu Runtime	56
14.5.3 Tối ưu Memory	58
14.6. <i>Các hướng tối ưu nâng cao</i>	60
14.7. <i>Kết luận</i>	60
CHƯƠNG 15: Đưa GenAI vào Hoạt động: Ghi nhật ký, Giám sát và Lỗi	62
15.1. <i>Giới thiệu về logging, monitoring và cơ chế phát sinh lỗi</i>	62
15.1.1. Tổng quan	62
15.1.2. Logging (Ghi log)	62
15.1.3. Monitoring (Giám sát hệ thống)	63
15.1.4. Raising Errors (Xử lý lỗi)	64
15.2. <i>Ứng dụng GenAI trong việc xây dựng các mô hình thiết kế code ở mức cao</i>	65
15.2.1. Ý tưởng	65
15.2.2. Decorator Pattern	65
15.2.3. Triển khai CoT ngược với Decorator	66
15.3. <i>Ứng dụng CoT ngược với ChatGPT và OpenAI</i>	69
15.3.1. Khái niệm	69
15.3.2. Ứng dụng CoT ngược với ChatGPT	69
15.3.3. Ứng dụng CoT ngược với OpenAI	72
15.4. <i>Sử dụng Few-shot Learning và Fine-tuning làm Style Guide</i>	73
15.4.1. Few-shot learning với GitHub Copilot	74

15.4.2. Few-shot learning với ChatGPT	75
15.4.3. Fine-tuning với OpenAI API	79
15.5. <i>Kết luận</i>	84
CHƯƠNG 16: Architecture, Design, and the Future	85
16.1. <i>Giới thiệu</i>	85
16.2. <i>Sự phát triển nhanh chóng của GenAI</i>	85
16.3. <i>Tác động đến kinh tế phần mềm</i>	85
16.4. <i>Mức độ chấp nhận của GenAI</i>	86
16.5. <i>Thay đổi trong vai trò của lập trình viên</i>	87
16.5.1. Lập trình viên mới (Junior Developer).....	87
16.5.2. Lập trình viên cấp cao và chuyên gia (Senior Developer)	88
16.6. <i>GenAI và SWEBOK</i>	88
16.6.1. Các lĩnh vực chịu tác động mạnh (AI-Dominant)	89
16.6.2. Các lĩnh vực vẫn do con người dẫn dắt (Human-Centric)	89
16.7. <i>Dân chủ hóa lập trình (Democratization)</i>	90
16.8. <i>Ảnh hưởng đến hệ thống legacy và tổ chức</i>	91
16.8.1. Hồi sinh các hệ thống Legacy.....	91
16.8.2. Tăng cường năng lực và tính linh hoạt của tổ chức	91
16.9. <i>Tương lai của ngôn ngữ lập trình</i>	92
16.10. <i>Vibe Coding</i>	92
16.11. <i>Tương lai của software engineering</i>	93
16.11.1. Tầm nhìn Ngắn hạn (1-3 năm tới).....	93
16.11.2. Tầm nhìn Dài hạn (4-10 năm tới).....	94
16.12. <i>Liệu AI có thay thế lập trình viên?</i>	95
16.12.1. Tại sao AI chưa thể thay thế hoàn toàn con người?	95
16.12.2. Lập trình viên tương lai sẽ làm gì?.....	95
16.13. <i>Rủi ro và quản trị</i>	96
16.13.1. Những thách thức cốt lõi	96
16.13.2. Quản trị nghiêm ngặt trong các lĩnh vực nhạy cảm.....	96
16.14. <i>Tác động đến giáo dục và tuyển dụng</i>	97

16.14.1. Sự dịch chuyển trong Giáo dục Đào tạo	97
16.14.2. Cách mạng trong Tuyển dụng.....	97
16.15. <i>Kết luận</i>	98
DANH MỤC TÀI LIỆU THAM KHẢO	99

DANH MỤC HÌNH ẢNH

Hình 1: Tạo docstring bằng Copilot.....	17
Hình 2: Docstring sau khi được tạo bằng Copilot.....	18
Hình 3: Tạo docstring cho toàn bộ file.....	19
Hình 4: Kiểm tra các docstring bị lỗi thời.....	24
Hình 5: Sử dụng OpenAI API.....	25
Hình 6: Bài tập n-grams	29
Hình 7: Kết quả bài tập n-grams	29
Hình 8: Quy trình thực thi – Thiết lập framework	31
Hình 9: Quy trình thực thi – Kích hoạt lệnh sinh test	32
Hình 10: Quy trình thực thi – Kiểm tra và hiệu chỉnh	32
Hình 11: Sử dụng ChatGPT để viết Unit Test.....	34
Hình 12: Hình vẽ mô tả bài toán.....	39
Hình 13: Hai hình chữ nhật có phần giao nhau.....	40
Hình 14: Hai hình chữ nhật không giao nhau	40
Hình 15: Dữ liệu hình chữ nhật không hợp lệ.....	41
Hình 16: Sử dụng GenAI cho chu trình TDD	42
Hình 17: Kết quả đạt được	42
Hình 18: Dùng GenAI để khởi tạo dữ liệu.....	44
Hình 19: Kết quả nhận được	44
Hình 20: Phân tích Runtime và Space Complexity của bài toán Fibonacci.....	48
Hình 21: Mô phỏng bộ nhớ của chương trình.....	48
Hình 22: Đo thời gian thực thi	49
Hình 23: Vai trò của Github Copilot trong Profiling.....	50
Hình 24: Kết quả thực thi.....	50
Hình 25: Profiling Memory	51

Hình 26: Kết quả thực thi.....	52
Hình 27: Cạn kiệt bộ nhớ (Out of Memory - RAM):.....	53
Hình 28: thu thập các điểm dữ liệu thực tế về mức độ tiêu thụ RAM và thời gian thực thi (Runtime).....	54
Hình 29: Kiểm chứng khả năng dự báo của GenAI.....	55
Hình 30: Chained Prompt	56
Hình 31: Bài toán tìm số Fibonacci khi áp dụng các kỹ thuật toán học cao cấp.....	57
Hình 32: Tối ưu Memory bằng kỹ thuật Chunking.....	58
Hình 33: Kết quả thực nghiệm trước/sau khi tối ưu bằng kỹ thuật Chunking	59
Hình 34: Quét khối đoạn code rồi gỡ prompt	63
Hình 35: Kết quả prompt monitoring.....	64
Hình 36: Raising Errors (Xử lý lỗi)	65
Hình 37: Decorator Pattern	66
Hình 38: Decorator Pattern (t.t)	66
Hình 39: Inverse Chain-of-Thought (Inverse CoT)	67
Hình 40: Khai báo decorator.....	68
Hình 41: Copilot tự gợi ý code dựa trên decorator	69
Hình 42: Ứng dụng CoT ngược với ChatGPT	70
Hình 43: Kết quả ứng dụng CoT ngược với ChatGPT	71
Hình 44: Các decorator được sinh ra	72
Hình 45: Ví dụ System Prompt.....	73
Hình 46: Ví dụ User Prompt	73
Hình 47: file style guide chứa các ví dụ code mẫu	74
Hình 48: ChatGPT sẽ sinh decorator theo đúng style của ví dụ	75
Hình 49: Ví dụ về input (INCOMPLETE_CODE).....	76
Hình 50: Ví dụ về output (COMPLETE_CODE).....	77
Hình 51: FizzBuzz Printer	78

Hình 52: ChatGPT sẽ sinh decorator	78
Hình 53: output chuẩn mà model cần học theo.....	79
Hình 54: Quy trình upload và huấn luyện mô hình fine-tuning (p1)	80
Hình 55: Quy trình upload và huấn luyện mô hình fine-tuning (p2)	81
Hình 56: Quy trình upload và huấn luyện mô hình fine-tuning (p3)	82
Hình 57: fine-tuned mode	83
Hình 58: Kết quả khi chạy prompt.....	84
Hình 59: Mức độ chấp nhận của GenAI	87
Hình 60: Các lĩnh vực vẫn do con người dẫn dắt.....	90

DANH MỤC BẢNG

Bảng 1: Các định dạng docstring phổ biến	16
--	----

BẢNG PHÂN CÔNG CÔNG VIỆC

STT	Họ và tên	MSSV	Nội dung công việc
1	Nguyễn Hữu Anh Khoa	3122411098	- Nhóm trưởng - Phân chia công việc, chuẩn bị cấu trúc mã nguồn, nội dung mẫu - Thực hiện Lab và soạn nội dung và thực hiện Chương 13, 14
2	Võ Thành Danh	3122411024	- Thực hiện Lab - Soạn nội dung Chương 15
3	Trang Gia Huy	3122411068	- Thực hiện Lab - Soạn nội dung Chương 12 và báo cáo tổng
4	Nguyễn Lê Quỳnh Hương	3122411078	- Thực hiện Lab - Soạn nội dung Chương 16 và slide thuyết trình

LỜI MỞ ĐẦU

Trong những năm gần đây, cuộc cách mạng về Trí tuệ nhân tạo (AI), đặc biệt là Trí tuệ nhân tạo tạo sinh (Generative AI - GenAI), đã tạo nên những thay đổi mang tính bước ngoặt trong nhiều lĩnh vực của đời sống xã hội. Đối với lĩnh vực kỹ thuật phần mềm, GenAI không chỉ dừng lại ở vai trò là một công cụ hỗ trợ mà đã trở thành một người bạn đồng hành, làm thay đổi tư duy và quy trình làm việc truyền thống của các lập trình viên.

Việc ứng dụng các mô hình ngôn ngữ lớn (LLMs) như GPT của OpenAI hay các công cụ hỗ trợ mã nguồn như GitHub Copilot đã mở ra những cơ hội chưa từng có. Tuy nhiên, để làm chủ được những công nghệ này một cách hiệu quả, đảm bảo chất lượng mã nguồn và tối ưu hóa chi phí vận hành, người kỹ sư cần có một cách tiếp cận hệ thống, từ việc hiểu rõ cơ chế hoạt động của API đến việc rèn luyện kỹ năng đặt câu lệnh (Prompt Engineering).

Báo cáo này được thực hiện nhằm mục đích nghiên cứu và thực hành quá trình ứng dụng GenAI vào toàn bộ chu kỳ phát triển phần mềm (SDLC). Nội dung báo cáo sẽ đi sâu vào các khía cạnh:

- Tổng quan về cơ hội của GenAI trong việc chuyển đổi từ tự động hóa đơn thuần sang hỗ trợ toàn diện vòng đời phần mềm.
- Kỹ thuật tương tác với OpenAI API thông qua lập trình Python, quản lý tài nguyên và chi phí.
- Thực hành sử dụng GitHub Copilot trên các môi trường phát triển phổ biến (VS Code, PyCharm, Jupyter Notebook).
- Xây dựng bộ quy tắc (Best Practices) để tối ưu hóa hiệu quả khi đặt câu hỏi cho ChatGPT và các công cụ hỗ trợ.

Thông qua quá trình nghiên cứu và thực hiện các bài thực hành (Labs) dựa trên tài liệu “Supercharged Coding with GenAI” của NXB Packt, báo cáo hy vọng sẽ đưa ra cái nhìn thực tiễn và những kỹ năng cần thiết để nâng cao năng suất lập trình trong kỷ nguyên AI.

Do kiến thức về lĩnh vực này vẫn đang phát triển không ngừng, báo cáo chắc chắn không tránh khỏi những thiếu sót. Em rất mong nhận được những ý kiến đóng góp quý báu từ Thầy để nội dung được hoàn thiện hơn.

CHƯƠNG 12: Documenting Code with GenAI

12.1. Giới thiệu

Trong quá trình phát triển phần mềm, việc viết code không phải là nhiệm vụ duy nhất của lập trình viên. Một yếu tố quan trọng không kém là tài liệu hóa mã nguồn (code documentation), giúp các lập trình viên khác (hoặc chính bản thân trong tương lai) hiểu, bảo trì và mở rộng hệ thống một cách hiệu quả.

Chương 12 tập trung vào việc sử dụng Generative AI (GenAI) như GitHub Copilot, ChatGPT và OpenAI API để tự động hóa quá trình tạo và quản lý docstrings trong Python. Đây là một bước tiến quan trọng trong việc nâng cao năng suất và chất lượng phần mềm.

12.2. Tổng quan về Software Documentation

12.2.1. Khái niệm

Trong quy trình phát triển phần mềm hiện đại, Software Documentation (Tài liệu phần mềm) không đơn thuần là những tệp văn bản đi kèm, mà đóng vai trò là cầu nối tri thức giữa nhà phát triển, hệ thống và người dùng cuối. Có thể hiểu đây là một tập hợp các tài liệu mang tính hệ thống, được thiết kế nhằm hỗ trợ tối đa việc nghiên cứu, vận hành và duy trì hiệu suất của phần mềm trong suốt vòng đời của nó.

Hệ thống tài liệu này bao quát nhiều khía cạnh khác nhau, từ những chi tiết kỹ thuật nội bộ đến các hướng dẫn mang tính ứng dụng thực tế, bao gồm:

- **Chú thích mã nguồn (Comments):** Những dòng giải thích ngắn gọn trực tiếp trong code nhằm giúp lập trình viên nắm bắt tư duy logic nhanh chóng.
- **Tài liệu API (API Documentation):** Các bản mô tả chi tiết về cách thức giao tiếp và tích hợp giữa các thành phần phần mềm hoặc giữa các hệ thống khác nhau.
- **Sơ đồ kiến trúc (System Architecture Diagrams):** Cái nhìn tổng quan về cấu trúc, mối liên kết và luồng dữ liệu của toàn bộ hệ thống.
- **Hướng dẫn sử dụng (User Guides):** Tài liệu hướng dẫn thao tác dành cho người dùng cuối, giúp họ khai thác tối đa tính năng của ứng dụng.
- **Tài liệu kiểm thử và CI/CD:** Các quy trình về kiểm tra chất lượng (testing) và các thiết lập tự động hóa trong triển khai phần mềm.

Tuy nhiên, trong phạm vi nội dung của chương này, chúng ta sẽ tập trung sâu vào khía cạnh tài liệu bên trong mã nguồn. Trọng tâm nghiên cứu sẽ xoay quanh Docstrings—một phương thức trình bày tài liệu đặc thù, cho phép giải thích chi tiết chức năng của các hàm, lớp và module một cách chuyên nghiệp và có cấu trúc ngay tại nơi mã nguồn được viết ra.

12.2.2. Vai trò của documentation

Việc duy trì một hệ thống tài liệu chuẩn mực không chỉ là một yêu cầu kỹ thuật mà còn là yếu tố sống còn quyết định sự thành công lâu dài của một dự án phần mềm. Vai trò của documentation được thể hiện rõ nét qua ba khía cạnh chiến lược sau:

Thứ nhất, tài liệu giúp tối ưu hóa khả năng tiếp cận và thấu hiểu mã nguồn. Trong môi trường làm việc nhóm hoặc khi một lập trình viên tiếp quản dự án cũ, việc đọc hiểu hàng ngàn dòng code thuần túy là một thách thức lớn. Documentation đóng vai trò như một "bản đồ tư duy", giúp người đọc nhanh chóng nắm bắt được luồng xử lý và mục đích của các module mà không cần phải phân tích từng dòng lệnh thực thi.

Thứ hai, đây là nền tảng vững chắc cho công tác bảo trì và mở rộng hệ thống. Phần mềm luôn vận động và thay đổi để đáp ứng nhu cầu mới. Nếu thiếu tài liệu, việc chỉnh sửa một tính năng nhỏ có thể dẫn đến những lỗi dây chuyền khó kiểm soát. Một hệ thống documentation tốt cho phép các kỹ sư tự tin thực hiện các thay đổi, nâng cấp hoặc tái cấu trúc (refactoring) mã nguồn mà vẫn đảm bảo tính toàn vẹn của hệ thống ban đầu.

Thứ ba, về mặt kinh tế, documentation góp phần giảm thiểu chi phí dài hạn. Mặc dù việc soạn thảo tài liệu đòi hỏi sự đầu tư về thời gian ở giai đoạn đầu, nhưng nó giúp tiết kiệm đáng kể nguồn lực trong tương lai. Nó hạn chế tối đa thời gian "chết" khi nhân sự thay đổi và giảm bớt các cuộc họp giải thích không cần thiết, từ đó tối ưu hóa hiệu suất làm việc của toàn đội ngũ.

Đặc biệt, một nguyên tắc cốt lõi cần được quán triệt trong công tác này là sự chuyển dịch trọng tâm từ mô tả "Cái gì" (WHAT) sang giải thích "Tại sao" (WHY). Thay vì chỉ trình bày những điều hiển nhiên mà mã nguồn đã thể hiện (như tên hàm hay biến), tài liệu cần tập trung làm sáng tỏ tư duy thiết kế, các giả định ngầm định và lý do tại sao một giải pháp cụ thể lại được lựa chọn thay vì các phương án khác. Chính những thông tin về "tại sao" này mới là giá trị tri thức thực sự giúp người kế nhiệm hiểu được linh hồn của bản thiết kế.

12.3. Docstrings trong Python

12.3.1. Định nghĩa

Trong ngôn ngữ lập trình Python, Docstring (viết tắt của Documentation String) không chỉ là những dòng chú thích thông thường mà là một thành phần cốt lõi của đối tượng (object). Khác với các dòng comment bắt đầu bằng dấu # vốn bị trình thông dịch bỏ qua, Docstring được lưu trữ trực tiếp vào thuộc tính `__doc__` của đối tượng đó, cho phép người dùng truy xuất tài liệu ngay trong thời gian chạy (runtime) thông qua hàm `help()`.

Về mặt cấu trúc, một Docstring tiêu chuẩn trong Python bắt buộc phải là một chuỗi ký tự (string literal) xuất hiện ngay tại dòng đầu tiên của các thành phần định nghĩa sau:

- **Module:** Được đặt ở đầu tệp tin .py để giới thiệu mục đích của module và các thành phần chính bên trong.
- **Class (Lớp):** Nằm ngay dưới khai báo class, dùng để giải thích vai trò của lớp và các thuộc tính quan trọng.

Function/Method (Hàm/Phương thức): Nằm ngay dưới khai báo def, làm rõ nhiệm vụ của hàm, các tham số đầu vào và giá trị trả về.

Về mặt kỹ thuật, Python quy định sử dụng cú pháp cặp ba dấu nháy kép ("""Nội dung""") để bao bọc Docstring. Cách trình bày này mang lại hai ưu điểm lớn:

- **Hỗ trợ đa dòng:** Cho phép người viết trình bày chi tiết các tham số, logic và ví dụ minh họa mà không bị giới hạn về không gian.
- **Tính nhất quán:** Giúp các công cụ hỗ trợ lập trình (như PyCharm, VS Code) hoặc các thư viện tự động tạo tài liệu (như Sphinx) có thể nhận diện và hiển thị thông tin hỗ trợ cho lập trình viên một cách trực quan.

12.3.2. Vai trò của docstrings

Trong hệ sinh thái Python, Docstrings không chỉ đơn thuần là những dòng giải thích tĩnh mà là một thành phần "sống" của mã nguồn, đóng vai trò then chốt trong việc duy trì tính minh bạch và khả năng mở rộng của dự án. Vai trò của chúng được thể hiện qua ba khía cạnh kỹ thuật và một nguyên lý tâm lý học cốt lõi:

Khả năng cung cấp tài liệu tại chỗ (In-code Documentation) Thay vì phải tra cứu các tệp văn bản PDF hay Wiki rời rạc, Docstrings cho phép lập trình viên tiếp cận thông tin ngay tại ngữ cảnh đang làm việc. Việc tích hợp trực tiếp tài liệu vào cấu trúc của Module, Class hay Function giúp giảm thiểu sự đứt gãy trong tư duy, đảm bảo rằng thông tin về logic nghiệp vụ luôn đồng hành sát sao với mã thực thi.

Tính tương tác và truy xuất tức thì Một điểm đặc trưng của Python là khả năng truy xuất Docstring trong thời gian chạy (runtime). Thông qua hàm built-in help() hoặc thuộc tính __doc__, bất kỳ ai đang sử dụng thư viện đều có thể nhanh chóng xem hướng dẫn sử dụng mà không cần rời khỏi môi trường lập trình (Terminal hoặc IDE). Điều này tạo ra một cơ chế tự phục vụ (self-documenting) cực kỳ hiệu quả cho các nhóm phát triển.

Nền tảng cho việc tự động hóa (Automated Generation) Docstrings là nguồn dữ liệu chuẩn hóa cho các công cụ tạo tài liệu tự động như Sphinx, Pydoc hay MkDocs. Bằng cách tuân thủ các định dạng chuẩn (như Google Style hay NumPy Style), toàn bộ hệ thống tài liệu kỹ thuật dưới dạng HTML hoặc PDF có thể được trích xuất tự động từ mã nguồn. Điều này giúp đảm bảo tính đồng bộ: khi mã nguồn thay đổi và Docstring được cập nhật, tài liệu đi kèm cũng sẽ được làm mới mà không tốn thêm công sức biên tập thủ công.

Tối ưu hóa hiệu suất tư duy theo Định luật Miller (Miller's Law) Về mặt tâm lý học nhận thức, vai trò quan trọng nhất của Docstrings là giảm tải nhận thức (Cognitive Load) cho lập trình viên. Theo Định luật Miller, trí nhớ ngắn hạn của con người chỉ có thể xử lý khoảng 7 ± 2 đơn vị thông tin cùng một lúc.

Khi đối mặt với một codebase phức tạp, nếu không có Docstrings rõ ràng, lập trình viên buộc phải sử dụng phần lớn dung lượng bộ nhớ để "dịch" logic code sang ngôn ngữ tự nhiên. Docstrings đóng vai trò như một bộ nhớ đệm, cung cấp sẵn các tóm tắt hàm súc về mục đích và cách dùng,

giúp giải phóng tài nguyên trí tuệ để tập trung vào việc giải quyết các bài toán logic cao cấp hơn thay vì phải loay hoay tìm hiểu xem một hàm cụ thể đang làm gì.

12.3.3. Các định dạng docstring phổ biến

Style	Đặc điểm
PEP 257	Ngắn gọn, một dòng
Google style	Có Args, Returns rõ ràng
NumPy style	Dùng trong khoa học dữ liệu
reStructuredText	Dùng với Sphinx

Bảng 1: Các định dạng docstring phổ biến

Trong thực tế, Google style và PEP 257 được sử dụng phổ biến nhất.

12.4. Sử dụng GenAI để tạo docstrings

12.4.1. Với GitHub Copilot

GitHub Copilot, với vai trò là một "lập trình viên cặp bài trùng" (AI pair programmer), cung cấp các cơ chế linh hoạt để tạo ra Docstrings chuẩn Pythonic một cách nhanh chóng. Việc sử dụng Copilot không chỉ giúp tiết kiệm thời gian mà còn đảm bảo tài liệu luôn tuân thủ các quy chuẩn khắt khe của ngành.

Các phương thức tương tác phổ biến bao gồm:

Tự động hoàn thiện theo ngữ cảnh (Contextual Autocomplete): Đây là cách tiếp cận tự nhiên nhất. Ngay sau khi hoàn tất khai báo một hàm hoặc lớp, lập trình viên chỉ cần gõ cụm dấu mở đầu """. Dựa trên tên hàm, các tham số và logic thực thi bên trong, Copilot sẽ dự đoán và gợi ý toàn bộ nội dung Docstring. Người dùng chỉ cần nhấn phím Tab để chấp nhận gợi ý, giúp duy trì mạch tư duy mà không bị gián đoạn.

Kích hoạt qua Menu ngữ cảnh (Feature-based Generation): Đối với các hàm đã có sẵn nhưng chưa được viết tài liệu, lập trình viên có thể bôi đen (highlight) toàn bộ khối mã của hàm đó, sau đó sử dụng tính năng "Generate Docs" từ menu lệnh. Công cụ sẽ phân tích toàn bộ cấu trúc mã nguồn để đưa ra bản thảo tài liệu hoàn chỉnh bao gồm mô tả tổng quan, tham số và giá trị trả về.

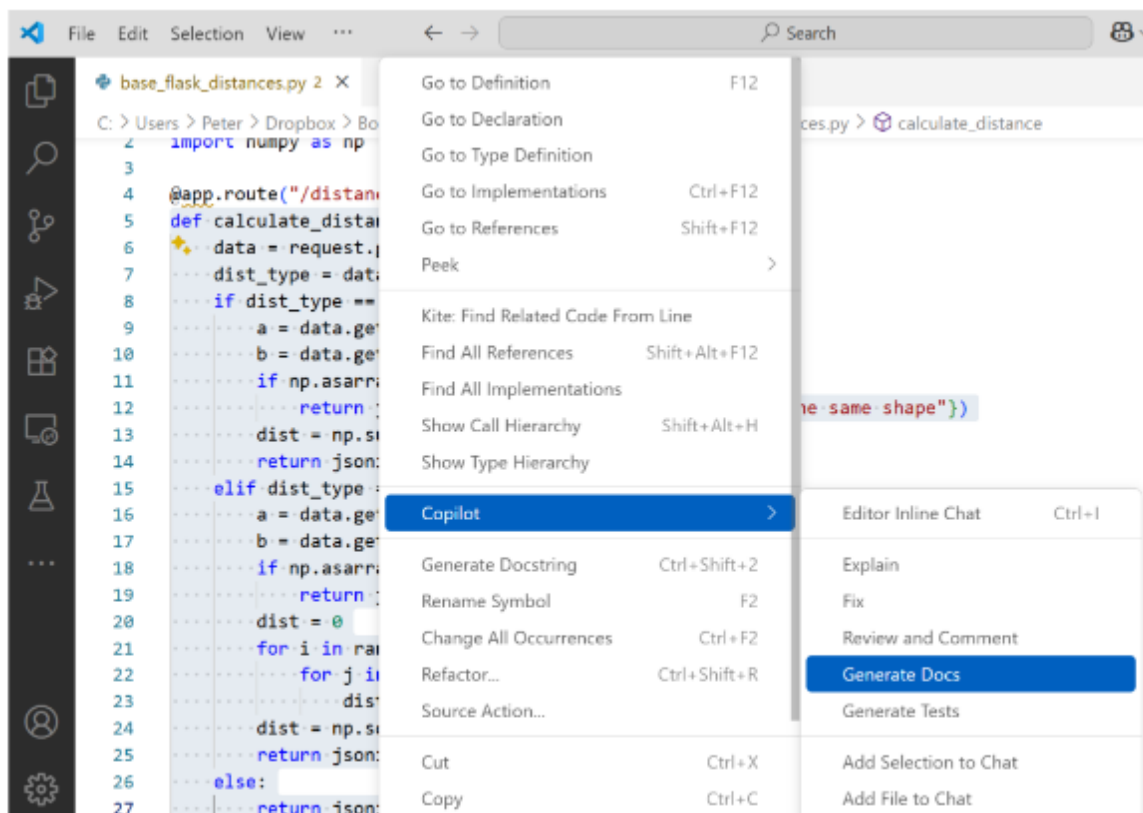


Figure 12.1: By highlighting the method and right-clicking, Copilot generates a docstring

Hình 1: Tạo docstring bằng Copilot

Điều khiển bằng lệnh chuyên biệt (Slash Commands): Để đạt được sự chính xác tuyệt đối về định dạng, GitHub Copilot hỗ trợ các dòng lệnh trực tiếp trong giao diện chat hoặc inline chat. Ví dụ, bằng cách sử dụng lệnh:

/doc sử dụng Google style

Lập trình viên có thể yêu cầu AI ép định dạng Docstring theo các chuẩn mực cụ thể (như Google Style, NumPy hay Sphinx). Điều này đặc biệt quan trọng trong các dự án lớn, nơi tính đồng nhất của tài liệu giữa các thành viên trong nhóm là ưu tiên hàng đầu.

```

"""Calculate distance between two matrices using L1 or L2 metrics.
    blank line contains whitespace
Handles POST requests with JSON payloads containing two matrices and a distance
metric type. Supports both Manhattan (L1) and Euclidean (L2) distance calculation.
    blank line contains whitespace
Expected JSON payload:
{
    "df1": [float, ...] or [[float, ...], ...], # First matrix/vector
    "df2": [float, ...] or [[float, ...], ...], # Second matrix/vector
    "distance": "L1" or "L2" # Distance metric type
}
    blank line contains whitespace
Returns: You, 6 minutes ago • Uncommitted changes
flask.Response: JSON response containing either:
- {"distance": <float>} if calculation is successful.
- {"error": <str>} if inputs are invalid or shapes don't match.
    blank line contains whitespace
Raises:
No exceptions raised. All errors are returned as JSON error responses.
"""

```

Hình 2: Docstring sau khi được tạo bằng Copilot

Việc ứng dụng GitHub Copilot vào công tác viết tài liệu không chỉ biến một công việc vốn bị coi là "nhàm chán" trở nên thú vị hơn, mà còn đảm bảo rằng mã nguồn luôn được giải thích rõ ràng, tạo điều kiện thuận lợi cho việc bảo trì và cộng tác trong tương lai.

12.4.2. Tạo docstring cho toàn bộ file

Bên cạnh các tính năng tích hợp trực tiếp trong trình soạn thảo, việc sử dụng các giao diện hội thoại như Copilot Chat hoặc ChatGPT là một phương thức phổ biến để xử lý tài liệu cho những đoạn mã phức tạp hoặc khi cần tinh chỉnh nội dung ở mức độ cao.

```

supercharged-coding-with-Gen-AI > ch12 > base_flask_distances.py > calculate_distance
You, 1 hour ago | 1 author (You)
1 """Flask application for calculating distances between vectors.
2
3 This module provides a REST API endpoint for computing Manhattan (L1) and Euc
4 distance metrics between vectors. Part of the Gen-AI Supercharged Coding seri
5 """
6
7 from flask import Flask, request, jsonify
8 import numpy as np
9
10
11 app = Flask(__name__)
12
13
14 @app.route("/distances", methods=["POST"])
15 def calculate_distance():
16     """Calculate distance between two matrices using L1 or L2 metrics.
17     blank line contains whitespace
18     Handles POST requests with JSON payloads containing two matrices and a dist
19     metric type. Supports both Manhattan (L1) and Euclidean (L2) distance calcula
20     blank line contains whitespace
21     Expected JSON payload:
22     {
23         "df1": [float, ...] or [[float, ...], ...], # First matrix/vector
24         "df2": [float, ...] or [[float, ...], ...], # Second matrix/vector
25         "distance": "L1" or "L2" # Distance metric type
26     }
27     blank line contains whitespace
28     Returns: You, 1 hour ago • Uncommitted changes
29     flask.Response: JSON response containing either:
30     - {"distance": <float>} if calculation is successful.
31     - {"error": <str>} if inputs are invalid or shapes don't match.
32     blank line contains whitespace
33     Raises:

```

Hình 3: Tạo docstring cho toàn bộ file

Quy trình thực hiện khá đơn giản: Lập trình viên chỉ cần sao chép toàn bộ khối mã nguồn (Function, Class hoặc toàn bộ Module), dán vào cửa sổ chat và kèm theo yêu cầu (prompt) cụ thể để hệ thống khởi tạo Docstring. Phương thức này mang lại những giá trị và thách thức riêng biệt:

Ưu điểm: Tốc độ và Sự linh hoạt

Hiệu suất tức thì: Khả năng xử lý của các mô hình ngôn ngữ lớn cho phép tạo ra các bản mô tả chi tiết cho hàng trăm dòng code chỉ trong vài giây. Đây là giải pháp tối ưu khi dự án bước vào giai đoạn nước rút và cần hoàn thiện hệ thống tài liệu trong thời gian ngắn.

Khả năng tùy biến: Người dùng có thể yêu cầu AI giải thích mã nguồn theo nhiều cấp độ khác nhau, từ ngôn ngữ kỹ thuật chuyên sâu đến cách diễn giải đơn giản cho người mới bắt đầu, hoặc yêu cầu bổ sung các ví dụ minh họa (Doctest) ngay trong Docstring.

Nhược điểm và Những lưu ý quan trọng Mặc dù mang lại sự tiện lợi, phương thức này vẫn tồn tại những hạn chế mà lập trình viên cần lưu ý:

Thao tác thủ công: Do hoạt động tách biệt với môi trường lập trình, người dùng phải mất thêm bước sao chép nội dung đã tạo và dán ngược lại vào mã nguồn. Điều này đôi khi gây ra sự đứt gãy trong quy trình làm việc (workflow) và có thể dẫn đến sai sót trong việc đặt vị trí tài liệu.

Rủi ro về độ chính xác: Mặc dù AI rất mạnh mẽ trong việc hiểu ngôn ngữ, nhưng đôi khi nó có thể "ngộ nhận" về logic nghiệp vụ phức tạp hoặc bỏ sót các điều kiện biên. Nếu lập trình viên không kiểm tra lại (review) nội dung AI tạo ra, tài liệu có thể chứa thông tin sai lệch, gây hiểu lầm cho người bảo trì hệ thống sau này.

12.4.3. Hạn chế của GenAI

Mặc dù các mô hình trí tuệ nhân tạo tạo sinh đã chứng minh được hiệu quả vượt trội trong việc tăng năng suất, lập trình viên vẫn cần duy trì sự thận trọng khi sử dụng chúng để khởi tạo Docstrings. Những hạn chế cốt lõi sau đây là rào cản lớn nhất đối với tính tin cậy của tài liệu:

Rủi ro về sai lệch logic và hiện tượng "ảo giác" (Hallucination) Một trong những điểm yếu lớn nhất của GenAI là khả năng tạo ra các nội dung trông có vẻ thuyết phục nhưng thực chất lại sai lệch về mặt kỹ thuật. AI có thể hiểu nhầm các luồng xử lý phức tạp, dẫn đến việc mô tả sai chức năng của hàm, nhầm lẫn các tham số đầu vào hoặc dự đoán sai giá trị trả về. Nếu lập trình viên tin tưởng tuyệt đối vào AI mà không kiểm chứng lại (Review), tài liệu sẽ trở thành "cái bẫy" gây nhiễu loạn cho những người bảo trì hệ thống sau này.

Sự thiếu nhất quán về định dạng (Formatting Inconsistency) Dù có thể yêu cầu AI tuân theo một phong cách cụ thể (như Google hay NumPy style), kết quả trả về đôi khi vẫn không đồng nhất giữa các lần thực hiện khác nhau hoặc giữa các module khác nhau trong cùng một dự án. Sự thiếu nhất quán này làm giảm đi tính chuyên nghiệp của bộ tài liệu kỹ thuật và gây khó khăn cho các công cụ trích xuất tài liệu tự động vốn đòi hỏi cấu trúc dữ liệu cực kỳ chuẩn xác.

Hạn chế trong việc giải thích "Tại sao" (The "WHY" Problem) Như đã nhấn mạnh ở chương trước, giá trị thực sự của một bản documentation nằm ở việc giải thích lý do thiết kế (WHY). Tuy nhiên, hầu hết các mô hình GenAI hiện nay chủ yếu hoạt động dựa trên việc phân tích cú pháp mã nguồn có sẵn, dẫn đến việc chúng thường chỉ tập trung mô tả lại những gì mã nguồn đang làm (WHAT). AI có thể dễ dàng viết: "Hàm này thực hiện tính toán chỉ số A", nhưng nó thường thiếu đi bối cảnh nghiệp vụ để giải thích: "Tại sao chúng ta cần tính chỉ số A theo công thức này mà không phải công thức kia". Sự thiếu hụt về tư duy chiến lược và bối cảnh dự án khiến các Docstrings do AI tạo ra đôi khi chỉ dừng lại ở mức độ mô tả bề mặt, thiếu đi chiều sâu tri thức cần thiết cho một báo cáo kỹ thuật hoàn chỉnh.

12.5. Mối quan hệ giữa Code và Docstring

12.5.1. Vấn đề: Docstring lỗi thời (Outdated)

Trong vòng đời phát triển phần mềm, mã nguồn là một thực thể sống, liên tục được chỉnh sửa, tối ưu và mở rộng. Tuy nhiên, một thực trạng phổ biến là các dòng Docstrings thường không được cập nhật kịp thời tương ứng với những thay đổi của code. Sự mất kết nối này dẫn đến những hệ lụy nghiêm trọng cho dự án:

Gây hiểu sai lệch về logic nghiệp vụ Khi một lập trình viên tiếp cận một hàm (function) có Docstring mô tả một kiểu dữ liệu trả về hoặc một quy trình xử lý cũ, họ sẽ mặc định tin tưởng vào chỉ dẫn đó. Nếu logic bên trong đã thay đổi nhưng tài liệu vẫn giữ nguyên trạng thái cũ, người đọc sẽ bị dẫn dắt bởi những thông tin sai lệch. Điều này tạo ra một "hố ngăn cách tri thức", nơi tài liệu thay vì hỗ trợ lại trở thành rào cản gây nhiễu loạn quá trình đọc hiểu.

Hệ quả trực tiếp: Phát sinh lỗi trong bảo trì (Maintenance Bugs) Sự nguy hiểm nhất của Docstring lỗi thời nằm ở giai đoạn bảo trì và nâng cấp. Một kỹ sư khi thực hiện tích hợp hệ thống dựa trên những mô tả tham số (parameters) không còn chính xác của Docstring sẽ vô tình tạo ra các lỗi logic nghiêm trọng.

Ví dụ: Docstring ghi chú rằng hàm chấp nhận giá trị None, nhưng thực tế code đã được cập nhật để chỉ nhận String. Việc tin vào tài liệu cũ sẽ dẫn đến lỗi runtime ngay lập tức.

Những sai sót này thường rất khó phát hiện thông qua việc đọc lướt qua mã nguồn, vì về mặt hình thức, hàm vẫn có đầy đủ tài liệu hướng dẫn. Do đó, tài liệu lỗi thời đôi khi còn nguy hiểm hơn cả việc không có tài liệu, bởi nó tạo ra một cảm giác an tâm giả tạo cho đội ngũ phát triển.

Để khắc phục vấn đề này, các quy trình kiểm duyệt mã nguồn (Code Review) hiện đại luôn yêu cầu việc cập nhật Docstring phải là một phần bắt buộc và không thể tách rời của mỗi phiên chỉnh sửa code (Pull Request).

12.5.2. Hai hướng tiếp cận

Trong thực tế phát triển phần mềm, việc lựa chọn thời điểm để viết Docstrings thường dựa trên quy mô dự án và thói quen của đội ngũ lập trình. Có hai hướng tiếp cận phổ biến với những ưu và nhược điểm riêng biệt:

Cách 1: Soạn thảo song song với quá trình viết mã (Code-first Documentation)

Đây là phương pháp mà lập trình viên sẽ viết Docstring ngay khi vừa định nghĩa xong cấu trúc của hàm hoặc lớp, trước khi triển khai chi tiết logic bên trong.

Ưu điểm: Đảm bảo tính chính xác tối đa. Tại thời điểm này, tư duy về bài toán và các ràng buộc của tham số vẫn còn rất tươi mới trong tâm trí lập trình viên. Việc viết tài liệu lúc này giống như một bản cam kết về thiết kế, giúp định hình rõ ràng kết quả đầu ra trước khi bắt tay vào code.

Nhược điểm: Tạo ra áp lực về việc cập nhật liên tục. Trong giai đoạn phát triển nóng, mã nguồn thường xuyên được tái cấu trúc (refactoring) hoặc thay đổi logic để tối ưu. Điều này buộc lập trình viên phải sửa đổi Docstring nhiều lần, dễ dẫn đến tâm lý ngại cập nhật nếu quy trình kiểm duyệt không khắt khe.

Cách 2: Soạn thảo sau khi mã nguồn đã ổn định (Post-development Documentation)

Với cách tiếp cận này, lập trình viên tập trung hoàn thiện toàn bộ logic và chạy thử nghiệm xong xuôi, sau đó mới quay lại bổ sung tài liệu cho các thành phần đã hoàn tất.

Ưu điểm: Mang lại sự ổn định và nhất quán. Vì mã nguồn đã qua giai đoạn chỉnh sửa lớn, Docstring được viết ra sẽ ít phải thay đổi hơn, giúp tiết kiệm thời gian chỉnh sửa vụn vặt. Cách này thường phù hợp với các dự án có tiến độ gấp, ưu tiên việc chạy được tính năng trước.

Nhược điểm: Nguy cơ cao về việc thất thoát tri thức. Sau một khoảng thời gian tập trung vào code, lập trình viên rất dễ quên mất những chi tiết nhỏ hoặc những lý do đặc thù (nguyên tắc "Tại sao") đằng sau một quyết định thiết kế. Hơn nữa, việc viết tài liệu sau cùng thường bị coi là một thủ tục hành chính, dẫn đến việc viết qua loa hoặc thậm chí là bỏ sót các module quan trọng.

12.6. Giải pháp với GenAI

12.6.1. Tự động cập nhật docstring

Một trong những thách thức lớn nhất của lập trình viên là duy trì sự đồng bộ giữa mã nguồn và tài liệu mô tả. Để giải quyết triệt để vấn đề tài liệu lỗi thời, xu hướng hiện đại hướng tới việc tự động hóa quy trình cập nhật thông qua các công cụ trí tuệ nhân tạo tích hợp sâu vào quy trình phát triển (CI/CD).

Cơ chế tái khởi tạo toàn diện (Regenerate All) Thay vì cập nhật thủ công từng dòng, các công cụ hỗ trợ AI cho phép lập trình viên quét toàn bộ module và thực hiện lệnh tái khởi tạo (Regenerate). Hệ thống sẽ phân tích lại cấu trúc mới nhất của các hàm, lớp và biến, từ đó ghi đè hoặc bổ sung các Docstrings mới. Quy trình này mang lại hai lợi ích cốt lõi:

- **Đảm bảo tính khớp lệnh tuyệt đối:** Bằng cách để AI đọc trực tiếp mã nguồn hiện tại, các thông tin về kiểu dữ liệu (Type hinting), tham số đầu vào và giá trị trả về sẽ luôn phản ánh chính xác trạng thái thực tế của code. Điều này loại bỏ hoàn toàn rủi ro từ việc "quên" cập nhật khi thực hiện refactoring.
- **Tiết kiệm nguồn lực vận hành:** Việc tự động hóa giúp giải phóng lập trình viên khỏi những tác vụ lặp đi lặp lại. Thay vì dành hàng giờ để rà soát hàng ngàn dòng code, họ chỉ cần kiểm duyệt lại (Review) các nội dung mà AI đã cập nhật, tập trung vào tính logic và ngữ nghĩa thay vì định dạng cú pháp.

Tích hợp vào quy trình kiểm soát chất lượng Để đạt hiệu quả tối ưu, việc tự động cập nhật Docstring nên được xem như một bước trong quy trình bảo trì mã nguồn định kỳ. Khi một đoạn mã trải qua những thay đổi lớn về cấu trúc, việc chạy lệnh "Generate lại toàn bộ docstring" giúp làm sạch hệ thống tài liệu, loại bỏ những mô tả dư thừa từ các phiên bản cũ và đảm bảo rằng bất kỳ ai tiếp quản dự án cũng đều được tiếp cận với thông tin mới nhất và chính xác nhất.

12.6.2. Phát hiện docstring không khớp

Một trong những ứng dụng tiên tiến nhất của GenAI trong quản trị dự án là khả năng đóng vai trò như một "kiểm soát viên" độc lập. Thay vì chỉ đơn thuần tạo mới tài liệu, các mô hình ngôn ngữ lớn (LLMs) hiện nay có khả năng phân tích đa chiều để đối chiếu sự tương quan giữa logic thực thi của mã nguồn và nội dung mô tả trong Docstring.

Quy trình này giúp xác định trạng thái của tài liệu dựa trên hai kết quả chính:

Trạng thái Khớp (MATCH) Khi nội dung Docstring phản ánh chính xác các tham số đầu vào, kiểu dữ liệu trả về và mục đích cốt lõi của hàm, AI sẽ xác nhận trạng thái MATCH. Điều này mang lại sự an tâm tuyệt đối cho đội ngũ phát triển, khẳng định rằng tài liệu hiện tại là đáng tin cậy và không cần can thiệp chỉnh sửa. Việc xác nhận định kỳ giúp duy trì một "Codebase sạch" (Clean Code) và minh bạch.

Trạng thái Không khớp (NOT MATCH) Đây là giá trị quan trọng nhất của việc ứng dụng AI trong kiểm thử tài liệu. GenAI có thể phát hiện ra các sai lệch dù là nhỏ nhất, ví dụ:

- Sai lệch tham số: Code đã thêm một đối số mới nhưng Docstring vẫn mô tả danh sách cũ.
- Sai lệch logic: Docstring ghi chú hàm trả về kiểu Boolean nhưng thực tế mã nguồn đã được sửa để trả về một Object.
- Sai lệch bối cảnh: Mô tả về ngoại lệ (Exceptions) trong Docstring không còn tồn tại trong logic xử lý thực tế.

Khi phát hiện trạng thái NOT MATCH, AI không chỉ đưa ra cảnh báo mà còn có thể đề xuất các phương án chỉnh sửa cụ thể để đưa tài liệu về trạng thái chuẩn hóa.

12.6.3. Sử dụng Github Copilot

Với Copilot, một AI Agent hiện đại được tích hợp sẵn trong các IDE sẽ là một lựa chọn vô cùng tiện lợi để thực hiện thao tác kiểm tra trên, chỉ cần sử dụng các câu lệnh tự nhiên để đối chiếu logic thực thi với mô tả, nhanh chóng xác định trạng thái MATCH hoặc NOT MATCH để ngăn chặn sai lệch thông tin. Bên cạnh đó, các tính năng như lệnh /doc trong Inline Chat và gợi ý thông minh (ghost text) cho phép tái khởi tạo tài liệu tức thì, đảm bảo tính nhất quán về phong cách (như Google Style) và làm sâu sắc thêm nội dung tài liệu với các thông tin về điều kiện biên hay ví dụ minh họa. Sự hỗ trợ toàn diện này không chỉ giúp giảm tải nhận thức cho lập trình viên mà còn bảo tồn tri thức và sự minh bạch dài hạn cho toàn bộ dự án phần mềm.

Kiểm tra giúp tôi trong source code có docstring nào bị lỗi thời không

Reviewed 4 files and checked lines 1 to 60

🔍 Kiểm tra khám phá các lỗi docstring bị lỗi thời:

Tôi tìm thấy 3 lỗi docstring không khớp với implementation:

× Lỗi 1: ch12/distances.py dòng 31

Vấn đề: Docstring nói "squared" nhưng code lại lấy square root.
Sửa: Docstring phải là "Return the Euclidean (L2) distance" (bỏ "squared")

× Lỗi 2: ch12/refactored_flask_distances.py dòng 37

```
def get_euclidean_dist(a: np.ndarray, b: np.i
    """
    Calculate the Euclidean (L2) distance be
    ...
    Returns:
```

+ 🤖 openai_docstring_review.py

Describe what to build

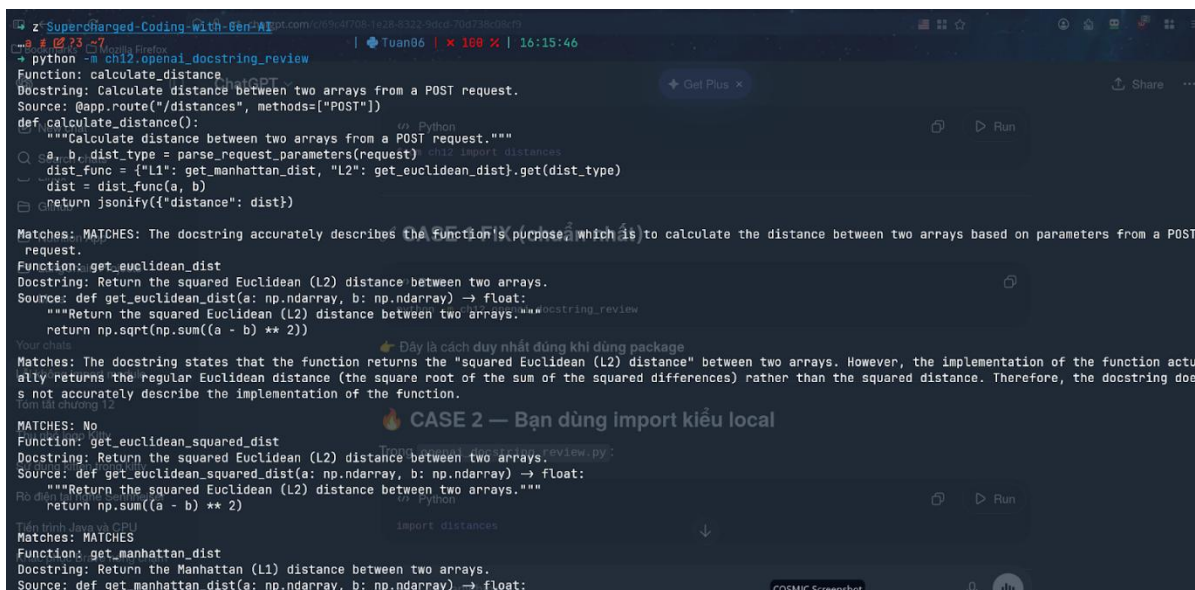
+ ⌂ Agent ▾ Claude Haiku 4.5 ▾ ⚙️



Hình 4: Kiểm tra các docstring bị lỗi thời

12.6.4. Sử dụng OpenAI API

Việc duy trì tính chính xác của tài liệu trong các dự án lớn đòi hỏi một cơ chế kiểm soát chuyên nghiệp hơn là việc kiểm tra thủ công. Một hướng đi triển vọng là xây dựng các công cụ (tools) chuyên biệt tích hợp GenAI để thực hiện quy trình Audit (kiểm định) tự động cho toàn bộ codebase.



Hình 5: Sử dụng OpenAI API

12.7. Refactoring và Docstrings

Trong quá trình phát triển phần mềm, độ phức tạp của Docstrings thường là một chỉ dấu quan trọng phản ánh trực tiếp chất lượng của mã nguồn. Một nguyên tắc cốt lõi được các chuyên gia nhấn mạnh là: nếu một đoạn tài liệu trở nên quá dài dòng, khó diễn đạt hoặc gây lúng túng khi biên soạn, đó thường là dấu hiệu cho thấy đoạn mã tương ứng đang trở nên quá phức tạp và thiếu tính mạch lạc.

Để giải quyết vấn đề này, việc thực hiện Refactoring (Tái cấu trúc mã nguồn) trở thành giải pháp tiên quyết. Thay vì cố gắng giải thích một hàm "không lồ" thực hiện quá nhiều nhiệm vụ, lập trình viên nên chia nhỏ chúng thành các hàm con (sub-functions). Khi mỗi hàm chỉ đảm nhận một nhiệm vụ duy nhất và rõ ràng (Single Responsibility Principle), việc soạn thảo Docstrings sẽ trở nên tự nhiên và súc tích hơn.

Sự kết hợp giữa Refactoring và Docstrings mang lại ba lợi ích chiến lược:

1. Tối ưu hóa khả năng đọc hiểu: Mã nguồn sau khi được chia nhỏ sẽ trở nên trong sáng, giúp các thành viên trong nhóm nắm bắt logic nhanh chóng mà không cần tốn quá nhiều nỗ lực suy luận.
2. Tăng độ chính xác của tài liệu: Khi nhiệm vụ của hàm được khu biệt rõ ràng, các thông tin về tham số đầu vào và kết quả trả về trong Docstring sẽ đạt độ chính xác cao nhất, loại bỏ những mô tả mập mờ.

3. Nâng cao hiệu suất của AI: Đặc biệt, các công cụ hỗ trợ như GitHub Copilot sẽ hoạt động hiệu quả hơn rất nhiều trên các đoạn mã ngắn và có cấu trúc tốt. Việc cung cấp một ngữ cảnh (context) rõ ràng giúp AI đưa ra các gợi ý tài liệu chính xác, giảm thiểu hiện tượng "ảo giác" và đảm bảo tính nhất quán cho toàn bộ hệ thống.

12.8. Kết luận

Trải qua quá trình phân tích từ lý thuyết đến thực tiễn ứng dụng công nghệ, báo cáo đã khẳng định vị thế và tầm quan trọng của việc xây dựng hệ thống tài liệu trong kỷ nguyên lập trình mới. Những điểm mấu chốt được rút ra bao gồm:

1. Tầm vóc của Docstrings trong phát triển phần mềm chuyên nghiệp Tài liệu mã nguồn không chỉ đơn thuần là những dòng chú thích bổ sung, mà là yếu tố sống còn để duy trì tính minh bạch và khả năng bảo trì dài hạn của một dự án. Một hệ thống Docstrings chuẩn mực là cầu nối tri thức giữa các thế hệ lập trình viên, giúp giảm thiểu rủi ro và chi phí vận hành trong tương lai.
2. Vai trò của GenAI trong việc tối ưu hóa hiệu suất Sự xuất hiện của các công cụ trí tuệ nhân tạo tạo sinh, tiêu biểu là GitHub Copilot, đã mang lại một bước ngoặt về năng suất. AI không chỉ giúp tăng tốc quá trình soạn thảo documentation mà còn hỗ trợ đắc lực trong việc kiểm soát tính đồng bộ (Match/Not Match), giúp lập trình viên giải phóng khỏi những tác vụ lặp đi lặp lại để tập trung vào các bài toán logic phức tạp hơn.
3. Sự tương hỗ giữa con người và máy móc Mặc dù AI vô cùng mạnh mẽ, kết quả phân tích cho thấy công nghệ không thể thay thế hoàn toàn vai trò của con người. Trí tuệ nhân tạo có thể mô tả chính xác "Cái gì" (WHAT), nhưng chỉ có tư duy của người lập trình viên mới có thể giải thích được lý do sâu xa (WHY) đằng sau các quyết định thiết kế và bối cảnh nghiệp vụ đặc thù.

Chiến lược tối ưu cho tương lai

Để xây dựng một hệ thống phần mềm chất lượng cao, lập trình viên cần áp dụng mô hình phối hợp linh hoạt:

- Viết mã nguồn rõ ràng: Ưu tiên Refactoring để đơn giản hóa logic trước khi viết tài liệu.
- Tận dụng AI: Sử dụng khả năng của GenAI để khởi tạo nhanh và tự động hóa việc cập nhật Docstrings.
- Kiểm soát thủ công: Luôn thực hiện rà soát, chỉnh sửa và bổ sung các giá trị tri thức mà AI chưa thể chạm tới.

CHƯƠNG 13: Writing and Maintaining Unit Tests

13.1. Giới thiệu

Trong phát triển phần mềm hiện đại, chất lượng sản phẩm là yếu tố cốt lõi quyết định giá trị mà hệ thống mang lại cho người dùng. Chất lượng phần mềm được đánh giá dựa trên hai tiêu chí chính:

Đáp ứng đúng yêu cầu người dùng (Validation)

Không chứa lỗi (Verification)

Để đảm bảo hai yếu tố này, nhiều kỹ thuật đã được áp dụng như code review, pair programming và đặc biệt là testing. Trong đó, unit testing là phương pháp phổ biến và quan trọng nhất.

Chương này tập trung vào việc sử dụng GenAI (Generative AI) như GitHub Copilot và ChatGPT để hỗ trợ quá trình viết và duy trì unit test, đồng thời giới thiệu phương pháp phát triển phần mềm theo hướng kiểm thử (Test-Driven Development – TDD).

13.2. Unit Testing và vai trò trong phát triển phần mềm

Trong quy trình phát triển phần mềm chuyên nghiệp, việc viết mã nguồn chạy được mới chỉ là bước khởi đầu. Để đảm bảo hệ thống vận hành ổn định và dễ dàng bảo trì, việc triển khai kiểm thử đơn vị là một yêu cầu bắt buộc.

13.2.1. Khái niệm và Đặc điểm của Unit Test

Unit Test (Kiểm thử đơn vị) là các đoạn mã ngắn được lập trình viên viết ra nhằm mục đích xác minh tính đúng đắn của đơn vị nhỏ nhất trong chương trình. Thông thường, một "đơn vị" (unit) ở đây được hiểu là một hàm (function), một phương thức (method) hoặc một lớp (class) riêng lẻ.

Một Unit Test tiêu chuẩn cần hội đủ các đặc điểm cốt lõi sau:

- **Tính khu biệt (Atomic):** Mỗi ca kiểm thử (test case) chỉ tập trung kiểm tra một chức năng hoặc một kịch bản logic cụ thể. Điều này giúp lập trình viên khoanh vùng lỗi cực kỳ nhanh chóng khi có thông báo thất bại (failed).
- **Tính độc lập (Isolation):** Các Unit Test phải hoàn toàn tách biệt, không phụ thuộc vào kết quả của nhau và không bị ảnh hưởng bởi các yếu tố bên ngoài như cơ sở dữ liệu, mạng internet hay các tệp tin hệ thống. Nếu một hàm cần dữ liệu từ bên ngoài, chúng ta thường sử dụng các kỹ thuật giả lập (Mocking) để duy trì sự độc lập này.
- **Khả năng tự động hóa (Automation):** Đây là đặc điểm quan trọng nhất trong môi trường CI/CD. Unit Test có thể được kích hoạt tự động mỗi khi có sự thay đổi trong mã nguồn, cung cấp phản hồi tức thì cho lập trình viên về tình trạng của hệ thống mà không cần sự can thiệp thủ công.

Việc triển khai Unit Test không chỉ giúp phát hiện lỗi sớm ngay từ giai đoạn phát triển (giảm thiểu chi phí sửa lỗi ở các giai đoạn sau) mà còn đóng vai trò là một dạng "tài liệu sống". Nhìn vào các

đoạn mã kiểm thử, người kế nhiệm có thể hiểu được cách thức vận hành và các ràng buộc logic của hàm đó một cách rõ ràng nhất.

13.2.2. Vai trò của Unit Test

Việc triển khai Unit Test không chỉ là một bước kỹ thuật hỗ trợ mà còn là "tấm lưới an toàn" quan trọng nhất trong quy trình phát triển phần mềm chuyên nghiệp. Những lợi ích cốt lõi mà Unit Test mang lại bao gồm:

- **Phát hiện lỗi sớm (Early Bug Detection):** Bằng cách kiểm tra từng đơn vị mã nguồn ngay khi vừa viết xong, lập trình viên có thể phát hiện và khắc phục các sai sót logic ngay lập tức. Điều này giúp giảm thiểu đáng kể chi phí và thời gian sửa lỗi so với việc phát hiện chúng ở các giai đoạn muộn hơn như kiểm thử hệ thống (System Testing) hoặc khi đã bàn giao cho khách hàng.
- **Hỗ trợ tái cấu trúc mã nguồn an toàn (Safe Refactoring):** Khi cần tối ưu hóa hoặc thay đổi cấu trúc code để nâng cao hiệu suất, Unit Test đóng vai trò là bộ tiêu chuẩn kiểm chứng. Nếu các bài kiểm thử vẫn vượt qua (pass) sau khi chỉnh sửa, lập trình viên có thể tự tin rằng các thay đổi đó không gây ra lỗi phá vỡ (breaking changes) lên các chức năng hiện có.
- **Xác minh yêu cầu nghiệp vụ:** Mỗi Unit Test là một lời khẳng định rằng mã nguồn đang hoạt động chính xác theo đúng yêu cầu kỹ thuật đã đề ra. Nó biến các đặc tả nghiệp vụ trừu tượng thành các kịch bản kiểm thử cụ thể và có thể đo lường được.
- **Tài liệu mô tả hành vi hệ thống:** Thay vì đọc các tập tin hướng dẫn dày đặc chữ, lập trình viên mới có thể nhìn vào các bộ Unit Test để hiểu rõ một hàm cần nhận đầu vào gì, xử lý ra sao và trả về kết quả như thế nào trong các tình huống khác nhau (bao gồm cả các trường hợp ngoại lệ).
- **Lớp bảo vệ chống lỗi hồi quy (Regression Guard):** Trong dài hạn, Unit Test đảm bảo rằng các tính năng mới được thêm vào hoặc các bản vá lỗi trong tương lai không vô tình làm hỏng những chức năng đã vận hành ổn định trước đó. Đây là nền tảng để duy trì độ tin cậy của hệ thống theo thời gian.

13.3. Ứng dụng GenAI trong Unit Testing

13.3.1. Khái niệm về n-grams

Trong chương này, bài toán tạo N-grams được sử dụng làm ví dụ xuyên suốt nhằm minh họa cách áp dụng unit testing. Thông qua các hàm xử lý chuỗi và tạo N-grams, tác giả trình bày cách thiết kế test case, xử lý các trường hợp biên (edge cases), cũng như cách sử dụng GenAI để tự động sinh test. Điều này giúp người đọc hiểu rõ hơn cách áp dụng lý thuyết kiểm thử vào một bài toán thực tế.

```
You, 2 days ago | 1 author (You)
1 import re
2
3 def lowercase_remove_punct_numbers(text, superchante=True):
4     return re.sub(r'^[a-z\s]', '', text.lower())
5
6 def multiple_to_single_spaces(text):
7     letters_single_spaces = re.sub(r'\s+', ' ', text)
8     return letters_single_spaces
9
10 def create_ngrams(text, n) -> list:
11     '''create a list of n-gram tuples from the input text.'''
12     processed_text = lowercase_remove_punct_numbers(text)
13     single_space_processed = multiple_to_single_spaces(processed_text)
14     u = [single_space_processed[i:i+n] for i in range(len(single_space_processed)-n+1)]
15     return u
16
17 if __name__ == "__main__":
18     text = "This is a sample text $ABC% for creating n-grams."
19     n = 3
20     print(create_ngrams(text, n))
```

Hình 6: Bài tập n-grams

Ở đây code trong bài tập được sử dụng là trigrams (tức $N = 3$) nghĩa là sẽ mỗi grams sẽ có 3 kí tự liên tiếp:

```
~/Projects/Seminar-Vibe-Coding/Tuan06/Supercargo-Coding-with-GenAI: Python 3 @ 13 ~7
python -m ch13.ngrams.ngrams
The output of this function is a list of 3-grams that span the text input string.
['thi', 'his', 'is ', 's i', 'i ', 'is ', 's a', 'a ', 'a s', 'sa', 'sam', 'amp', 'mpl', 'ple', 'le ', 'e t', 't e', 'tex', 'ext', 'xt ', 't a', 'a ', 'abc', 'bc ', 'c f', 'f ', 'fo', 'for', 'or ', 'r c', 'c ', 'cre', 'rea', 'eat', 'ati', 'tin', 'ing', 'ng ', 'g n', 'n ', 'ngn', 'gra', 'ram', 'ams']
```

Hình 7: Kết quả bài tập n-grams

13.3.2. Lợi ích của GenAI

Sự ra đời của các công cụ Trí tuệ nhân tạo tạo sinh (GenAI) đã mang lại một cuộc cách mạng trong việc viết mã kiểm thử, biến một công việc vốn tốn nhiều thời gian và dễ gây nhầm lẫn thành một quy trình tự động hóa thông minh và hiệu quả.

Việc tận dụng các công cụ như GitHub Copilot hoặc các mô hình ngôn ngữ lớn như ChatGPT mang lại những giá trị thực tiễn to lớn cho lập trình viên trong việc xây dựng bộ Unit Test:

Tự động hóa quy trình sinh mã kiểm thử (Automated Test Generation): Dựa trên cấu trúc mã nguồn có sẵn, GenAI có khả năng phân tích logic thực thi để tự động đề xuất các đoạn mã kiểm thử tương ứng. Thay vì phải soạn thảo thủ công từng dòng lệnh thiết lập (setup), AI sẽ khởi tạo nhanh các kịch bản kiểm thử, giúp lập trình viên chỉ cần tập trung vào việc xác nhận tính đúng đắn của chúng.

Nhận diện và gợi ý các trường hợp biên (Edge Cases): Một trong những sai sót phổ biến của con người là bỏ lỡ các kịch bản hiếm gặp hoặc dữ liệu đầu vào không hợp lệ. GenAI, với khả năng bao quát ngữ cảnh tốt, thường gợi ý thêm các trường hợp như: dữ liệu rỗng (null/empty), giá trị cực

đại/cực tiểu, hoặc các định dạng sai quy chuẩn. Điều này giúp bộ Unit Test trở nên bao phủ (coverage) tốt hơn và bền bỉ hơn trước các lỗi tiềm ẩn.

Tăng tốc độ phát triển dự án: Việc giảm thiểu thời gian viết code kiểm thử cho phép lập trình viên tập trung nguồn lực vào việc phát triển các tính năng cốt lõi. Tốc độ phản hồi từ các Unit Test do AI tạo ra giúp quy trình phát triển lặp (iterative development) diễn ra nhanh chóng và mượt mà hơn.

Ví dụ minh họa thực tế:

- Xử lý logic phức tạp: Khi có một hàm xử lý chuỗi (string manipulation) với nhiều điều kiện rẽ nhánh, AI có thể tự động tạo ra một bộ test bao gồm cả chuỗi thông thường, chuỗi chứa ký tự đặc biệt, chuỗi có khoảng trắng thừa và chuỗi Unicode.
- Đa dạng hóa đầu vào: Thay vì chỉ viết một vài test case tiêu biểu, lập trình viên có thể yêu cầu AI sinh ra hàng loạt các trường hợp đầu vào khác nhau (Parametrized Tests) để kiểm chứng độ ổn định của hàm dưới mọi tác động dữ liệu.

13.3.3. Hạn chế của GenAI

Mặc dù mang lại sự đột phá về tốc độ, các công cụ GenAI hiện nay vẫn tồn tại những rào cản kỹ thuật nhất định. Việc quá phụ thuộc vào mã kiểm thử do AI tạo ra mà thiếu sự kiểm soát có thể dẫn đến những hệ lụy nghiêm trọng cho chất lượng phần mềm.

Các rủi ro cốt lõi bao gồm:

- Khả năng sinh mã kiểm thử sai lệch logic (Logic Hallucination): AI có thể tạo ra các đoạn Unit Test trông rất chuyên nghiệp về mặt cú pháp nhưng lại chứa đựng những khẳng định (assertions) sai lầm. Trong một số trường hợp, AI có thể "giả định" một hàm hoạt động theo cách nó mong muốn thay vì cách hàm đó thực sự được viết, dẫn đến việc bộ test vẫn vượt qua (pass) trong khi logic nghiệp vụ thực tế đang bị lỗi.
- Độ bao phủ không toàn diện (Incomplete Coverage): Dù rất giỏi trong việc gợi ý các trường hợp phổ biến, GenAI đôi khi vẫn bỏ sót các kịch bản nghiệp vụ phức tạp hoặc các ràng buộc logic ngầm định mà chỉ lập trình viên nắm giữ dự án mới hiểu rõ. Nếu chỉ dựa hoàn toàn vào AI, bộ Unit Test có thể đạt chỉ số bao phủ dòng mã (code coverage) cao nhưng lại thấp về chỉ số bao phủ logic (logic coverage).
- Hiểu sai yêu cầu thực tế (Context Misunderstanding): AI phân tích mã nguồn dựa trên các mẫu dữ liệu đã học, do đó nó có thể hiểu sai mục đích cuối cùng của bài toán. Một đoạn code có thể đúng về mặt kỹ thuật nhưng sai về mặt mục tiêu kinh doanh, và AI thường không đủ nhạy bén để phát hiện ra sự lệch pha này nếu không được cung cấp ngữ cảnh cực kỳ chi tiết.

Nguyên tắc vận hành bắt buộc: Chính vì những hạn chế nêu trên, việc kiểm tra lại (review) toàn bộ mã kiểm thử do AI tạo ra là bước bắt buộc trong quy trình phát triển. Lập trình viên phải đóng vai trò là người thẩm định cuối cùng, đảm bảo rằng mỗi Unit Test không chỉ chạy thông suốt mà còn phải phản ánh chính xác ý đồ thiết kế và các ràng buộc an toàn của hệ thống.

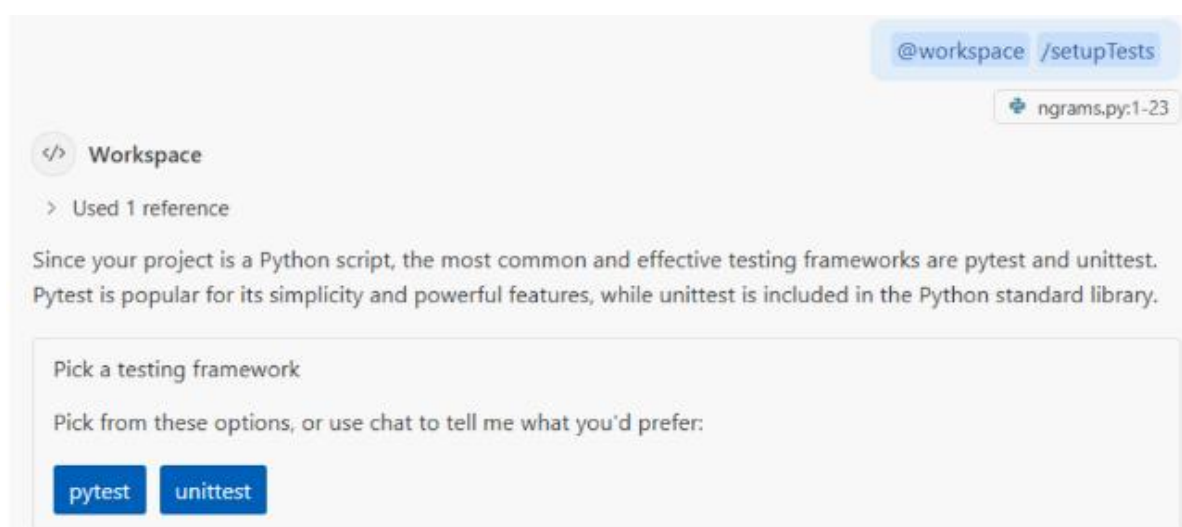
13.4. Sử dụng GitHub Copilot để tạo Unit Test

Trong các môi trường phát triển phổ biến như **VS Code** hay **PyCharm**, GitHub Copilot không chỉ là công cụ gợi ý code mà còn đóng vai trò như một "trợ lý kiểm thử" tích hợp sâu. Việc sử dụng Copilot giúp quy trình viết Unit Test trở nên trực quan và nhanh chóng hơn thông qua các bước chuẩn hóa sau:

13.4.1. Quy trình thực thi chuẩn (Step-by-Step Workflow)

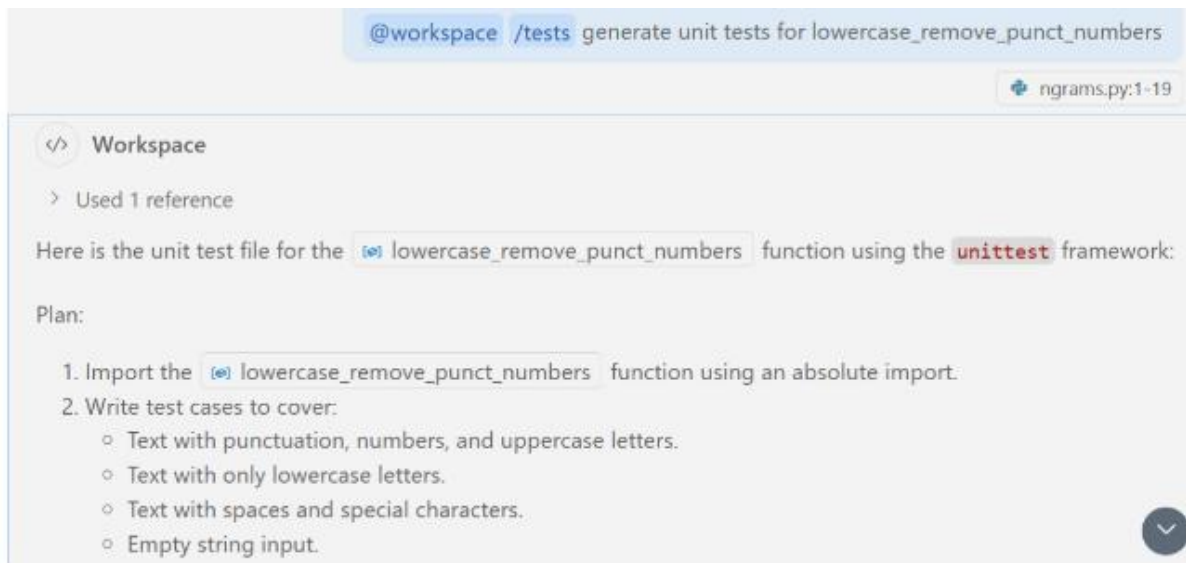
Để đạt được kết quả tốt nhất, lập trình viên nên tuân thủ quy trình bốn bước cốt lõi:

- **Thiết lập Framework:** Trước khi bắt đầu, cần xác định và cấu hình khung kiểm thử phù hợp (ví dụ: unittest hoặc pytest cho Python, Jest cho JavaScript). Copilot sẽ dựa vào các khai báo này để sinh mã đúng cú pháp của thư viện tương ứng.



Hình 8: Quy trình thực thi – Thiết lập framework

- **Xác định mục tiêu:** Lập trình viên lựa chọn hàm, phương thức hoặc lớp cụ thể cần được kiểm tra.
- **Kích hoạt lệnh sinh Test:** Sử dụng các lệnh đặc tả (như /tests trong Copilot Chat) hoặc yêu cầu trực tiếp qua Inline Chat để AI phân tích logic và khởi tạo các kịch bản kiểm thử.



Hình 9: Quy trình thực thi – Kích hoạt lệnh sinh test

- Kiểm tra và hiệu chỉnh: Đây là bước quan trọng nhất. Sau khi AI trả về kết quả, lập trình viên tiến hành rà soát, điều chỉnh các giá trị mong đợi (Expected Output) và làm sạch mã nguồn kiểm thử.

```

1 import unittest
2 from src.ngrams import lowercase_remove_punct_numbers
3
4 # File: test_ngrams.py
5
6 class TestLowercaseRemovePunctNumbers(unittest.TestCase):
7     def test_mixed_input(self):
8         text = "Hello, World! 123"
9         expected = "hello world"
10        self.assertEqual(lowercase_remove_punct_numbers(text), expected)
11
12    def test_lowercase_only(self):
13        text = "this is a test"
14        expected = "this is a test"
15        self.assertEqual(lowercase_remove_punct_numbers(text), expected)
16
17    def test_punctuation_and_numbers(self):
18        text = "!@#%$%^&*()1234567890"
19        expected = ""
20        self.assertEqual(lowercase_remove_punct_numbers(text), expected)
21
22    def test_empty_string(self):
23        text = ""
24        expected = ""
25        self.assertEqual(lowercase_remove_punct_numbers(text), expected)
26
27    def test_spaces_and_special_characters(self):
28        text = " Hello!! World?? "
29        expected = " hello world "

```

Hình 10: Quy trình thực thi – Kiểm tra và hiệu chỉnh

13.4.2. Các đặc điểm hỗ trợ linh hoạt

GitHub Copilot cung cấp khả năng can thiệp đa dạng tùy theo quy mô của dự án:

- Phạm vi kiểm thử đa dạng: AI có thể hỗ trợ tạo nhanh các ca kiểm thử cho từng hàm đơn lẻ hoặc quét toàn bộ tệp tin để xây dựng một bộ Test Suite hoàn chỉnh, giúp đảm bảo độ bao phủ mã nguồn (Code Coverage) ở mức tối ưu.

- Tương tác đa phương thức: Lập trình viên có thể bổ sung các trường hợp kiểm thử mới thông qua giao diện Chat (hỏi-đáp về các kịch bản phức tạp) hoặc sử dụng gợi ý trực tiếp ngay trong tệp tin (Inline) để bổ sung nhanh các dòng lệnh kiểm tra.

13.4.3. Những lưu ý quan trọng khi vận hành

Dù quy trình rất tiện lợi, người dùng cần lưu ý hai điểm mấu chốt để đảm bảo chất lượng hệ thống:

- Loại bỏ mã dư thừa: Đôi khi AI có thể sinh ra các đoạn test lặp lại hoặc không mang lại giá trị thực tế (redundant tests). Việc giữ lại quá nhiều test không cần thiết sẽ làm chậm quá trình CI/CD và gây khó khăn cho việc bảo trì.
- Bắt buộc Review thủ công: Mọi đoạn mã do Copilot tạo ra cần được xem xét dưới lăng kính chuyên môn. Việc tin tưởng tuyệt đối vào AI mà không kiểm tra lại có thể dẫn đến tình trạng "False Positive" — bộ test báo vượt qua (pass) nhưng thực tế mã nguồn vẫn chứa lỗi tiềm ẩn do AI hiểu sai logic nghiệp vụ.

13.5. Sử dụng ChatGPT để viết Unit Test

Khác với các công cụ tích hợp trực tiếp (In-IDE) như GitHub Copilot, ChatGPT đóng vai trò là một "chuyên gia tư vấn kiểm thử" độc lập với khả năng xử lý ngôn ngữ tự nhiên mạnh mẽ. Công cụ này cho phép lập trình viên xây dựng bộ Unit Test thông qua các câu lệnh (prompts) chi tiết và mang tính chiến thuật cao.

```
charged-Coding-with-Gen-AI > ch13 > chatgpt_prompts.txt
You are a Python testing assistant.
Given Python code enclosed within {{{ }}}, generate unit tests using the
unittest framework. For each function or method in the code:
1.Create a corresponding test method within a unittest.TestCase subclass.
2.Use meaningful test method names that reflect the function being tested.
3.Include appropriate assertions based on the function's logic and
expected behavior.
4.Use mock objects or patching where necessary (e.g., for I/O, APIs, or
external dependencies).
5.If a function has multiple logical branches or edge cases, include test
cases for them.
6.Do not include the original code in the output—only the test code.Chapter 13
319
7.Import any modules or classes necessary for the tests to run.
8.Format your output as a complete, valid Python test file using the
unittest module.

Input:
python
{{{
import re

def lowercase_remove_punct_numbers(text, supercharte=True):
    return re.sub(r'^a-z\s]', '', text.lower())

def multiple_to_single_spaces(text):
    letters_single_spaces = re.sub(r'\s+', ' ', text)
    return letters_single_spaces

def create_ngrams(text, n) -> list:
    '''create a list of n-gram tuples from the input text.'''
```

Hình 11: Sử dụng ChatGPT để viết Unit Test

13.5.1. Ưu điểm và Khả năng hỗ trợ phân tích

Việc sử dụng ChatGPT trong quy trình kiểm thử mang lại những lợi ích vượt trội về mặt tư duy và tốc độ:

- Khởi tạo Test Cases tốc độ cao: Chỉ với việc cung cấp đoạn mã nguồn và yêu cầu cụ thể, ChatGPT có thể sinh ra toàn bộ tệp tin kiểm thử với đầy đủ các thư viện và cấu trúc cần thiết trong vài giây.
- Giải thích nguyên nhân thất bại (Root Cause Analysis): Một trong những điểm mạnh đặc biệt của ChatGPT là khả năng phân tích. Khi một đoạn test bị lỗi (fail), lập trình viên có thể dán nội dung lỗi vào chat để AI phân tích, giải thích nguyên nhân và đề xuất phương án sửa đổi mã nguồn hoặc cập nhật lại kịch bản test.
- Gợi ý kịch bản kiểm thử thông minh: Thay vì chỉ viết test cho logic hiện có, ChatGPT có thể tư vấn thêm các kịch bản kiểm thử dựa trên kinh nghiệm từ kho dữ liệu khổng lồ của

nó, chẳng hạn như các lỗi logic phổ biến trong xử lý dữ liệu hoặc các lỗ hổng bảo mật tiềm ẩn.

13.5.2. Những thách thức và hạn chế

Tuy nhiên, việc sử dụng ChatGPT cũng đòi hỏi sự thận trọng và kỹ năng điều phối từ phía người dùng:

- **Rủi ro về tính chính xác:** Tương tự các mô hình ngôn ngữ khác, ChatGPT có thể sinh ra mã kiểm thử sai logic hoặc không tương thích với phiên bản thư viện hiện tại của dự án.
- **Quy trình tinh chỉnh Prompt (Prompt Refinement):** Để nhận được kết quả chất lượng, lập trình viên thường phải thực hiện nhiều lần điều chỉnh câu lệnh, cung cấp thêm ngữ cảnh hoặc các ràng buộc kỹ thuật. Việc viết một prompt thiếu rõ ràng dễ dẫn đến kết quả AI hiểu sai yêu cầu bài toán.

13.6. Data-Driven Testing

Trong các hệ thống phức tạp, một hàm số có thể phải xử lý hàng trăm kịch bản dữ liệu khác nhau. Việc viết thủ công từng đoạn mã kiểm thử cho mỗi kịch bản là một quy trình kém hiệu quả và khó bảo trì. Đây là lúc phương pháp Data-Driven Testing (DDT) trở thành giải pháp tối ưu.

13.6.1. Khái niệm cốt lõi của Data-Driven Testing

Data-Driven Testing (Kiểm thử dựa trên dữ liệu) là một phương pháp kiểm thử phần mềm trong đó dữ liệu kiểm thử (đầu vào và kết quả kỳ vọng) được tách biệt hoàn toàn khỏi mã nguồn của các bài kiểm thử (test scripts).

Thay vì viết nhiều hàm test riêng lẻ cho cùng một logic xử lý, lập trình viên chỉ cần xây dựng một khung kiểm thử duy nhất (test skeleton) và nạp vào đó một tập hợp dữ liệu (dataset). Khung này sẽ tự động lặp qua từng bộ dữ liệu để xác minh tính đúng đắn của hàm.

Cấu trúc cơ bản của DDT bao gồm:

- **Input Data:** Các tham số đầu vào được cung cấp cho hàm cần kiểm thử.
- **Expected Output:** Kết quả mà chúng ta mong đợi hàm sẽ trả về tương ứng với đầu vào đó.
- **Execution Logic:** Mã nguồn kiểm thử dùng chung, có nhiệm vụ đọc dữ liệu và thực hiện lệnh so sánh (assertion).

13.6.2. Ưu điểm

Việc tách biệt dữ liệu kiểm thử ra khỏi mã nguồn thực thi không chỉ là một thay đổi về mặt kỹ thuật mà còn mang lại những lợi ích chiến lược cho quy trình quản lý chất lượng phần mềm:

Giảm thiểu sự lặp lại của mã nguồn (Reduce Code Duplication): Đây là ưu điểm rõ ràng nhất. Thay vì phải viết hàng chục hàm kiểm thử có cấu trúc tương tự nhau chỉ để thay đổi giá trị đầu vào, lập trình viên chỉ cần duy trì một khung kiểm thử (test script) duy nhất. Điều này tuân thủ

triệt để nguyên tắc DRY (Don't Repeat Yourself), giúp bộ mã nguồn kiểm thử trở nên gọn gàng và dễ bảo trì hơn.

- **Mở rộng độ bao phủ kịch bản (Comprehensive Coverage):** Phương pháp này cho phép hệ thống kiểm thử vận hành trên một tập hợp dữ liệu không lồ bao gồm cả các trường hợp thông thường, các giá trị biên (edge cases) và dữ liệu không hợp lệ. Khả năng "quét" qua nhiều kịch bản khác nhau giúp phát hiện ra những lỗi logic tiềm ẩn mà việc kiểm thử thủ công hoặc viết test đơn lẻ thường bỏ sót.
- **Khả năng mở rộng linh hoạt (High Scalability):** Khi phát sinh các yêu cầu nghiệp vụ mới hoặc cần bổ sung các trường hợp kiểm thử bổ sung, lập trình viên không cần phải can thiệp hay viết thêm mã nguồn kiểm thử. Thay vào đó, chỉ cần cập nhật thêm các dòng dữ liệu vào tập tin cấu hình hoặc danh sách đầu vào. Điều này giúp quy trình kiểm thử thích ứng cực nhanh với sự thay đổi của dự án mà không làm tăng độ phức tạp của codebase.

13.6.3. Vai trò của GenAI

Sự kết hợp giữa phương pháp Data-Driven Testing và công nghệ GenAI tạo ra một quy trình kiểm thử cực kỳ mạnh mẽ. Thay vì phải tự tay nhập liệu hàng trăm dòng dữ liệu giả lập, lập trình viên có thể tận dụng AI để thực hiện các tác vụ sau:

- **Tự động hóa danh sách Test Cases:** Dựa trên logic của hàm nguồn, GenAI có khả năng phân tích các nhánh điều kiện (if-else, switch-case) để tự động liệt kê một danh sách các kịch bản kiểm thử cần thiết. Điều này đảm bảo rằng mọi luồng xử lý dữ liệu đều được đưa vào tập dữ liệu kiểm thử mà không bị bỏ sót do sơ suất của con người.
- **Khởi tạo dữ liệu phong phú và thực tế:** Một trong những thách thức của kiểm thử là tạo ra dữ liệu "giống thật". GenAI có thể sinh ra các tập dữ liệu đa dạng như: tên người theo vùng miền, định dạng địa chỉ chính xác, số điện thoại hợp lệ, hoặc các chuỗi JSON phức tạp. Việc kiểm thử trên dữ liệu có độ nhiễu và tính thực tế cao giúp phát hiện các lỗi tiềm ẩn mà dữ liệu giả lập đơn giản (như "test1", "abc") thường không bộc lộ.
- **Mở rộng các kịch bản ngoại lệ:** AI rất mạnh trong việc suy luận các trường hợp biên hoặc dữ liệu "xấu" (malformed data). Lập trình viên có thể yêu cầu AI: "Hãy sinh cho tôi 50 bộ dữ liệu đầu vào bao gồm cả các trường hợp vi phạm định dạng hoặc giá trị rỗng". Khả năng này giúp tăng cường độ bền bỉ (robustness) cho hệ thống một cách nhanh chóng.

13.7. Test-Driven Development (TDD)

Trong các phương pháp luận phát triển phần mềm hiện đại, Test-Driven Development (TDD) nổi lên như một triết lý đảo ngược quy trình truyền thống, đặt việc kiểm thử lên vị trí tiên phong trong quá trình sáng tạo mã nguồn.

13.7.1. Khái niệm và Chu trình Red-Green-Refactor

Test-Driven Development (TDD) là một phương pháp phát triển phần mềm dựa trên một chu trình lặp đi lặp lại cực ngắn. Thay vì viết mã nguồn trước rồi mới viết kiểm thử để xác minh, lập trình viên sẽ thực hiện theo quy trình "Viết kiểm thử trước khi viết mã thực thi".

Quy trình này thường được biết đến với chu trình Red-Green-Refactor:

- **Bước 1: Viết Test (Red):** Lập trình viên viết một đoạn Unit Test cho một chức năng chưa tồn tại. Khi chạy bộ kiểm thử này, nó chắc chắn sẽ thất bại (báo lỗi đỏ) vì chưa có mã nguồn xử lý tương ứng. Bước này giúp xác định rõ ràng yêu cầu và mục tiêu của hàm cần viết.
- **Bước 2: Viết Code để Pass Test (Green):** Lập trình viên viết một lượng mã nguồn vừa đủ—thậm chí là tối giản nhất—chỉ để làm cho bài kiểm thử vừa viết chuyển sang trạng thái vượt qua (báo lỗi xanh). Trọng tâm ở giai đoạn này là tính đúng đắn về chức năng, chưa đặt nặng vấn đề tối ưu hóa.
- **Bước 3: Tối ưu hóa (Refactor):** Sau khi mã nguồn đã vượt qua bài kiểm thử, lập trình viên tiến hành tái cấu trúc (refactor) lại đoạn mã đó để làm cho nó sạch hơn, dễ đọc hơn và hiệu quả hơn. Nhờ có bộ test đã viết ở Bước 1, việc tối ưu hóa diễn ra cực kỳ an toàn vì bất kỳ sai sót nào làm hỏng chức năng đều sẽ bị phát hiện ngay lập tức.

13.7.2. Lợi ích

Việc áp dụng TDD không chỉ là thay đổi thứ tự viết code mà còn là một cuộc cách mạng trong tư duy thiết kế phần mềm. Những lợi ích quan trọng nhất bao gồm:

- **Cải thiện thiết kế hệ thống (Better Design):** Vì lập trình viên phải viết kịch bản sử dụng (test) trước khi viết mã thực thi, họ buộc phải suy nghĩ kỹ về giao diện (interface), các tham số đầu vào và kết quả trả về. Điều này giúp mã nguồn trở nên lỏng lẻo về sự phụ thuộc (loosely coupled) và có tính gắn kết cao (highly cohesive), tránh được tình trạng viết code thừa hoặc quá phức tạp ngay từ đầu.
- **Tăng khả năng kiểm thử (High Testability):** Một hệ thống được xây dựng bằng TDD mặc nhiên là một hệ thống dễ kiểm thử. Do mã nguồn được sinh ra để thỏa mãn các bài test, các thành phần trong chương trình thường được module hóa tốt, giúp việc bảo trì và mở rộng sau này trở nên cực kỳ thuận tiện mà không gặp phải các "góc tối" khó tiếp cận bằng code kiểm thử.
- **Đảm bảo độ bao phủ mã nguồn tối ưu (Full Code Coverage):** Trong quy trình TDD, không có dòng code thực thi nào được viết ra mà không có một bài test tương ứng yêu cầu nó. Điều này giúp dự án đạt được chỉ số Code Coverage gần như tuyệt đối một cách tự nhiên. Lập trình viên có thể hoàn toàn tự tin rằng mọi nhánh logic, mọi điều kiện rẽ nhánh đều đã được kiểm chứng trước khi chuyển sang tính năng tiếp theo.
- **Tạo dựng sự tự tin cho đội ngũ:** TDD cung cấp một "hàng rào bảo vệ" vững chắc. Khi thực hiện các thay đổi lớn hoặc nâng cấp hệ thống, bộ test suite khổng lồ tích lũy được từ quá trình TDD sẽ ngay lập tức cảnh báo nếu có bất kỳ sai sót nào xảy ra, giúp duy trì sự ổn định xuyên suốt vòng đời dự án.

13.7.3. Phương pháp thực hiện

Mặc dù TDD mang lại nhiều lợi ích lý thuyết, nhưng trong môi trường làm việc thực tế, việc áp dụng kiểm thử thường xoay quanh hai hướng tiếp cận chính tùy thuộc vào mục tiêu của dự án:

- **Viết kiểm thử trước (Test-First/TDD):** Lập trình viên thiết lập các tiêu chuẩn kiểm thử trước khi viết mã nguồn. Cách tiếp cận này ưu tiên tính thiết kế và độ an toàn của hệ thống. Nó đặc biệt hiệu quả cho các dự án có logic nghiệp vụ phức tạp, đòi hỏi sự chính xác tuyệt đối và tính bảo trì dài hạn.
- **Viết mã trước, kiểm thử sau (Code-First/Test-After):** Đây là cách tiếp cận phổ biến hơn trong các giai đoạn làm Prototype hoặc các dự án cần tốc độ triển khai cực nhanh. Lập trình viên tập trung hiện thực hóa tính năng trước, sau đó mới bổ sung các bộ Unit Test để gia cố độ ổn định. Phương pháp này linh hoạt nhưng dễ dẫn đến tình trạng "nợ kỹ thuật" nếu các bộ test không được bổ sung kịp thời.

13.8. Kết hợp GenAI với TDD

Việc áp dụng Test-Driven Development (TDD) truyền thống thường bị coi là tốn thời gian do đòi hỏi lập trình viên phải viết mã kiểm thử trước khi viết mã tính năng. Tuy nhiên, sự xuất hiện của GenAI đã thay đổi hoàn toàn định kiến này bằng cách tự động hóa các bước lặp trong chu trình phát triển. Ta sẽ mô tả quá trình sử dụng GenAI với TDD thông qua bài toán sau.

13.8.1. Mô tả bài toán

Để biểu diễn một hình chữ nhật trên mặt phẳng tọa độ Oxy một cách tối giản và hiệu quả, chúng ta sử dụng phương pháp xác định thông qua hai điểm đối góc: **góc dưới bên trái** và **góc trên bên phải**.

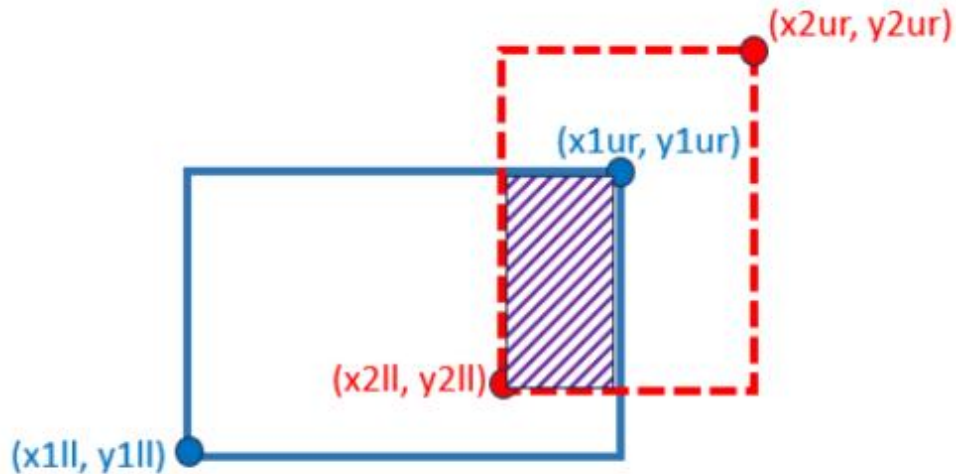
Cụ thể, mỗi hình chữ nhật được định nghĩa bởi một bộ 4 giá trị số thực (x_l, y_l, x_u, y_u) , trong đó:

- (x_l, y_l) : Tọa độ góc dưới bên trái (Lower-Left), đại diện cho giá trị nhỏ nhất của trục hoành và trục tung mà hình chữ nhật chiếm chỗ.
- (x_u, y_u) : Tọa độ góc trên bên phải (Upper-Right), đại diện cho giá trị lớn nhất của trục hoành và trục tung.

Ý nghĩa của cách biểu diễn

Cách biểu diễn này mang lại nhiều lợi thế trong lập trình:

- **Tính toán diện tích:** $S = (x_u - x_l) \times (y_u - y_l)$.
- **Kiểm tra điểm thuộc hình chữ nhật:** Một điểm $P(x, y)$ nằm trong hình chữ nhật nếu $x_l \leq x \leq x_u$ và $y_l \leq y \leq y_u$.
- **Kiểm tra giao nhau:** Việc xác định hai hình chữ nhật có chồng lấp lên nhau hay không trở nên đơn giản hơn thông qua việc so sánh trực tiếp các biên tọa độ này.



Hình 12: Hình vẽ mô tả bài toán

13.8.2. Các trường hợp cần xử lý

Dựa trên mô tả bài toán tại mục 8.1, chúng ta cần xác định rõ các kịch bản kiểm thử (Test Cases) để chuẩn bị cho quy trình TDD. Việc phân loại này giúp đảm bảo mã nguồn sẽ xử lý chính xác cả các trường hợp thông thường lẫn các lỗi dữ liệu tiềm ẩn.

Trước khi bắt tay vào viết mã thực thi, việc liệt kê các trường hợp biên và kịch bản sử dụng là bước then chốt trong chu trình **Red** của TDD. Đối với bài toán quản lý hình chữ nhật, chúng ta cần tập trung vào 3 nhóm chính:

13.8.2.1. Hai hình chữ nhật có phần giao nhau (Overlap)

Đây là trường hợp phổ biến nhất, khi hai hình chữ nhật có ít nhất một vùng không gian chung.

- **Đặc điểm:** Vùng giao nhau phải có diện tích lớn hơn 0 ($S_{\text{intersection}} > 0$).
- **Ví dụ minh họa:** $\text{rect1} = (0, 0, 4, 4)$, $\text{rect2} = (2, 2, 6, 6)$
- **Kết quả mong đợi:** Vùng giao là $(2, 2, 4, 4)$ với diện tích bằng 4.

1. Intersecting rectangles (Figure 13.10):

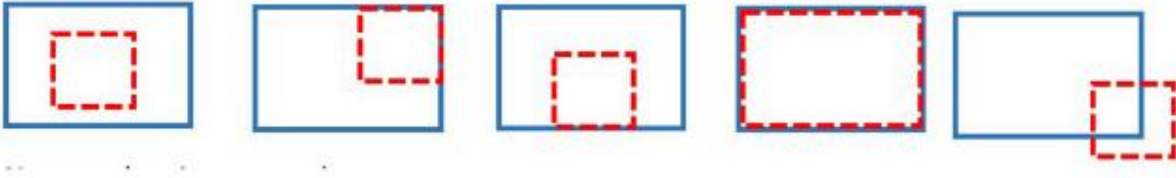


Figure 13.10: Example test cases for overlapping rectangles

Hình 13: Hai hình chữ nhật có phần giao nhau

13.8.2.2. Hai hình chữ nhật không giao nhau (Disjoint)

Trường hợp này xảy ra khi các hình chữ nhật nằm tách biệt hoàn toàn hoặc chỉ tiếp xúc nhau tại biên/đỉnh mà không chồng lấn diện tích.

- **Đặc điểm:** Không có phần diện tích chung (Sintersection=0).
- **Ví dụ minh họa:**
 - $rect1=(0,0,2,2)$
 - $rect2=(3,3,5,5)$
- **Kết quả mong đợi:** Hàm xử lý trả về diện tích bằng 0 hoặc thông báo không giao nhau.

2. Non-intersecting rectangles (Figure 13.11):

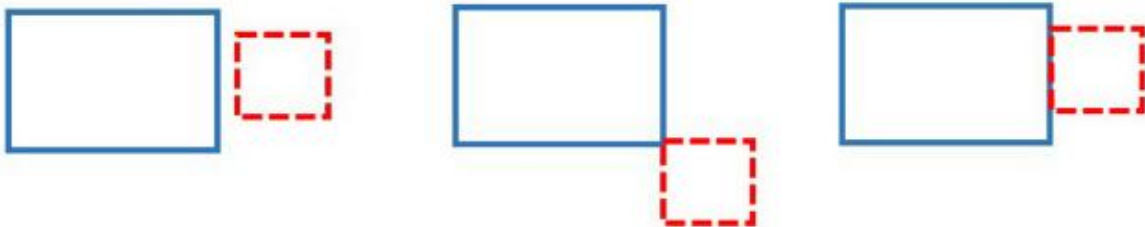


Figure 13.11: Example test cases for non-overlapping rectangles

Hình 14: Hai hình chữ nhật không giao nhau

13.8.2.3. Dữ liệu hình chữ nhật không hợp lệ (Invalid Data)

Đây là kịch bản quan trọng để kiểm tra tính bền bỉ của ứng dụng (Error Handling).

- **Đặc điểm:** Tọa độ góc dưới bên trái lớn hơn hoặc bằng tọa độ góc trên bên phải ($x_{ll} \geq x_{ur}$ hoặc $y_{ll} \geq y_{ur}$), khiến hình chữ nhật không thể tồn tại trong thực tế.

- **Ví dụ minh họa:**

- `rect=(4,4,2,2)`

- **Kết quả mong đợi:** Hệ thống cần nhận diện đây là dữ liệu sai và thực hiện **ngắt chương trình (raise exception)** hoặc trả về thông báo lỗi cụ thể thay vì tiếp tục tính toán sai lệch.

3. Invalid rectangles (Figure 13.12):



Figure 13.12: Example test cases for invalid rectangles that fail our definition

Hình 15: Dữ liệu hình chữ nhật không hợp lệ

13.8.4. Sử dụng GenAI cho chu trình TDD

GenAI đóng vai trò là một "cỗ máy thực thi" giúp hiện thực hóa các ý tưởng thiết kế một cách nhanh chóng:

- **Sinh Test từ yêu cầu (Prompt to Test):** Lập trình viên chỉ cần cung cấp đặc tả tính năng bằng ngôn ngữ tự nhiên, AI sẽ tự động khởi tạo bộ Unit Test tương ứng (Giai đoạn **Red**).
- **Sinh Code từ Test (Test to Code):** Sau khi đã có bộ test, bạn có thể yêu cầu AI: "*Hãy viết mã nguồn tối giản để pass bộ test này*". AI sẽ phân tích các kỳ vọng (assertions) trong test để viết ra đoạn code chức năng phù hợp (Giai đoạn **Green**).
- **Sửa lỗi dựa trên kết quả thực thi:** Nếu bộ test thất bại, GenAI có khả năng đọc hiểu thông báo lỗi (traceback) để đề xuất các phương án điều chỉnh mã nguồn ngay lập tức.

Để ví dụ cho quy trình này ta sẽ sử dụng ChatGPT làm GenAI để sinh TDD:

Prompt sử dụng:

```

CONTEXT: You are provided with
1. Python function enclosed with {{{ FUNCTION }}} that has a
2. Test suite code enclosed with {{{ UNIT TESTS }}}.
3. Error message enclosed with {{{ ERROR }}}.
TASK: Fix the function implementation
FUNCTION: {{{def rect_intersection_area(rect1, rect2):
return NoneWriting and Maintaining Unit Tests
328
}}}}
UNIT TESTS: {{{ ... omitted for brevity ... }}}
ERROR: {{{ ... omitted for brevity ... }}}
CODE:      You, 4 days ago • Uncommitted changes

```

Hình 16: Sử dụng GenAI cho chu trình TDD

Kết quả được trả về:

```

You, 6 days ago | 1 author (You)
1 def rect_intersection_area(rect1, rect2):      You, 6 days ago
2     x1_1, y1_1, x2_1, y2_1 = rect1
3     x1_2, y1_2, x2_2, y2_2 = rect2
4
5     # Validate input rectangles
6     def is_invalid_rect(x1, y1, x2, y2):
7         return x1 >= x2 or y1 >= y2
8
9     if is_invalid_rect(x1_1, y1_1, x2_1, y2_1) or is_invalid_rect(x1_2, y1_2, x2_2, y2_2):
10        raise ValueError("Invalid rectangle with non-positive width or height")
11
12    # Find intersection bounds
13    inter_left = max(x1_1, x1_2)
14    inter_right = min(x2_1, x2_2)
15    inter_bottom = max(y1_1, y1_2)
16    inter_top = min(y2_1, y2_2)
17
18    # Check if rectangles intersect
19    if inter_left >= inter_right or inter_bottom >= inter_top:
20        return 0
21
22    # Compute and return intersection area
23    return (inter_right - inter_left) * (inter_top - inter_bottom)
24

```

Hình 17: Kết quả đạt được

13.8.5. Quy trình phối hợp thực tế

Một quy trình làm việc tối ưu khi kết hợp AI vào TDD bao gồm các bước lặp sau:

- **Cung cấp ngữ cảnh:** Lập trình viên đưa vào bộ test hiện tại kèm theo thông báo lỗi cụ thể.
- **Đề xuất giải pháp:** GenAI phân tích sự sai lệch giữa kết quả thực tế và kỳ vọng để đề xuất đoạn code sửa lỗi.
- **Thực thi và tinh chỉnh:** Lập trình viên áp dụng code, chạy lại test. Nếu vẫn fail, tiếp tục phản hồi cho AI cho đến khi hệ thống báo xanh (Pass).

13.9. Tạo dữ liệu kiểm thử bằng GenAI

Trong kiểm thử phần mềm, chất lượng của mã kiểm thử phụ thuộc trực tiếp vào chất lượng của dữ liệu đầu vào. Việc sử dụng các giá trị ngẫu nhiên hoặc quá đơn giản thường làm bỏ sót các lỗi logic tinh vi.

13.9.1. Vai trò chiến lược của dữ liệu kiểm thử

Một bộ dữ liệu kiểm thử được coi là hiệu quả khi hội đủ ba yếu tố cốt lõi sau:

- **Tính thực tế (Realism):** Dữ liệu cần phản ánh đúng các kịch bản mà ứng dụng sẽ gặp phải trong môi trường vận hành thực tế. Đối với bài toán hình chữ nhật, điều này có nghĩa là các tọa độ phải nằm trong dải giá trị cho phép của hệ tọa độ đồ họa và có các tỉ lệ (rộng/cao) đa dạng.
- **Tính đa dạng (Diversity):** Bộ dữ liệu không chỉ bao gồm các trường hợp "thuận buồm xuôi gió" (Happy Path) mà phải bao phủ được toàn bộ các trường hợp biên.
 - *Ví dụ:* Các hình chữ nhật nằm sát mép nhau, các hình lồng nhau hoàn toàn, hoặc các hình có diện tích cực nhỏ (0.00001).
- **Khả năng lặp lại (Repeatability):** Dữ liệu kiểm thử phải được đóng gói sao cho mọi thành viên trong đội ngũ phát triển hoặc hệ thống CI/CD đều có thể chạy lại và thu được kết quả đồng nhất. Điều này giúp xác nhận rằng lỗi đã thực sự được sửa và không có lỗi mới phát sinh.

13.9.2. Ứng dụng GenAI

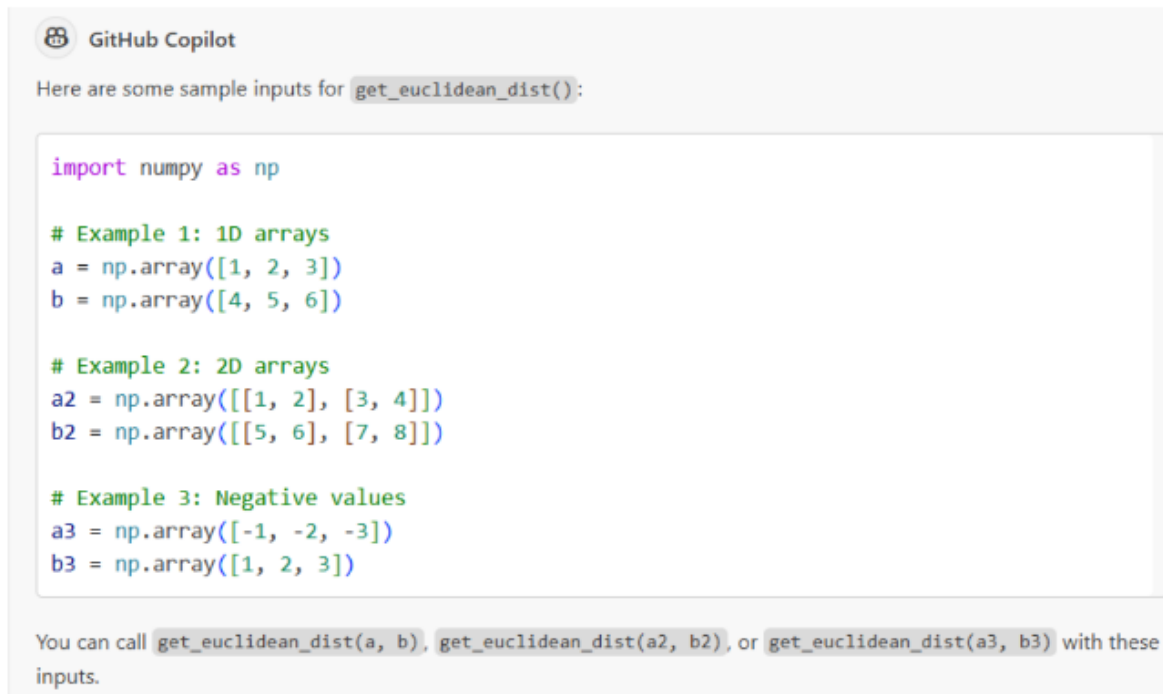
Dựa trên những vai trò đã phân tích, ta sẽ đi sâu vào các khả năng thực thi cụ thể của GenAI trong việc khởi tạo dữ liệu, giúp lập trình viên không còn phải đối mặt với việc nhập liệu thủ công nhàm chán.

Chúng ta sẽ sử dụng Github Copilot cho việc tạo ra dữ liệu kiểm thử cho hàm `get_euclidean_dist()` có nội dung:

```
def get_euclidean_dist(a: np.ndarray, b: np.ndarray) -> float:
    """Return the squared Euclidean (L2) distance between two arrays."""
    return np.sqrt(np.sum((a - b) ** 2))
```

Hình 18: Dùng GenAI để khởi tạo dữ liệu

Kết quả nhận được:



Hình 19: Kết quả nhận được

Như vậy, trong quy trình TDD, các nhà phát triển viết mã kiểm thử đơn vị trước, và mã triển khai được viết để vượt qua các bài kiểm thử đơn vị. Các bài kiểm thử đơn vị có thể được viết bởi các nhà phát triển sử dụng các phương pháp GenAI tiêu chuẩn để xác minh rằng mã triển khai sẽ đáp ứng các yêu cầu. Từ VS Code hoặc PyCharm, GitHub Copilot có thể tạo mã triển khai một cách lặp đi lặp lại chỉ từ mã kiểm thử đơn vị. ChatGPT cung cấp chức năng tương tự thông qua một mẫu nhắc nhở. Phần tiếp theo đưa ra các khuyến nghị về cách sử dụng tốt nhất các phương pháp GenAI để hoàn thành mã triển khai và kiểm thử.

13.9.3. Khả năng thực thi của GenAI trong khởi tạo dữ liệu

GenAI không chỉ sinh ra văn bản đơn thuần mà còn có khả năng cấu trúc hóa dữ liệu theo nhiều định dạng và ngữ cảnh chuyên biệt, phục vụ trực tiếp cho các bộ Unit Test phức tạp:

Tạo bảng dữ liệu đa định dạng (CSV, JSON, SQL): Thay vì tự gõ từng dòng tọa độ cho bài toán hình chữ nhật, bạn có thể yêu cầu GenAI: *"Hãy tạo một tệp JSON chứa 20 cặp hình chữ nhật, bao gồm cả trường hợp giao nhau và không giao nhau"*. AI sẽ trả về dữ liệu được định dạng chuẩn xác, giúp bạn dễ dàng nạp vào các hàm kiểm thử theo phương pháp **Data-Driven Testing**.

Sinh dữ liệu chuyên ngành đặc thù: Đối với các dự án phần mềm trong lĩnh vực y tế, sinh học hoặc logistics, việc tìm kiếm dữ liệu mẫu thường rất khó khăn. GenAI có thể mô phỏng:

- **Y tế/Sinh học:** Các chuỗi mã DNA, danh mục mã bệnh lý (ICD-10) hoặc các chỉ số xét nghiệm giả lập.
- **Logistics:** Các địa chỉ giao hàng thực tế tại một quốc gia cụ thể, mã vận đơn hoặc tọa độ GPS.
- **Tài chính:** Số thẻ tín dụng giả lập (tuân thủ thuật toán Luhn) hoặc lịch sử giao dịch mẫu.

Tạo dữ liệu đầu vào (Input) mẫu cho hàm: Khi bạn có một hàm xử lý (ví dụ: `calculate_overlap_area`), GenAI có thể phân tích các tham số đầu vào của hàm đó và tự động đề xuất một bộ giá trị mẫu (Mock data) ngay trong mã nguồn. Điều này đặc biệt hữu ích khi bạn cần tạo ra các đối tượng (objects) phức tạp hoặc các cấu trúc lồng nhau để kiểm thử.

13.10. Kết luận

Chương 13 đã đi sâu vào hệ sinh thái của việc đảm bảo chất lượng phần mềm, từ những khái niệm nền tảng đến những phương pháp luận tiên tiến nhất đang được định hình lại bởi kỷ nguyên AI.

Xuyên suốt chương này, chúng ta đã xây dựng một bức tranh toàn diện về:

- **Unit Testing:** Khẳng định vai trò là "lớp phòng thủ" đầu tiên và quan trọng nhất, giúp phát hiện lỗi sớm và giảm thiểu chi phí bảo trì.
- **Ứng dụng GenAI trong Kiểm thử:** Cách tận dụng các công cụ như GitHub Copilot và ChatGPT để tự động hóa việc viết mã test, giải thích lỗi và tối ưu hóa các kịch bản kiểm thử.
- **Data-Driven Testing (DDT):** Phương pháp tách biệt dữ liệu khỏi mã nguồn, giúp tăng độ bao phủ và giảm sự lặp lại, biến việc kiểm thử trở nên linh hoạt và dễ mở rộng.
- **Test-Driven Development (TDD):** Triết lý "kiểm thử trước, code sau", tạo ra một quy trình thiết kế phần mềm chặt chẽ, an toàn và hướng tới chất lượng nội tại.
- **Chiến lược tạo dữ liệu:** Tầm quan trọng của dữ liệu thực tế, đa dạng và cách GenAI giúp chúng ta khởi tạo những tập dữ liệu chuyên ngành phức tạp chỉ trong vài giây.

Thông điệp quan trọng nhất của toàn bộ nội dung này không phải là việc AI thay thế lập trình viên, mà là **sự cộng hưởng sức mạnh**:

Quy trình tối ưu = Tư duy chiến lược của Con người + Tốc độ thực thi của GenAI

Kết quả cuối cùng là một quy trình phát triển **hiệu quả hơn, nhanh chóng hơn** nhưng vẫn đảm bảo được **độ tin cậy và chất lượng cao nhất** cho sản phẩm phần mềm.

CHƯƠNG 14: GenAI for Runtime and Memory Management

14.1. Giới thiệu

Trong kỷ nguyên Big Data, tối ưu hóa hiệu năng không còn là tùy chọn mà là yêu cầu sống còn của quy trình phát triển phần mềm (SDLC). Hai chỉ số quyết định sự thành bại của hệ thống chính là Runtime (tốc độ thực thi) và Memory usage (mức độ chiếm dụng bộ nhớ). Một chương trình vận hành tốt trên dữ liệu nhỏ có thể dễ dàng đổ vỡ khi đối mặt với quy mô dữ liệu khổng lồ nếu thuật toán không được tối ưu hóa. Chương 14 tập trung khai thác sức mạnh của Generative AI, điển hình là ChatGPT và GitHub Copilot, đóng vai trò như những chuyên gia tư vấn hiệu năng chuyên sâu. Thay vì thực hiện các bước đo đạc thủ công phức tạp, lập trình viên có thể tận dụng AI để quét mã nguồn, phát hiện các "điểm nghẽn" logic, và phân tích độ phức tạp thuật toán. Đặc biệt, GenAI hỗ trợ dự đoán giới hạn xử lý (max capacity), giúp hệ thống sẵn sàng cho khả năng mở rộng (scaling). Từ đó, AI đề xuất các giải pháp cải thiện cụ thể như cấu trúc dữ liệu hiệu quả hơn hoặc kỹ thuật xử lý song song, giúp cân bằng giữa tốc độ xử lý và tài nguyên sử dụng, đảm bảo phần mềm luôn đạt hiệu suất tối ưu trong thực tế.

Chương 14 tập trung vào việc ứng dụng Generative AI (GenAI), đặc biệt là các mô hình như ChatGPT và GitHub Copilot, để:

- Phân tích hiệu năng chương trình
- Dự đoán giới hạn xử lý (max capacity)
- Đề xuất các phương pháp tối ưu hóa

14.2. Phân tích Runtime và Space Complexity

14.2.1 Runtime của chương trình

Runtime được hiểu là khoảng thời gian cần thiết để một chương trình hoàn thành toàn bộ tác vụ được giao. Đây là chỉ số trực quan nhất để người dùng và lập trình viên đánh giá tốc độ của ứng dụng. Trong thực tế, Runtime không phải là một hằng số cố định mà chịu tác động trực tiếp bởi ba yếu tố then chốt: Độ phức tạp thuật toán (quyết định số lượng bước tính toán), Kích thước dữ liệu đầu vào (dữ liệu càng lớn, khối lượng xử lý càng nhiều), và Cấu hình phần cứng (tốc độ xung nhịp của CPU và khả năng xử lý song song).

Một ví dụ điển hình để minh họa cho sự kém hiệu quả về Runtime là thuật toán Fibonacci sử dụng đệ quy thuần túy. Thuật toán này có độ phức tạp thời gian lên đến $O(2^n)$, nghĩa là thời gian thực thi sẽ tăng theo cấp số nhân dựa trên giá trị n đầu vào. Khi n chỉ tăng thêm một đơn vị nhỏ, số lượng các lời gọi hàm đệ quy sẽ bùng nổ, dẫn đến việc thời gian xử lý trở nên cực kỳ lớn và không khả thi trên các máy tính thông thường (ví dụ: tính Fibonacci của 50 có thể mất nhiều phút hoặc cả giờ). Điều này khẳng định rằng: Dù phần cứng có mạnh mẽ đến đâu, việc lựa chọn thuật toán phù hợp (như chuyển sang Quy hoạch động với $O(n)$) vẫn là yếu tố quyết định hàng đầu đến hiệu năng thực tế của chương trình.

```
def fibonacci_recursive(n):
    if n <= 1:
        return n
    return fibonacci_recursive(n - 1) + fibonacci_recursive(n - 2)
```

Hình 20: Phân tích Runtime và Space Complexity của bài toán Fibonacci

14.2.2 Bộ nhớ của chương trình

Song song với thời gian thực thi, Space (Độ phức tạp không gian) là chỉ số đo lường lượng bộ nhớ RAM mà chương trình chiếm dụng trong suốt quá trình vận hành. Trong kỷ nguyên dữ liệu lớn, việc quản lý bộ nhớ trở nên cực kỳ thách thức; một chương trình dù chạy nhanh đến đâu cũng sẽ trở nên vô dụng nếu nó yêu cầu lượng tài nguyên vượt quá khả năng đáp ứng của hệ thống, dẫn đến lỗi treo máy hoặc Out of Memory (OOM).

Một ví dụ điển hình trong phân tích dữ liệu là việc đọc một tệp CSV khổng lồ vào Pandas DataFrame. Mặc dù thao tác này rất tiện lợi, nhưng Pandas thường yêu cầu lượng RAM gấp nhiều lần kích thước thực tế của tệp trên ổ cứng để lưu trữ các cấu trúc dữ liệu nội bộ. Nếu tệp dữ liệu lên đến hàng chục GB, hệ thống sẽ ngay lập tức cạn kiệt bộ nhớ. Tương tự như Runtime, độ phức tạp không gian cũng được biểu diễn qua ký pháp Big-O để ước tính khả năng tiêu thụ tài nguyên. Chẳng hạn, một ma trận có n dòng và m cột sẽ có độ phức tạp không gian là $O(n \times m)$. Việc hiểu rõ giới hạn này giúp lập trình viên đưa ra các chiến thuật tối ưu như xử lý dữ liệu theo từng phần (chunking) hoặc sử dụng các cấu trúc dữ liệu thưa (sparse matrices) để tiết kiệm tài nguyên.

	video_1	video_2	video_3	video_4	video_5	video_6	...
user_1		0.5				1	
user_2	0.1			0.7		0.9	

Hình 21: Mô phỏng bộ nhớ của chương trình

14.2.3 Trade-off giữa Runtime và Memory

Trong thực tế phát triển phần mềm, việc tối ưu hóa hoàn hảo cả hai yếu tố Runtime và Space cùng lúc là một thử thách cực kỳ khó khăn. Thông thường, khi cải thiện một yếu tố, chúng ta buộc phải chấp nhận sự suy giảm ở yếu tố còn lại. Đây chính là nguyên lý "Đánh đổi" (Trade-off) — một trong những khái niệm nền tảng nhất trong thiết kế hệ thống và cấu trúc dữ liệu.

Hai ví dụ điển hình minh chứng cho nguyên lý này bao gồm:

- Kỹ thuật Caching (Bộ nhớ đệm): Để tăng tốc độ phản hồi cho người dùng, hệ thống lưu trữ sẵn các kết quả tính toán phức tạp hoặc dữ liệu từ database vào RAM.
 - Lợi ích: Tăng tốc độ thực thi (giảm Runtime) đáng kể.

- Chi phí: Tiêu tốn thêm một lượng bộ nhớ RAM lớn để duy trì các bản ghi đệm.
- Kỹ thuật Chunking (Xử lý theo khối): Thay vì nạp toàn bộ tệp dữ liệu hàng GB vào bộ nhớ, chương trình chia nhỏ dữ liệu thành từng phần (chunks) để xử lý tuần tự.
 - Lợi ích: Giảm thiểu tối đa lượng bộ nhớ sử dụng (giảm Space), giúp chạy được trên các máy cấu hình thấp.
 - Chi phí: Tăng tổng thời gian xử lý (tăng Runtime) do phát sinh thêm các thao tác đọc/ghi ổ cứng liên tục và quản lý vòng lặp xử lý.

Không có một thiết kế nào là "tốt nhất" một cách tuyệt đối. Vai trò của người kỹ sư phần mềm (với sự hỗ trợ của GenAI) là phân tích ngữ cảnh cụ thể để quyết định xem hệ thống cần ưu tiên tốc độ (như các ứng dụng giao dịch tài chính) hay ưu tiên tiết kiệm tài nguyên (như các thiết bị IoT hoặc nhúng).

14.3. Profiling (Đo hiệu năng)

Sau khi đã nắm vững các khái niệm lý thuyết về độ phức tạp, bước tiếp theo trong quy trình tối ưu hóa là **Profiling** — quá trình đo lường các chỉ số vận hành thực tế của mã nguồn để xác định chính xác các "điểm nghẽn" (bottlenecks).

14.3.1. Profiling Runtime (Đo thời gian thực thi)

Phương pháp phổ biến và đơn giản nhất để đo lường Runtime trong Python là sử dụng module tích hợp sẵn `time`. Quy trình này bao gồm ba bước cơ bản:

```

100, 23 hours ago | 1 author (100)
import time
from fibonacci import fibonacci_recursive, fibonacci_iterative

n = 35
start_time = time.time()
result = fibonacci_recursive(n)
end_time = time.time()

print(f"Result: {result}")
print(f"Runtime: {end_time - start_time} seconds")

```

Hình 22: Đo thời gian thực thi

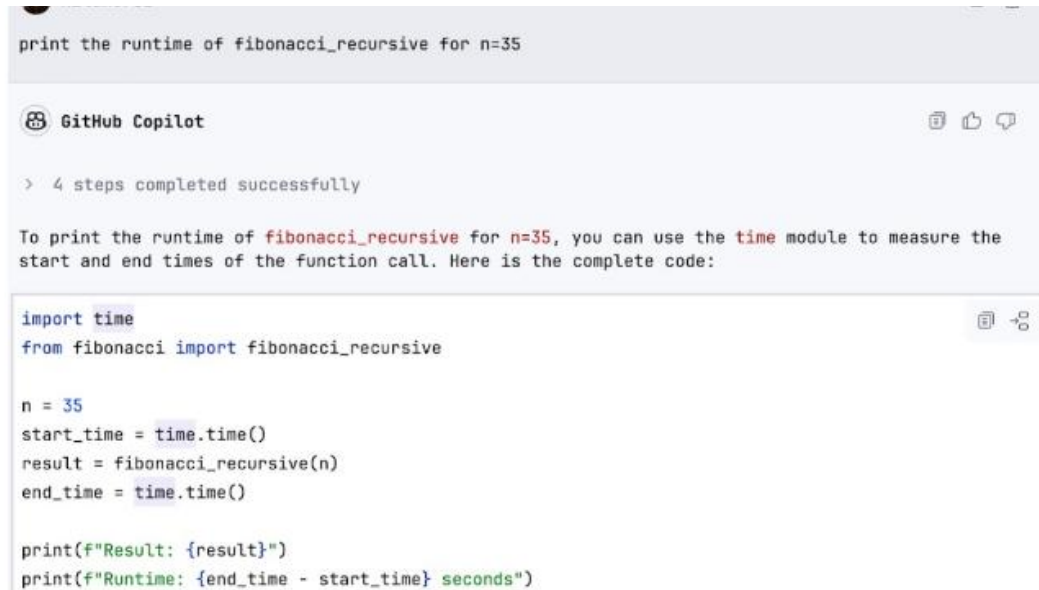
Ghi lại thời điểm bắt đầu: Sử dụng `time.perf_counter()` hoặc `time.time()` ngay trước khi hàm thực thi.

Thực hiện tác vụ: Chạy đoạn mã nguồn cần kiểm tra hiệu năng.

Tính toán chênh lệch: Ghi lại thời điểm kết thúc và lấy hiệu số để biết chính xác tổng thời gian xử lý.

Vai trò của GitHub Copilot trong Profiling:

Thay vì phải viết các đoạn mã đo lường lặp đi lặp lại, **GitHub Copilot** đóng vai trò là trợ lý đặc lực giúp tăng tốc quá trình này:



```
print the runtime of fibonacci_recursive for n=35
```

GitHub Copilot

> 4 steps completed successfully

To print the runtime of `fibonacci_recursive` for `n=35`, you can use the `time` module to measure the start and end times of the function call. Here is the complete code:

```
import time
from fibonacci import fibonacci_recursive

n = 35
start_time = time.time()
result = fibonacci_recursive(n)
end_time = time.time()

print(f"Result: {result}")
print(f"Runtime: {end_time - start_time} seconds")
```

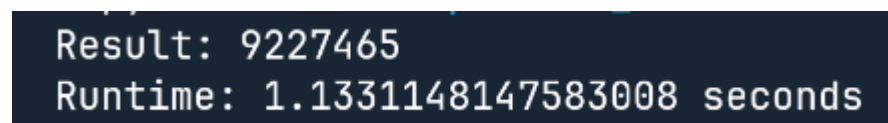
Hình 23: Vai trò của Github Copilot trong Profiling

Tự động sinh mã Profiling: Khi bạn viết một hàm, bạn chỉ cần gõ chú thích `// measure execution time`, Copilot sẽ tự động gợi ý toàn bộ cấu trúc mã bao gồm cả việc import thư viện `time` và tính toán kết quả trả về.

Gợi ý các công cụ chuyên sâu: Ngoài module `time` cơ bản, Copilot có thể gợi ý sử dụng các thư viện mạnh mẽ hơn như `timeit` (để đo các đoạn mã ngắn với độ chính xác cao) hoặc `cProfile` (để phân tích chi tiết thời gian tiêu tốn trong từng hàm con của một hệ thống lớn).

Phân tích nhanh kết quả: Dựa trên dữ liệu thời gian ghi nhận được, bạn có thể hỏi Copilot: "Với kết quả runtime này, đoạn code của tôi có đang gặp vấn đề về vòng lặp không?", AI sẽ đưa ra những nhận định sơ bộ về tính hợp lý của hiệu năng hiện tại.

Kết quả chạy của code trên:



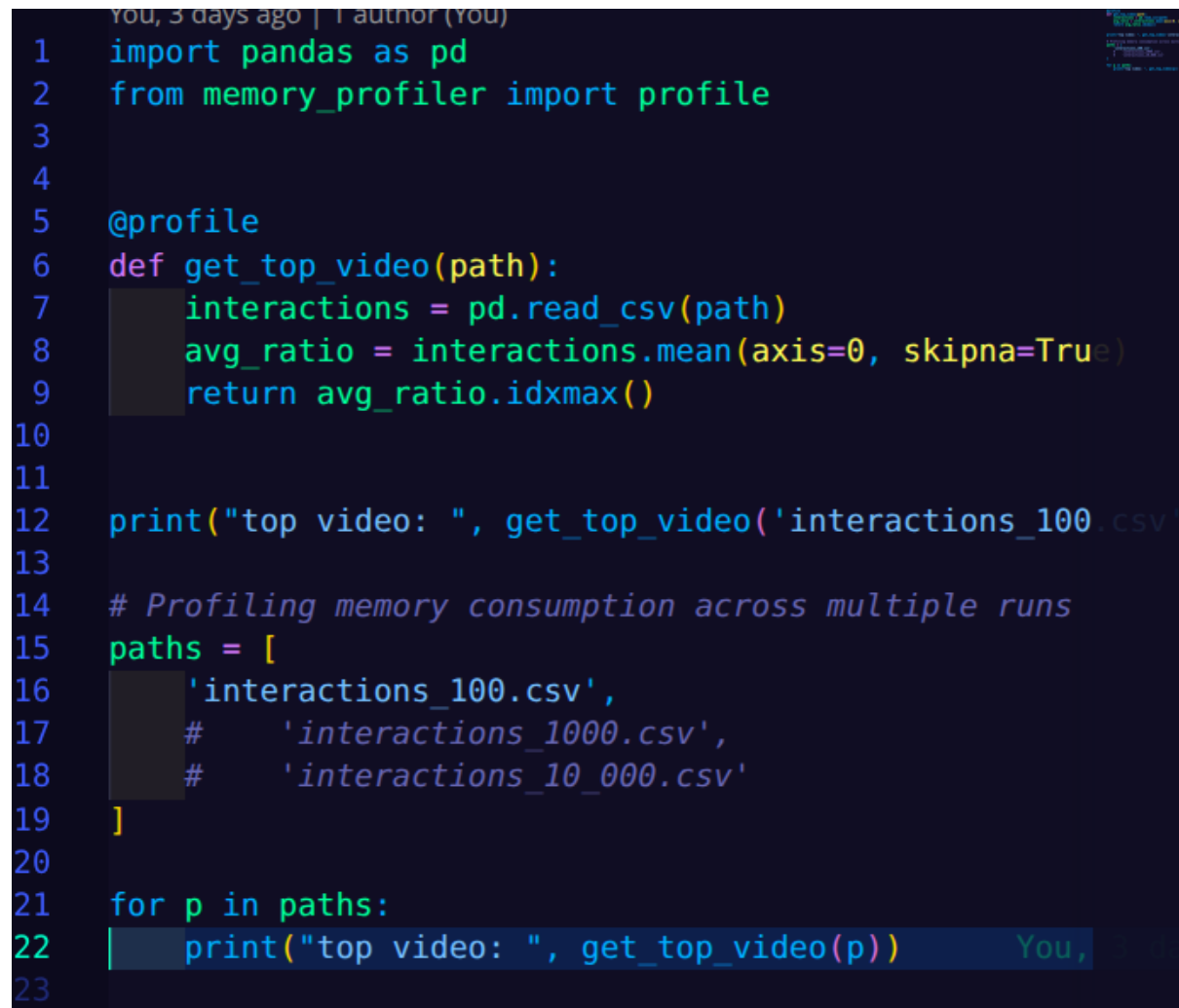
```
Result: 9227465
Runtime: 1.1331148147583008 seconds
```

Hình 24: Kết quả thực thi

14.3.2 Profiling Memory

Bên cạnh thời gian thực thi, việc kiểm soát lượng tài nguyên RAM mà chương trình chiếm dụng là yếu tố sống còn để đảm bảo hệ thống không bị treo hoặc gặp lỗi **Out of Memory**. Đối với các bài toán xử lý dữ liệu lớn, việc đo lường bộ nhớ cần sự chi tiết đến từng dòng lệnh.

Sử dụng thư viện `memory_profiler`: Đây là một trong những công cụ mạnh mẽ nhất trong hệ sinh thái Python, cho phép lập trình viên theo dõi sát sao sự biến thiên của bộ nhớ trong suốt vòng đời của một hàm.



```
1 import pandas as pd
2 from memory_profiler import profile
3
4
5 @profile
6 def get_top_video(path):
7     interactions = pd.read_csv(path)
8     avg_ratio = interactions.mean(axis=0, skipna=True)
9     return avg_ratio.idxmax()
10
11
12 print("top video: ", get_top_video('interactions_100.csv'))
13
14 # Profiling memory consumption across multiple runs
15 paths = [
16     'interactions_100.csv',
17     # 'interactions_1000.csv',
18     # 'interactions_10_000.csv'
19 ]
20
21 for p in paths:
22     print("top video: ", get_top_video(p))
23
```

Hình 25: Profiling Memory

Theo dõi RAM theo từng dòng code: Thay vì chỉ xem tổng lượng bộ nhớ cuối cùng, `memory_profiler` cung cấp một bảng thống kê chi tiết, cho thấy mỗi dòng lệnh cụ thể đã "ngốn" thêm bao nhiêu MiB RAM.

Xác định "điểm đen" tài nguyên: Công cụ này giúp lập trình viên chỉ mặt đặt tên chính xác đoạn code nào đang gây lãng phí bộ nhớ — ví dụ như việc khởi tạo một danh sách trung gian không cần thiết hoặc nạp toàn bộ tệp tin vào bộ nhớ thay vì xử lý theo luồng (streaming).

Kết quả nhận được từ code trên:

Line #	Mem usage	Increment	Occurrences	Line Contents
5	93.5 MiB	93.5 MiB	1	@profile
6				def get_top_video(path):
7	94.6 MiB	1.1 MiB	1	interactions = pd.read_csv(path)
8	94.9 MiB	0.4 MiB	1	avg_ratio = interactions.mean(axis=0, skipna=True)
9	95.0 MiB	0.1 MiB	1	return avg_ratio.idxmax()

top video: video_82
 Filename: /home/khoa/Projects/Seminar-Vibe-Coding/Tuan06/Supercharged-Coding-with-Gen-AI/ch14/profile_space.py

Line #	Mem usage	Increment	Occurrences	Line Contents
5	95.0 MiB	95.0 MiB	1	@profile
6				def get_top_video(path):
7	95.2 MiB	0.2 MiB	1	interactions = pd.read_csv(path)
8	95.2 MiB	0.0 MiB	1	avg_ratio = interactions.mean(axis=0, skipna=True)
9	95.2 MiB	0.0 MiB	1	return avg_ratio.idxmax()

The file interactions_10_000.csv contains 10,000 x 10,000 cells of type float64, requiring 8 bytes each, and the expected memory usage is approximately:

Hình 26: Kết quả thực thi

Ví dụ thực tế về sự bùng nổ bộ nhớ: Hãy xét bài toán xử lý một ma trận hoặc tệp dữ liệu có kích thước $10,000 \times 10,000$ phần tử số thực.

Một ma trận kích thước này chứa 100 triệu phần tử.

Nếu mỗi phần tử là một số thực 64-bit (8 bytes), chỉ riêng dữ liệu thô đã chiếm khoảng **800MB RAM**.

Khi cộng thêm các chi phí quản lý đối tượng của Python và các bản sao dữ liệu phát sinh trong quá trình tính toán, con số này có thể dễ dàng vượt ngưỡng 1GB hoặc 2GB, gây áp lực cực lớn lên các hệ thống có cấu hình trung bình.

14.4. Phân tích Max Capacity bằng ChatGPT

Trong kỹ thuật phần mềm, việc hiểu rõ ngưỡng chịu tải của hệ thống là yếu tố then chốt để đảm bảo tính ổn định khi triển khai thực tế. Thay vì đợi đến khi hệ thống gặp sự cố, chúng ta có thể sử dụng các mô hình ngôn ngữ lớn như ChatGPT để dự báo các kịch bản cực đoan.

14.4.1. Khái niệm Max Capacity (Giới hạn xử lý tối đa)

Max Capacity là ngưỡng giới hạn lớn nhất của dữ liệu đầu vào (input size) mà một chương trình hoặc hệ thống có thể xử lý ổn định trong một khung tài nguyên nhất định. Khi vượt quá ngưỡng này, chương trình sẽ rơi vào một trong hai tình trạng đình trệ nghiêm trọng:

Vượt giới hạn thời gian (Runtime Timeout): Chương trình chạy quá chậm so với yêu cầu thực tế. Ví dụ, một API tìm kiếm yêu cầu phản hồi trong dưới 2 giây, nhưng khi lượng dữ liệu đạt 1

triệu bản ghi, thời gian xử lý vọt lên 10 giây. Lúc này, dù chương trình vẫn đang chạy nhưng nó đã vượt quá "giới hạn chịu đựng" của người dùng hoặc hệ thống kết nối.

Cạn kiệt bộ nhớ (Out of Memory - RAM): Đây là giới hạn vật lý cứng. Khi kích thước dữ liệu đầu vào yêu cầu lượng RAM lớn hơn dung lượng tổng khả dụng của máy chủ, hệ điều hành sẽ buộc phải ngắt tiến trình (kill process). Điều này dẫn đến việc chương trình bị sập hoàn toàn và không thể hoàn thành tác vụ.

```
Runtime for fibonacci_recursive(n=10:41:5):
decorator_style_guide.py
fibonacci_recursive(10): 0.000008 seconds
fibonacci_recursive(15): 0.000066 seconds
fibonacci_recursive(20): 0.000726 seconds
fibonacci_recursive(25): 0.007778 seconds
fibonacci_recursive(30): 0.094141 seconds
fibonacci_recursive(35): 1.182233 seconds
fibonacci_recursive(40): 13.296337 seconds
```

Hình 27: Cạn kiệt bộ nhớ (Out of Memory - RAM):

Trong code fibonacci_recursive ta thấy rằng max_capacity của nó chỉ ở mức 40 phần tử.

Vai trò của ChatGPT trong việc xác định Max Capacity:

ChatGPT có khả năng phân tích mã nguồn và thực hiện các phép tính ước lượng logic để giúp lập trình viên dự đoán:

Ước tính toán học: Dựa trên độ phức tạp Big-O, AI có thể tính toán: "Nếu thuật toán là $O(n)$ và chạy $n=1,000$ mất 1 giây, thì với $n=1,000,000$, nó sẽ mất khoảng 11 ngày".

Phân tích phần cứng: Bằng cách cung cấp thông số RAM của server (ví dụ: 8GB), ChatGPT có thể cảnh báo: "Với cấu trúc dữ liệu hiện tại, bạn chỉ có thể xử lý tối đa 500,000 đối tượng trước khi tràn bộ nhớ".

Đề xuất ngưỡng an toàn: AI gợi ý các giới hạn (thresholds) để lập trình viên thiết lập các cơ chế bảo vệ như ngắt xử lý sớm hoặc phân trang dữ liệu.

14.4.2 Phương pháp

Để xác định giới hạn xử lý một cách khoa học mà không cần phải chờ đợi hàng giờ để chạy thử các bộ dữ liệu khổng lồ, chúng ta áp dụng quy trình phối hợp giữa đo lường thực tế (Profiling) và suy luận thông minh (AI Inference). Quy trình này gồm hai bước chiến lược:

Bước 1: Profiling với các tập dữ liệu đầu vào nhỏ (Small-scale Testing) Thay vì thử nghiệm ngay với 1 triệu bản ghi, lập trình viên thực hiện đo lường hiệu năng trên các mẫu dữ liệu tăng dần

(ví dụ: 100, 1.000, 10.000 bản ghi). Việc này giúp thu thập các điểm dữ liệu thực tế về mức độ tiêu thụ RAM và thời gian thực thi (Runtime).

```
paths = ['interactions_100.csv',
         'interactions_1000.csv',
         'interactions_10_000.csv']
for p in paths:
    print("top video: ", get_top_video(p))
```

Hình 28: thu thập các điểm dữ liệu thực tế về mức độ tiêu thụ RAM và thời gian thực thi (Runtime)

Bước 2: Sử dụng ChatGPT để dự báo (Extrapolation) Dựa trên các con số thực tế thu được từ Bước 1, chúng ta cung cấp ngữ cảnh cho ChatGPT để thực hiện các phép tính ngoại suy.

Cấu trúc một Prompt hiệu quả để dự báo Max Capacity thường bao gồm:

- **Mã nguồn của hàm:** Giúp ChatGPT xác định độ phức tạp Big-O ($O(n)$, $O(n^2)$,...).
- **Kết quả profiling thực tế:** Cung cấp các tham số như: "Với $n=10,000$, hàm chạy hết 1.5 giây và tốn 200MB RAM".
- **Giới hạn hệ thống (Constraints):** Nêu rõ ngưỡng chặn của môi trường thực tế, ví dụ: "Server chỉ có tối đa 4GB RAM khả dụng và yêu cầu Runtime không quá 30 giây".

Ví dụ minh họa:

"Tôi có hàm xử lý ảnh đính kèm. Kết quả profiling cho thấy xử lý 100 ảnh mất 5 giây và ngốn 1GB RAM. Với giới hạn 8GB RAM và thời gian tối đa 1 phút, hệ thống của tôi có thể xử lý tối đa bao nhiêu ảnh cùng lúc?"

Thông qua khả năng tính toán logic, ChatGPT sẽ đưa ra con số dự đoán chính xác về **Max Capacity**, giúp bạn thiết lập các cơ chế kiểm soát lỗi (Error Handling) hoặc phân trang dữ liệu trước khi hệ thống thực sự chạm ngưỡng đổ vỡ.

14.4.3 Kết quả thực nghiệm

Để kiểm chứng khả năng dự báo của GenAI, các thử nghiệm thực tế đã được tiến hành trên hai bài toán điển hình về giới hạn thời gian (Runtime) và giới hạn bộ nhớ (Space). Kết quả cho thấy các mô hình như ChatGPT không chỉ suy luận định tính mà còn có khả năng tính toán định lượng với độ chính xác rất cao.


```

Filename: /Users/hila/PycharmProjects/private/supercharge/ch14/profile_space.py
Line #   Mem usage   Increment   Occurrences   Line Contents
=====
5       93.2 MiB     93.2 MiB      1   @profile
6                               def get_top_video(path):
7       93.8 MiB     0.6 MiB      1       interactions = pd.read_csv(path)
8       94.0 MiB     0.2 MiB      1       avg_ratio = interactions.mean(axis=0, skipna=True)
9       94.0 MiB     0.0 MiB      1       return avg_ratio.idxmax()

top video: video_82
Filename: /Users/hila/PycharmProjects/private/supercharge/ch14/profile_space.py
Line #   Mem usage   Increment   Occurrences   Line Contents
=====
5       94.0 MiB     94.0 MiB      1   @profile
6                               def get_top_video(path):
7      124.0 MiB    29.9 MiB      1       interactions = pd.read_csv(path)
8      131.6 MiB     7.6 MiB      1       avg_ratio = interactions.mean(axis=0, skipna=True)
9      131.6 MiB     0.0 MiB      1       return avg_ratio.idxmax()

top video: video_629
Filename: /Users/hila/PycharmProjects/private/supercharge/ch14/profile_space.py
Line #   Mem usage   Increment   Occurrences   Line Contents
=====
5      130.7 MiB    130.7 MiB      1   @profile
6                               def get_top_video(path):
7     1020.4 MiB   889.7 MiB      1       interactions = pd.read_csv(path)
8     1113.4 MiB    93.0 MiB      1       avg_ratio = interactions.mean(axis=0, skipna=True)
9     1113.4 MiB     0.0 MiB      1       return avg_ratio.idxmax()

top video: video_7238

```

Hình 29: Kiểm chứng khả năng dự báo của GenAI

Khả năng của GenAI trong việc ước lượng hiệu năng mang lại giá trị thực tiễn khổng lồ. Thay vì phải tốn nhiều giờ đồng hồ để thử sai (trial and error) trên các bộ dữ liệu lớn, lập trình viên có thể tin tưởng vào các dự báo từ AI để:

Thiết lập các tham số cấu hình hệ thống an toàn.

Thiết kế các cơ chế kiểm soát lỗi (exception handling) khi dữ liệu đầu vào chạm ngưỡng giới hạn.

Đưa ra các quyết định nâng cấp phần cứng hoặc tối ưu thuật toán một cách kịp thời.

14.5. Tối ưu code bằng Chained Prompts

Khi đối mặt với các bài toán tối ưu hóa phức tạp, một câu lệnh (Prompt) đơn lẻ thường không đủ để bao quát hết các khía cạnh từ phân tích, đo lường đến thực thi mã nguồn. Đây là lúc kỹ thuật **Chained Prompt** phát huy sức mạnh vượt trội.

14.5.1. Khái niệm Chained Prompt

Chained Prompt (Chuỗi câu lệnh liên kết) là một kỹ thuật tương tác nâng cao với GenAI, trong đó người dùng không đưa ra yêu cầu cuối cùng ngay lập tức. Thay vào đó, quy trình được chia nhỏ thành một chuỗi các bước logic nối tiếp nhau:

Tính kế thừa: Kết quả đầu ra (Output) của Prompt trước sẽ được sử dụng làm dữ liệu đầu vào (Input) hoặc ngữ cảnh cho Prompt tiếp theo.

Tính chuyên sâu: Mỗi mắt xích trong chuỗi tập trung giải quyết một khía cạnh cụ thể, giúp AI duy trì sự tập trung cao độ và tránh bị "loãng" thông tin.

```
CONTEXT: You are provided with:
1. Python function enclosed with {{{ FUNCTION }}}
2. Runtime profiling enclosed with {{{ PROFILING }}}.
3. Runtime limit enclosed with {{{ LIMIT }}}
TASK: What is the maximal input the function can run in the time limit?

FUNCTION: {{{ ... }}}
PROFILING: {{{ ... }}}
LIMIT: {{{ ... }}}

MAXIMAL INPUT:
```

Hình 30: Chained Prompt

Tại sao Chained Prompt lại quan trọng đối với tối ưu hóa?

Trong bối cảnh SDLC (Vòng đời phát triển phần mềm), tối ưu hóa là một tiến trình có tính trình tự. Việc sử dụng chuỗi liên kết giúp đảm bảo AI hiểu sâu sắc vấn đề trước khi đưa ra giải pháp:

- **Prompt 1 (Phân tích):** Yêu cầu AI đọc mã nguồn và xác định độ phức tạp Big-O hiện tại.
- **Prompt 2 (Đo lường):** Yêu cầu AI sinh mã Profiling dựa trên cấu trúc vừa phân tích ở bước 1.
- **Prompt 3 (Tối ưu):** Sau khi người dùng cung cấp kết quả Profiling, yêu cầu AI đề xuất phương án tối ưu (ví dụ: chuyển từ vòng lặp sang Vectorization) để phá vỡ giới hạn Max Capacity.

Kỹ thuật này giúp giảm thiểu sai lệch (hallucination) của AI và tạo ra những giải pháp tối ưu hóa sát thực tế hơn, vì mỗi bước đi đều dựa trên những dữ liệu đã được xác thực từ bước trước đó.

14.5.2 Tối ưu Runtime

Tối ưu hóa thời gian thực thi (Runtime) không chỉ đơn thuần là tinh chỉnh mã nguồn, mà thường là một cuộc cách mạng về tư duy thuật toán. Minh chứng rõ rệt nhất chính là sự lộn xộn của bài toán tìm số Fibonacci khi áp dụng các kỹ thuật toán học cao cấp.

Prompt 1: TIME COMPLEXITY

Giải thích:

- Hàm gọi lại chính nó 2 lần: `fibonacci_recursive(n-1)` + `fibonacci_recursive(n-2)`
- Mỗi call tạo ra 2 branches, mỗi branch lại tạo 2 branches → cây nhị phân
- Tổng số calls: 2^n → **Exponential complexity**

Prompt 2: MAXIMAL INPUT

Phân tích từ benchmarks:

Input	Runtime	Growth
fib(35)	1.7051s	-
fib(40)	19.5045s	11.4×
fib(41)	~39s	2× (estimated)
fib(42)	~78s	2× (estimated)
fib(43)	~156s	2× × exceeds limit

Reasoning: $O(2^n)$ exponential growth → mỗi step tăng n, runtime gấp đôi. fib(42) ≈ 78s < 100s, fib(43) ≈ 156s > 100s

Comparison:

n	Recursive	Iterative	Fast Doubling
35	1.7s	~0.00001s	~0.00001s
40	19.5s	~0.00001s	~0.00001s
100	× crash	~0.00005s	~0.00001s
100,000	× impossible	~0.05s	~0.00001s

Fast Doubling giảm từ $O(2^n)$ → $O(\log n)$ = hơn 1 triệu lần nhanh hơn! ⚡

Hình 31: Bài toán tìm số Fibonacci khi áp dụng các kỹ thuật toán học cao cấp

Bước nhảy vọt về hiệu năng:

Thuật toán nguyên bản ($O(2^n)$): Như đã phân tích, phương pháp đệ quy thuần túy khiến hệ thống rơi vào trạng thái tê liệt khi n tiến dần đến 50-100.

Thuật toán cải tiến ($O(\log n)$): Bằng cách sử dụng phương pháp **Fast Doubling** (dựa trên các tính chất của ma trận Fibonacci), chúng ta có thể giảm số lượng phép tính từ hàng tỷ xuống chỉ còn vài chục phép tính đơn giản.

Kết quả thực nghiệm kinh ngạc: Với thuật toán Fast Doubling, việc tính toán số **Fibonacci thứ 1,000,000** (một con số khổng lồ có hàng trăm nghìn chữ số) có thể được hoàn thành chỉ trong vòng **vài mili-giây**.

Việc kết hợp khả năng gợi ý thuật toán của GenAI (như yêu cầu ChatGPT đề xuất giải pháp tốt hơn $O(n)$ cho Fibonacci) giúp lập trình viên nhanh chóng tiếp cận được các kỹ thuật tối ưu hóa chuyên sâu mà trước đây thường chỉ xuất hiện trong các cuộc thi lập trình thi đấu.

14.5.3 Tối ưu Memory

Khi đối mặt với các tập dữ liệu khổng lồ (Big Data) vượt quá dung lượng RAM vật lý, chiến lược tối ưu hóa bộ nhớ hiệu quả nhất không phải là mua thêm phần cứng, mà là thay đổi cách thức nạp và xử lý dữ liệu. Kỹ thuật **Chunking** (Chia nhỏ dữ liệu) chính là chìa khóa để giải quyết bài toán này.

Prompt 1: SPACE COMPLEXITY

Giải thích:

- Hàm gọi lại chính nó recursively tạo ra call stack
- Cây recursion có độ sâu tối đa = n (con đường dài nhất)
- Mỗi level trong call stack dùng memory $\rightarrow$ tổng $O(n)$ stack space
- Ví dụ: fib(5) $\rightarrow$ độ sâu stack ≈ 5

Call Stack Depth:

Prompt 2: MAXIMAL INPUT

(Giống kết quả trước - exponential growth $O(2^n)$ $\rightarrow$ fib(42) $\approx 78s < 100s$, fib(43) $\approx 156s > 100s$)

Hình 32: Tối ưu Memory bằng kỹ thuật Chunking

Nguyên lý hoạt động của Chunking: Thay vì sử dụng các phương thức nạp toàn bộ tệp tin vào bộ nhớ (như `pd.read_csv()` mặc định trong Pandas), chúng ta cấu hình chương trình để đọc dữ liệu theo từng khối nhỏ (ví dụ: mỗi lần chỉ đọc 10.000 dòng).

Nạp một phần: Chỉ một lượng nhỏ dữ liệu được đưa vào RAM tại một thời điểm.

Xử lý cục bộ: Thực hiện các tính toán (tính tổng, trung bình, lọc dữ liệu) trên khối đó.

Giải phóng và Lặp lại: Sau khi xử lý xong, dữ liệu cũ được giải phóng để nhường chỗ cho khối tiếp theo.

Tổng hợp: Kết quả cuối cùng là sự hợp nhất từ các kết quả trung gian của từng khối.

Kết quả thực nghiệm:

Trước khi tối ưu: Một tệp dữ liệu 10GB có thể gây sập hệ thống có 8GB RAM ngay lập tức với lỗi *Out of Memory*.

Sau khi áp dụng Chunking: Chương trình có thể xử lý tệp 10GB, 100GB hoặc thậm chí hàng TB dữ liệu chỉ với một lượng RAM cố định cực nhỏ (ví dụ: 500MB).

```
Prompt 3: IMPROVED ALGORITHM (Space Optimized)
Best Solution - Iterative O(1) Space:

def fibonacci_space_optimized(n):
    """Calculate nth Fibonacci number iteratively.
    Time Complexity: O(n)
    Space Complexity: O(1) - NO recursion stack!
    """
    if n <= 1:
        return n

    prev, curr = 0, 1
    for _ in range(2, n + 1):
        prev, curr = curr, prev + curr

    return curr

Comparison:



| Metric      | Recursive | Iterative         | Fast Doubling      |
|-------------|-----------|-------------------|--------------------|
| Time        | $O(2^n)$  | $O(n)$            | $O(\log n)$        |
| Space       | $O(n)$    | $O(1) \checkmark$ | $O(\log n)$        |
| fib(35)     | 1.7s      | 0.00001s          | 0.00001s           |
| fib(100)    | × crash   | 0.0001s           | 0.00001s           |
| Stack Depth | ~100      | 0 (loop)          | $\sim \log(100)=7$ |



Iterative giảm from  $O(n)$  →  $O(1)$  space = chỉ dùng 2 biến thay vì lưu cả call stack! 🚀
```

Hình 33: Kết quả thực nghiệm trước/sau khi tối ưu bằng kỹ thuật Chunking

14.6. Các hướng tối ưu nâng cao

Khi một ứng dụng chuyển từ giai đoạn thử nghiệm sang môi trường vận hành thực tế (Production) với quy mô hàng triệu người dùng, các kỹ thuật tối ưu hóa cơ bản đôi khi là chưa đủ. GenAI đóng vai trò như một kiến trúc sư hệ thống, đề xuất các giải pháp hạ tầng và cấu trúc dữ liệu chuyên sâu để tối đa hóa hiệu suất phần cứng.

Dưới đây là các kỹ thuật then chốt mà GenAI thường xuyên gợi ý cho các hệ thống lớn:

- **Xử lý song song (Parallel Processing & Multithreading):** Thay vì xử lý tuần tự từng tác vụ trên một lõi CPU, GenAI có thể giúp bạn viết mã sử dụng thư viện `multiprocessing` hoặc `concurrent.futures`. Kỹ thuật này cho phép tận dụng tối đa sức mạnh của các CPU đa nhân hiện đại, giúp rút ngắn Runtime đáng kể bằng cách thực hiện nhiều công việc cùng một lúc.
- **Tính toán trên GPU (GPU Computing):** Đối với các bài toán yêu cầu tính toán ma trận khổng lồ hoặc huấn luyện mô hình học sâu, GenAI sẽ đề xuất chuyển đổi từ CPU sang GPU (sử dụng thư viện như `CuPy`, `PyTorch` hoặc `TensorFlow`). Với hàng nghìn lõi xử lý nhỏ, GPU có thể tăng tốc độ tính toán nhanh hơn hàng chục, thậm chí hàng trăm lần so với CPU truyền thống.
- **Ma trận thưa (Sparse Matrix):** Trong các hệ thống gợi ý hoặc xử lý ngôn ngữ tự nhiên, chúng ta thường gặp các ma trận khổng lồ nhưng đa số các phần tử đều bằng 0. Thay vì lưu trữ mọi vị trí (gây lãng phí RAM), GenAI gợi ý sử dụng định dạng Sparse Matrix (như `scipy.sparse`). Kỹ thuật này chỉ lưu trữ các giá trị khác 0, giúp giảm mức chiếm dụng bộ nhớ từ vài GB xuống còn vài MB.
- **Định dạng dữ liệu tối ưu (Parquet vs. CSV):** GenAI thường khuyến nghị thay thế các định dạng văn bản thô (CSV, JSON) bằng các định dạng lưu trữ dạng cột (columnar storage) như **Apache Parquet**. Parquet không chỉ chiếm ít dung lượng ổ cứng hơn nhờ nén hiệu quả mà còn cho phép "tải dữ liệu chọn lọc", giúp tốc độ đọc/ghi dữ liệu trong các hệ thống Big Data nhanh hơn gấp nhiều lần.

Những kỹ thuật này chính là ranh giới giữa một ứng dụng cá nhân và một hệ thống cấp độ doanh nghiệp (Enterprise). Bằng cách sử dụng GenAI để hiện thực hóa các giải pháp này, lập trình viên có thể đảm bảo phần mềm luôn vận hành ổn định, tiết kiệm chi phí hạ tầng và có khả năng mở rộng (scalability) không giới hạn.

14.7. Kết luận

Chương 14 đã khép lại hành trình khám phá về tối ưu hóa hiệu năng — một trong những kỹ năng quan trọng nhất để chuyển đổi từ một người viết mã (coder) thành một kỹ sư phần mềm thực thụ. Qua các nội dung đã thảo luận, chúng ta có thể đi đến những nhận định cốt lõi sau:

Việc xây dựng một hệ thống phần mềm chất lượng cao trong kỷ nguyên hiện đại không còn dựa duy nhất vào kinh nghiệm cá nhân, mà là sự giao thoa giữa ba yếu tố chiến lược:

- **Kiến thức thuật toán nền tảng:** Hiểu rõ về độ phức tạp thời gian (O) và không gian (S) là điều kiện tiên quyết. Nếu không có tư duy thuật toán đúng đắn (như chuyển đổi từ đệ quy sang Fast Doubling), ngay cả những siêu máy tính mạnh nhất cũng có thể bị khuất phục bởi những bài toán đơn giản.
- **Công cụ Profiling thực nghiệm:** Các thư viện như `time` và `memory_profiler` đóng vai trò là "chiếc kính hiển vi", giúp chúng ta soi xét chính xác từng dòng lệnh để tìm ra những điểm nghẽn (bottlenecks) thực tế, thay vì chỉ dự đoán cảm tính.
- **Sức mạnh của GenAI:** Các mô hình như ChatGPT và GitHub Copilot không chỉ là trợ lý viết code, mà còn là chuyên gia tư vấn hiệu năng. Chúng hỗ trợ dự báo giới hạn xử lý (**Max Capacity**), đề xuất các kỹ thuật tối ưu hóa chuyên sâu (như **Chunking** hay **Vectorization**) và thực thi các chuỗi lệnh phức tạp (**Chained Prompt**) để cải thiện hệ thống một cách nhanh chóng.

Sự kết hợp hoàn hảo này giúp nâng cao đáng kể khả năng thiết kế và tối ưu hóa hệ thống. Điều này đặc biệt có ý nghĩa sống còn trong các bài toán **Xử lý dữ liệu lớn (Big Data)** và **Trí tuệ nhân tạo (AI)**, nơi mà mỗi mili-giây và mỗi MB bộ nhớ tiết kiệm được đều trực tiếp tạo ra giá trị kinh tế và nâng tầm trải nghiệm người dùng.

CHƯƠNG 15: Đưa GenAI vào Hoạt động: Ghi nhật ký, Giám sát và Lỗi

15.1. Giới thiệu về logging, monitoring và cơ chế phát sinh lỗi

15.1.1. Tổng quan

Khi một hệ thống phần mềm Python được triển khai lên môi trường production và phục vụ người dùng thực tế, việc đảm bảo hệ thống hoạt động ổn định là yêu cầu bắt buộc. Không chỉ dừng lại ở việc “chạy được”, hệ thống cần phải:

- Theo dõi hành vi trong quá trình thực thi
- Phát hiện lỗi và bất thường
- Phân tích nguyên nhân khi sự cố xảy ra

Chính vì vậy, ba kỹ thuật quan trọng trong giai đoạn going-live bao gồm:

- Logging (ghi log)
- Monitoring (giám sát hệ thống)
- Raising Errors (xử lý và phát sinh lỗi)

Ba yếu tố này là một phần thiết yếu trong Software Development Life Cycle (SDLC), giúp đảm bảo chất lượng phần mềm và hỗ trợ quá trình vận hành lâu dài.

15.1.2. Logging (Ghi log)

Khái niệm

Logging là quá trình ghi lại các thông tin về hoạt động của chương trình dưới dạng các bản ghi (log). Các log này đóng vai trò như một “nhật ký hệ thống”, giúp lập trình viên:

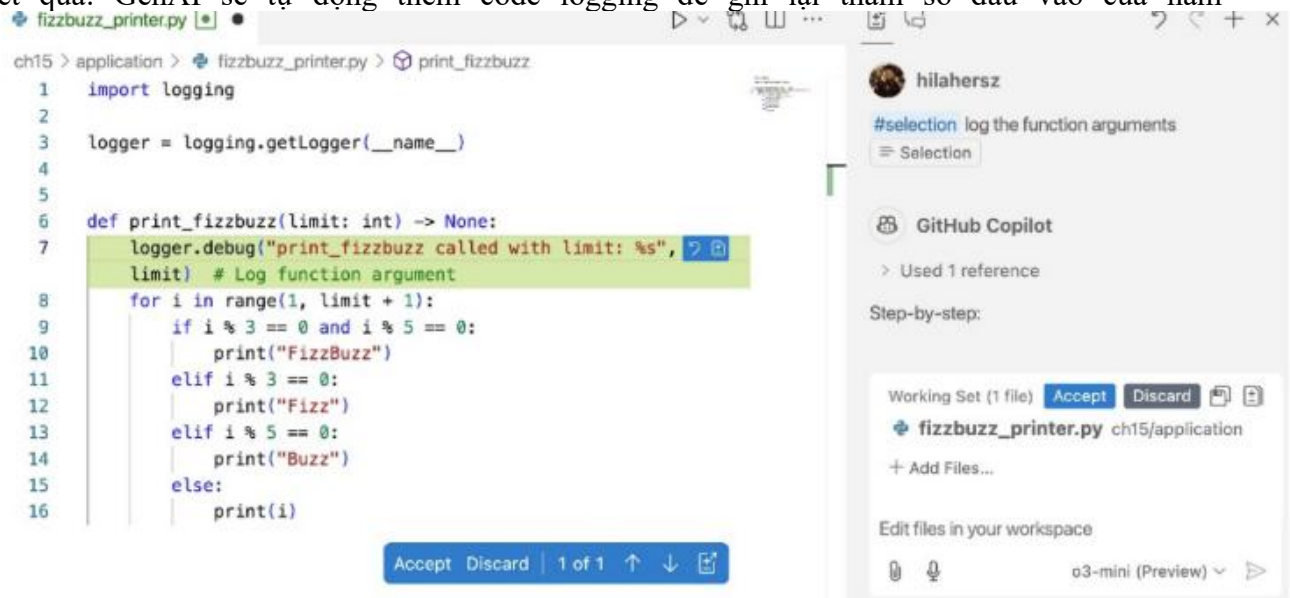
- Theo dõi luồng thực thi
- Debug lỗi
- Phân tích hành vi hệ thống

Ví dụ prompt với GenAI

Thay vì tự viết log, ta có thể dùng GitHub Copilot với VS Code bằng cách quét khối đoạn code rồi gõ prompt ví dụ:

```
#selection log the function arguments
```

Kết quả: GenAI sẽ tự động thêm code logging để ghi lại tham số đầu vào của hàm



Hình 34: Quét khối đoạn code rồi gõ prompt

15.1.3. Monitoring (Giám sát hệ thống)

Khái niệm

Monitoring là quá trình thu thập và theo dõi các metrics (chỉ số) của hệ thống trong quá trình hoạt động.

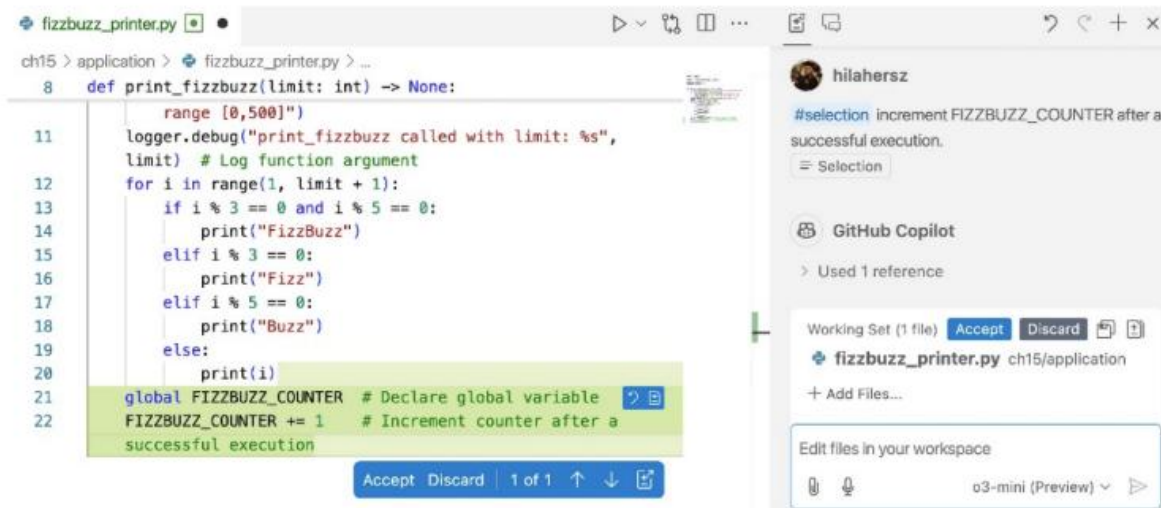
Ví dụ sử dụng Copilot trong VS Code để tạo biến global giúp đếm số lần gọi hàm

Prompt:

```
#selection increment FIZZBUZZ_COUNTER after a successful execution
```

Kết quả: AI tự tạo biến global FIZZBUZZ_COUNTER tăng lên mỗi lần gọi hàm

with the o3 mini model:



Hình 35: Kết quả prompt monitoring

Các chỉ số phổ biến

- Một số metrics thường được sử dụng:
- Số lần gọi hàm (function call count)
- Thời gian thực thi
- Tần suất sử dụng chức năng
- Hiệu năng hệ thống

15.1.4. Raising Errors (Xử lý lỗi)

Khái niệm

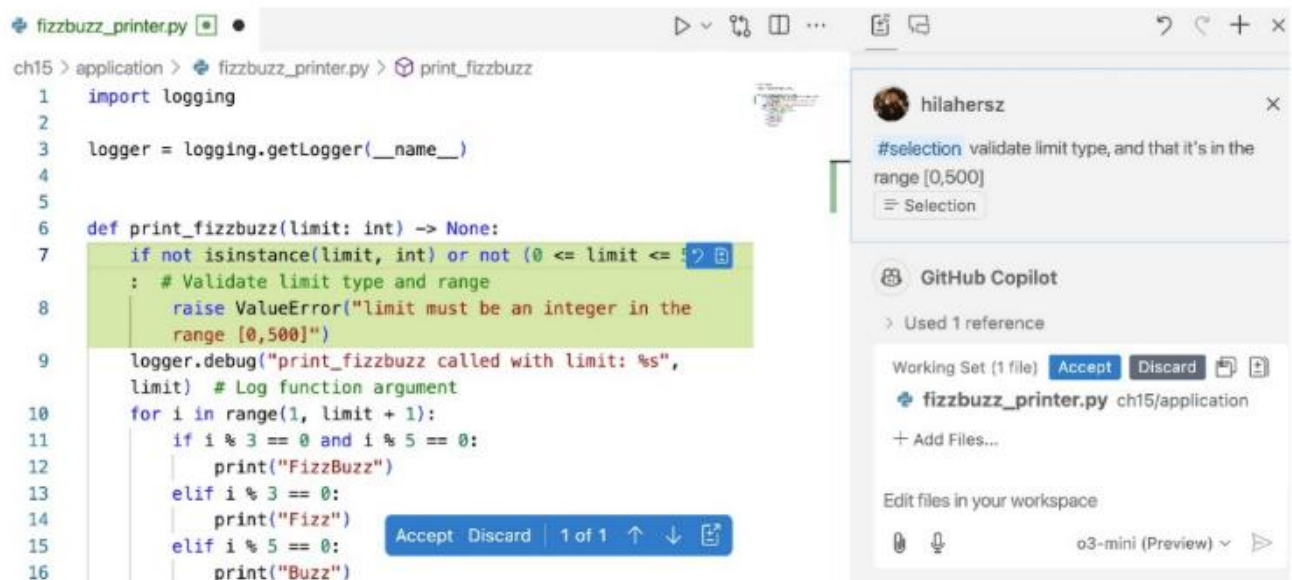
Raising Errors là việc kiểm tra input và chủ động phát sinh lỗi khi dữ liệu không hợp lệ.

Ví dụ validation input trong khoảng từ 0 đến 500 với Copilot trong VS Code

Prompt:

```
#selection validate limit type, and that it is in the range [0,500]
```


Kết quả: AI tự sinh code kiểm tra kiểu dữ liệu int trong phạm vi [0,500]



```
1 import logging
2
3 logger = logging.getLogger(__name__)
4
5
6 def print_fizzbuzz(limit: int) -> None:
7     if not isinstance(limit, int) or not (0 <= limit <= 500):
8         : # Validate limit type and range
9         raise ValueError("limit must be an integer in the
10         range [0,500]")
11
12     logger.debug("print_fizzbuzz called with limit: %s",
13     limit) # Log function argument
14     for i in range(1, limit + 1):
15         if i % 3 == 0 and i % 5 == 0:
16             print("FizzBuzz")
17         elif i % 3 == 0:
18             print("Fizz")
19         elif i % 5 == 0:
20             print("Buzz")
```

Hình 36: Raising Errors (Xử lý lỗi)

15.2. Ứng dụng GenAI trong việc xây dựng các mô hình thiết kế code ở mức cao

15.2.1. Ý tưởng

Thay vì nhúng trực tiếp các chức năng phụ vào trong function, ta sử dụng design pattern để tách riêng chúng.

15.2.2. Decorator Pattern

Decorator là một pattern cho phép:

- Thêm chức năng trước/sau khi gọi hàm
- Không thay đổi code gốc của hàm

Ví dụ:

```
def sample_decorator(func: callable) -> callable:
    def wrapper(*args, **kwargs):
        print("Function is wrapped")
        return func(*args, **kwargs)

    return wrapper
```

Hình 37: Decorator Pattern

```
@sample_decorator
def foo(*num):
    return len(nums)
```

Hình 38: Decorator Pattern (t.t)

sample_decorator:

- Nhận vào: func (hàm gốc)
- Trả về: wrapper (hàm mới)

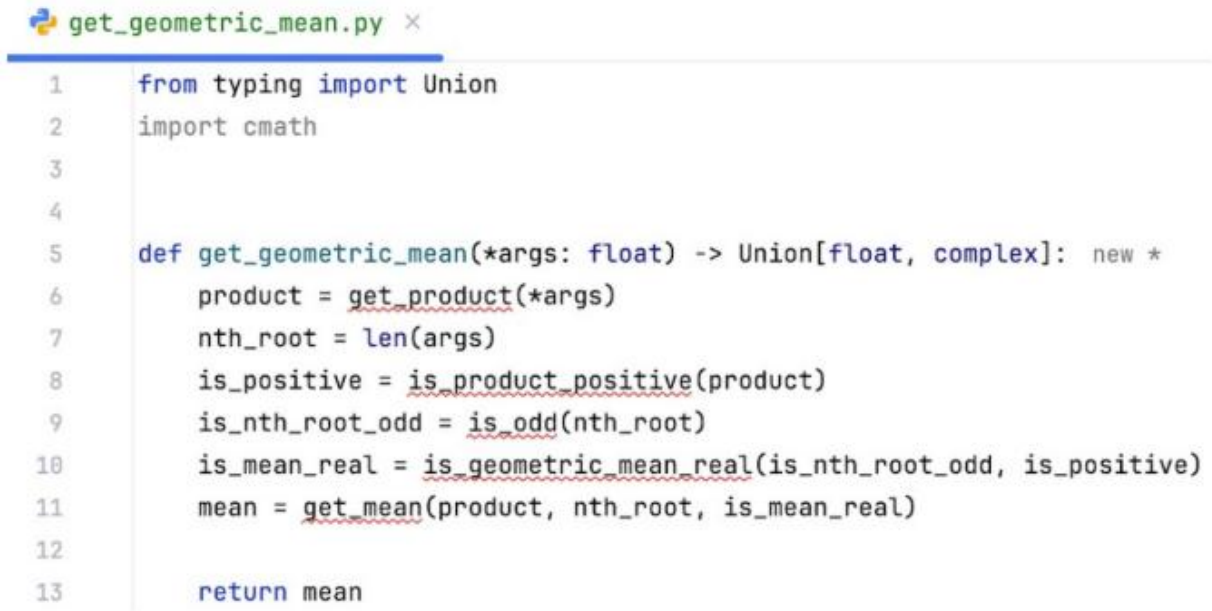
15.2.3. Triển khai CoT ngược với Decorator

Khái niệm Inverse CoT

Inverse Chain-of-Thought (Inverse CoT) là một kỹ thuật prompt trong đó:

- Ta định nghĩa trước các thành phần cấp cao (high-level)
- Nhưng chưa implement chi tiết
- Sau đó để GenAI tự sinh phần còn thiếu

Khác với CoT thông thường (từ dưới lên), Inverse CoT hoạt động theo hướng từ trên xuống



```
1 from typing import Union
2 import cmath
3
4
5 def get_geometric_mean(*args: float) -> Union[float, complex]: new *
6     product = get_product(*args)
7     nth_root = len(args)
8     is_positive = is_product_positive(product)
9     is_nth_root_odd = is_odd(nth_root)
10    is_mean_real = is_geometric_mean_real(is_nth_root_odd, is_positive)
11    mean = get_mean(product, nth_root, is_mean_real)
12
13    return mean
```

Hình 39: Inverse Chain-of-Thought (Inverse CoT)

Ý tưởng

Thay vì viết tất cả trong 1 function thì sẽ khai báo trước các decorator để cho AI tự sinh ra các decorator tương ứng

Ví dụ:

B1: khai báo decorator



```

1  import logging
2
3  logger = logging.getLogger(__name__)
4
5  FIZZBUZZ_COUNTER = 0
6
7
8  @log_function_args new *
9  @increment_counter
10 @validate_args_types_and_limits(0, 500)
11 def print_fizzbuzz(limit: int) -> None:

```

Hình 40: Khai báo decorator

B2: di chuyển chuột lên phía trên và gõ

```
def 1
```

Copilot sẽ hiểu:

“1” → log_function_args

Và tự động generate toàn bộ decorator

Kết quả: Copilot tự gợi ý code dựa trên decorator

```
fizzbuzz_printer.py x
1 import logging
2
3 logger = logging.getLogger(__name__)
4
5 FIZZBUZZ_COUNTER = 0
6
7
8 def log_function_args(func): new *
    def wrapper(*args, **kwargs):
        logger.info(f"Function {func.__name__} called with args: {args}, kwargs: {kwargs}")
        return func(*args, **kwargs)
    return wrapper
9
10
11 @log_function_args new *
12 @increment_counter
13 @validate_args_types_and_limits(0, 500)
14 def print_fizzbuzz(limit: int) -> None:
15     for i in range(1, limit + 1):
```

Hình 41: Copilot tự gợi ý code dựa trên decorator

15.3. Ứng dụng CoT ngược với ChatGPT và OpenAI

15.3.1. Khái niệm

GitHub Copilot là công cụ phù hợp nhất để áp dụng Inverse CoT do hỗ trợ thao tác trực tiếp trong IDE. Tuy nhiên, cùng kỹ thuật này vẫn có thể áp dụng với ChatGPT và OpenAI API, bằng cách xây dựng prompt theo cấu trúc rõ ràng.

Khác với Copilot (dựa vào vị trí con trỏ), ChatGPT và OpenAI API yêu cầu:

- Cung cấp toàn bộ đoạn code chưa hoàn chỉnh
- Sử dụng prompt có cấu trúc theo nguyên tắc five S's (đã giới thiệu ở các chương trước)

15.3.2. Ứng dụng CoT ngược với ChatGPT

Với ChatGPT, thay vì thao tác như Copilot, ta sử dụng CoT prompting tiêu chuẩn, bằng cách:

Cung cấp context (mô tả bài toán)

- Đưa ra single task (nhiệm vụ cụ thể)
- Đưa vào đoạn code chưa hoàn chỉnh

- Thêm lead-in cue (tín hiệu để model sinh code)

Ví dụ

```
CONTEXT: You are provided with a partial Python script in {{{FIZZBUZZ_PRINTER}}}, where some of the code is missing
TASK: Implement the decorators with the missing implementation in the code with the same style of the project files

FIZZBUZZ_PRINTER: {{{
import logging

logger = logging.getLogger(__name__)

FIZZBUZZ_COUNTER = 0

@log_function_args
@increment_counter
@validate_args_types_and_limits(0, 500)
def print_fizzbuzz(limit: int) -> None:
    for i in range(1, limit + 1):
        if i % 3 == 0 and i % 5 == 0:
            print("FizzBuzz")
        elif i % 3 == 0:
            print("Fizz")
        elif i % 5 == 0:
            print("Buzz")
        else:
            print(i)

}}}
```

CODE:

Hình 42: Ứng dụng CoT ngược với ChatGPT

Kết quả:

```

def log_function_args(func: callable) -> callable:
    def wrapper(*args, **kwargs):
        logger.info(
            f"Calling function '{func.__name__}' with args: {args} and
kwargs: {kwargs}"
        )
        return func(*args, **kwargs)
    return wrapper

def increment_counter(func: callable) -> callable:
    def wrapper(*args, **kwargs):
        global FIZZBUZZ_COUNTER
        FIZZBUZZ_COUNTER += 1
        logger.info(f"FIZZBUZZ_COUNTER incremented to {FIZZBUZZ_COUNTER}")
        return func(*args, **kwargs)
    return wrapper

def validate_args_types_and_limits(min_limit: int, max_limit: int):
    def decorator(func: callable) -> callable:
        def wrapper(*args, **kwargs):
            # Use inspect to bind the arguments to the function's
signature
            sig = inspect.signature(func)
            bound_args = sig.bind(*args, **kwargs)
            bound_args.apply_defaults()
            # Expecting the argument to be named 'limit'

```

Hình 43: Kết quả ứng dụng CoT ngược với ChatGPT

```

        if 'limit' not in bound_args.arguments:
            raise ValueError("Missing required argument 'limit'.")
        limit = bound_args.arguments['limit']
        if not isinstance(limit, int):
            raise TypeError("Argument 'limit' must be an integer.")
        if not (min_limit <= limit <= max_limit):
            raise ValueError(f"Argument 'limit' must be between {min_
limit} and {max_limit}.")
        return func(*args, **kwargs)
    return wrapper
return decorator

```

Hình 44: Các decorator được sinh ra

Khi sử dụng prompt trên, ChatGPT sẽ sinh đầy đủ các decorator:

- `log_function_args`
- `increment_counter`
- `validate_args_types_and_limits`

Các decorator này:

- Thực hiện logging tham số hàm
- Tăng biến đếm toàn cục
- Kiểm tra kiểu dữ liệu và phạm vi input

Ngoài ra, implementation có thể:

- Sử dụng `inspect.signature` để bind tham số
- Bao gồm `@functools.wraps` để giữ metadata của hàm

15.3.3. Ứng dụng CoT ngược với OpenAI

Cách làm với OpenAI API tương tự ChatGPT, nhưng prompt được tách thành:

- **System prompt** → mô tả context + nhiệm vụ
- **User prompt** → chứa code chưa hoàn chỉnh

Ví dụ System Prompt

```
SURROUND = "You are provided with a partial Python script in {{{FIZZBUZZ_PRINTER}}}, where some of the code is missing.  
SINGLE_TASK = "Implement the decorators with the missing implementation in the code."
```

Hình 45: Ví dụ System Prompt

Ví dụ User Prompt

```
def get_user_prompt(script_path: str) -> str:  
    with open(script_path, 'r') as file:  
        incomplete_code = file.read()  
  
    return f"""  
    FIZZBUZZ_PRINTER: {{{{{{ {incomplete_code} }}}}}}  
  
    CODE:  
    """  
    hilahersz, 7 months ago * ch15: Going live; logging, monitoring and error...
```

Hình 46: Ví dụ User Prompt

Nguyên lý hoạt động

- Đoạn code chưa hoàn chỉnh được đưa vào prompt
- Từ khóa CODE: đóng vai trò lead-in cue (tín hiệu để model sinh code)
- Model sẽ sinh phần code còn thiếu (decorators)

Việc áp dụng Inverse CoT với ChatGPT và OpenAI API cho thấy:

- Có thể đạt kết quả tương tự Copilot
- Cần sử dụng prompt có cấu trúc rõ ràng
- Output có thể khác về style nhưng vẫn đảm bảo chức năng

15.4. Sử dụng Few-shot Learning và Fine-tuning làm Style Guide

Mặc dù các decorator được sinh bởi GenAI (Copilot, ChatGPT, OpenAI API) đã đáp ứng đúng chức năng, nhưng chúng có thể không thống nhất về style code.

Ví dụ:

- Cách format logging với các tham số đầu vào khác nhau
- Có hoặc không dùng `@functools.wraps`

Cả 2 ví dụ trên đều có thể áp dụng few-shot learning hoặc fine-tuning để sinh code theo style mong muốn.

15.4.1. Few-shot learning với GitHub Copilot

Ý tưởng

Với Copilot, ta có thể cung cấp một file style guide chứa các ví dụ code mẫu.

Copilot sẽ:

- Học style từ file này
- Áp dụng style đó khi sinh code mới

Ví dụ:

Mở file `style_guide_decorator.py` và copy toàn bộ nội dung vào session làm việc sau đó quay lại file chính (`print_fizzbuzz.py`) rồi áp dụng lại Inverse CoT

```
import logging
import time
from functools import wraps
from typing import Any

logger: logging.Logger = logging.getLogger(__name__)

def time_it(func: callable) -> callable:
    @wraps(func)
    def wrapper(*args, **kwargs):
        start_time: float = time.time()
        res: Any = func(*args, **kwargs)
        end_time: float = time.time()
        logger.info(
            "Function called.",
            extra={
                "function": func.__name__,
                "args": args,
                "kwargs": kwargs,
                "error": "",
                "timing": f"{end_time - start_time} seconds"})
        return res
    return wrapper

@time_it
def my_func(a: int, b: int) -> int:
    return a + b
```

Hình 47: file style guide chứa các ví dụ code mẫu

Kết quả: ChatGPT sẽ sinh decorator theo đúng style của ví dụ sử dụng @wraps và áp dụng format logging giống mẫu.

```
def log_function_args(func: callable) -> callable: new *
    def wrapper(*args, **kwargs):
        message: str = "Function called."
        logger.info(message,
                      extra={"function": func.__name__,
                            "args": args,
                            "kwargs": kwargs,
                            "error": "",
                            "timing": ""})
        return func(*args, **kwargs)

    return wrapper

@log_function_args 2 usages new *
@increment_counter
@validate_args_types_and_limits(0, 500)
def print_fizzbuzz(limit: int) -> None:
```

Hình 48: ChatGPT sẽ sinh decorator theo đúng style của ví dụ

15.4.2. Few-shot learning với ChatGPT

Ý tưởng

Với ChatGPT, ta cần cung cấp:

- Ví dụ về input (INCOMPLETE_CODE)
- Ví dụ về output (COMPLETE_CODE)

Ví dụ

CONTEXT: You are provided with a partial Python script enclosed with {{{FIZZBUZZ_PRINTER}}} where some of the code is missing, and examples of a good implementation enclosed with {{{ EXAMPLES }}}

TASK: Implement the decorators with the missing implementation in the code while following the style guide.

EXAMPLES:

INCOMPLETE_CODE: {{{...omitted for brevity...}}}

COMPLETE_CODE: {{{...omitted for brevity...}}}

FIZZBUZZ_PRINTER: {{{...omitted for brevity...}}}

CODE:

```
INCOMPLETE_CODE: {{{
    hilahersz, 7 months ago • ch15: Going live; logging, monitoring and error...
import logging
import time

logger: logging.Logger = logging.getLogger(__name__)

@time_it
def my_func(a: int, b: int) -> int:
    return a + b
}}}
```

Hình 49: Ví dụ về input (INCOMPLETE_CODE)

```

COMPLETE_CODE: {{{

import logging
import time
from functools import wraps
from typing import Any

logger: logging.Logger = logging.getLogger(__name__)

def time_it(func: callable) -> callable:
    @wraps(func)
    def wrapper(*args, **kwargs):
        start_time: float = time.time()
        res: Any = func(*args, **kwargs)
        end_time: float = time.time()
        logger.info(
            "Function called.",
            extra={
                "function": func.__name__,
                "args": args,
                "kwargs": kwargs,
                "error": "",
                "timing": f"{end_time - start_time} seconds"})
        return res
    return wrapper

@time_it
def my_func(a: int, b: int) -> int:
    return a + b
}}}}

```

Hình 50: Ví dụ về output (COMPLETE_CODE)

```

FIZZBUZZ_PRINTER: {{{
import logging

logger = logging.getLogger(__name__)

FIZZBUZZ_COUNTER = 0

@log_function_args
@increment_counter
@validate_args_types_and_limits(0, 500)
def print_fizzbuzz(limit: int) -> None:
    for i in range(1, limit + 1):
        if i % 3 == 0 and i % 5 == 0:
            print("FizzBuzz")
        elif i % 3 == 0:
            print("Fizz")
        elif i % 5 == 0:
            print("Buzz")
        else:
            print(i)
}}}}

```

Hình 51: FizzBuzz Printer

Kết quả: ChatGPT sẽ sinh decorator theo đúng style của ví dụ sử dụng @wraps và áp dụng format logging giống mẫu

ChatGPT o3-mini-high

Share



```

def log_function_args(func: Callable) -> Callable:
    @wraps(func)
    def wrapper(*args: Any, **kwargs: Any) -> Any:
        logger.info(
            "Function called.",
            extra={
                "function": func.__name__,
                "args": args,
                "kwargs": kwargs,
                "error": ""
            }
        )
        return func(*args, **kwargs)
    return wrapper

```

Hình 52: ChatGPT sẽ sinh decorator

15.4.3. Fine-tuning với OpenAI API

Ý tưởng

Fine-tuning là quá trình huấn luyện model với dữ liệu mẫu giúp model tự động sinh code theo style mong muốn

Ví dụ về xây dựng một mẫu huấn luyện với:

- System prompt: mô tả ngữ cảnh và nhiệm vụ
- User prompt: chứa đoạn code chưa hoàn chỉnh
- Assistant response: kết quả mong muốn (code hoàn chỉnh theo style định sẵn)

Mỗi phản hồi của assistant được gán `weight = 1`, thể hiện đây là output chuẩn mà model cần học theo.

```
{
  "messages": [
    {
      "role": "system",
      "content": "You will be provided with a Python function signature enclosed with {{{ FUNCTION }}}. Your task is to implement it."
    },
    {
      "role": "user",
      "content": "FUNCTION: {{{def get_arithmetic_mean(a, b)}}}\n CODE: ",
    },
    {
      "role": "assistant",
      "content": "def get_arithmetic_mean(a, b): \n    return (a+b)/2", "weight": 1
    }
  ]
}
```

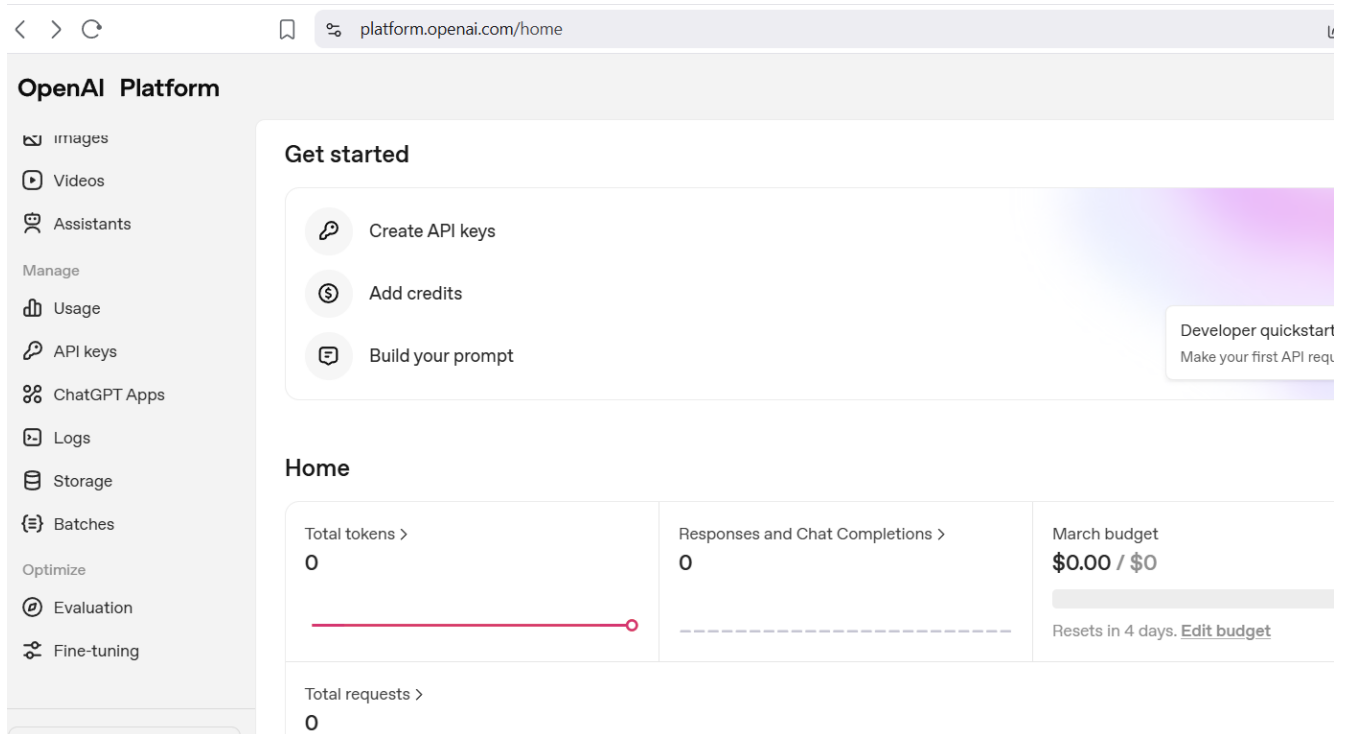
Hình 53: output chuẩn mà model cần học theo

Quy trình upload và huấn luyện mô hình fine-tuning

Sau khi chuẩn bị file dữ liệu huấn luyện ở định dạng JSONL như trên, bước tiếp theo là tiến hành upload và huấn luyện mô hình trên nền tảng OpenAI.

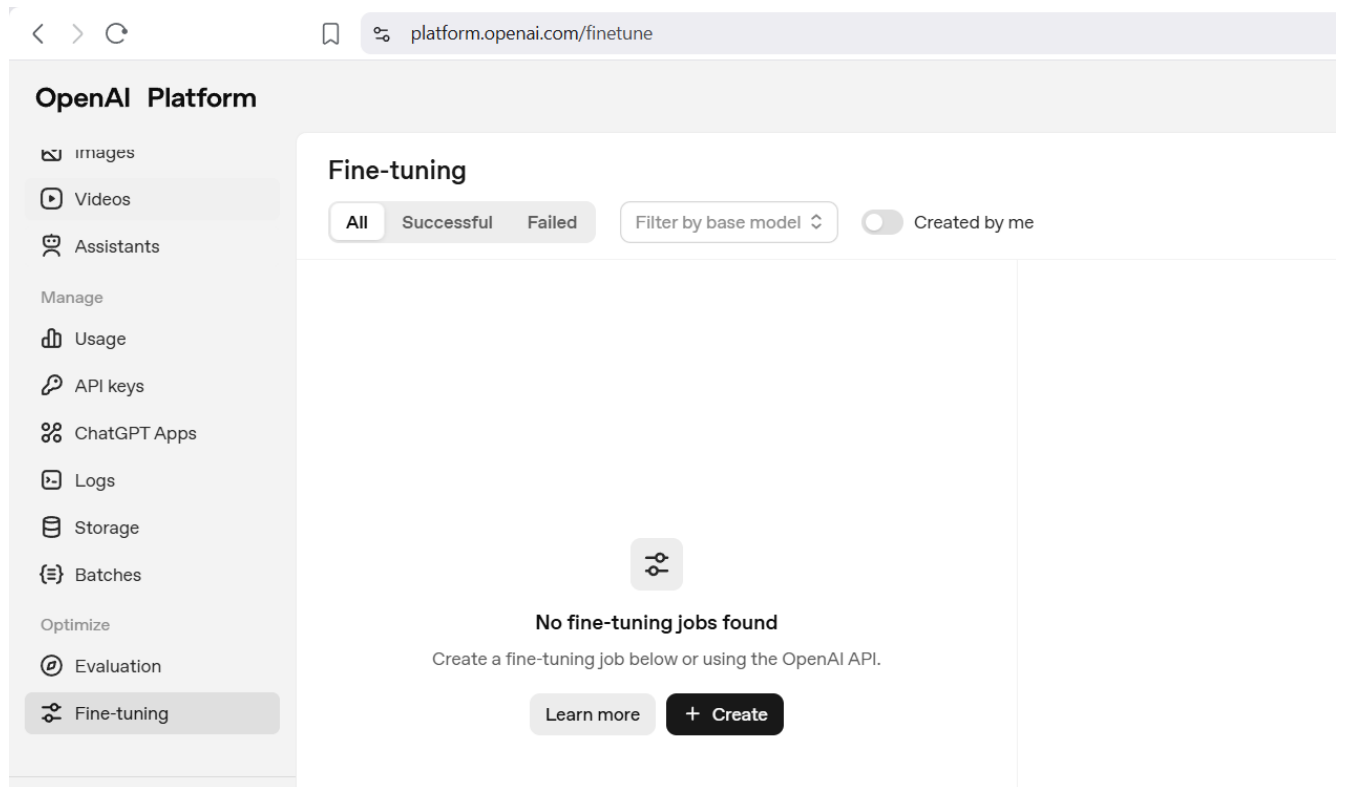
Cụ thể, quy trình được thực hiện như sau:

Truy cập trang platform.openai.com



Hình 54: Quy trình upload và huấn luyện mô hình fine-tuning (p1)

Chọn mục Dashboard → Fine-tune



Hình 55: Quy trình upload và huấn luyện mô hình fine-tuning (p2)

Upload file JSONL đã chuẩn bị

Create a fine-tuned model

Method
Specify the method to be used for fine-tuning.

Supervised

Base Model

gpt-4o-mini-2024-07-18

Suffix
Add a custom suffix that will be appended to the output model name.

decorator-style

Seed
The seed controls the reproducibility of the job. Passing in the same seed and job parameters should produce the same results, but may differ in rare cases. If a seed is not specified, one will be generated for you.

Random

Training data
Add a jsonl file to use for training. By providing the file, you confirm that you have the rights to use the data.

☐ Upload new ☒ Select existing [Browse files ↗](#)

file-FxKnfmvzkV8FiwQjYd589V

[Learn about fine-tuning ↗](#) [Cancel](#) [Create](#)

Hình 56: Quy trình upload và huấn luyện mô hình fine-tuning (p3)

Sau khi upload hoàn tất, hệ thống sẽ bắt đầu quá trình huấn luyện (fine-tuning).

Theo tài liệu, quá trình này thường mất khoảng 15 phút để hoàn thành

So sánh mô hình sau khi fine-tuning

Sau khi quá trình fine-tuning kết thúc:

Trạng thái job sẽ chuyển sang Succeeded

MODEL	
ft:gpt-4o-mini-2024-07-18:pazpaz-the-coder:decorator-style:AydMPYCg	
Status	Succeeded
Job ID	ftjob-zDa73ZywRRDInw0HngNeAfUy
Training Method	Supervised
Suffix	decorator-style
Base model	gpt-4o-mini-2024-07-18
Output model	ft:gpt-4o-mini-2024-07-18:pazpaz-the-coder:decorator-style:AydMPYCg
Created at	Feb 8, 2025, 1:24 PM

Hình 57: fine-tuned mode

Một mô hình mới (fine-tuned model) sẽ được tạo

Mô hình này có thể được sử dụng trong playground (để test) hoặc tích hợp trực tiếp vào ứng dụng thông qua API

Việc so sánh giữa: Base model (mô hình gốc) và Fine-tuned model cho phép đánh giá mức độ cải thiện về hiệu năng và chất lượng của mô hình

So sánh

System prompt:

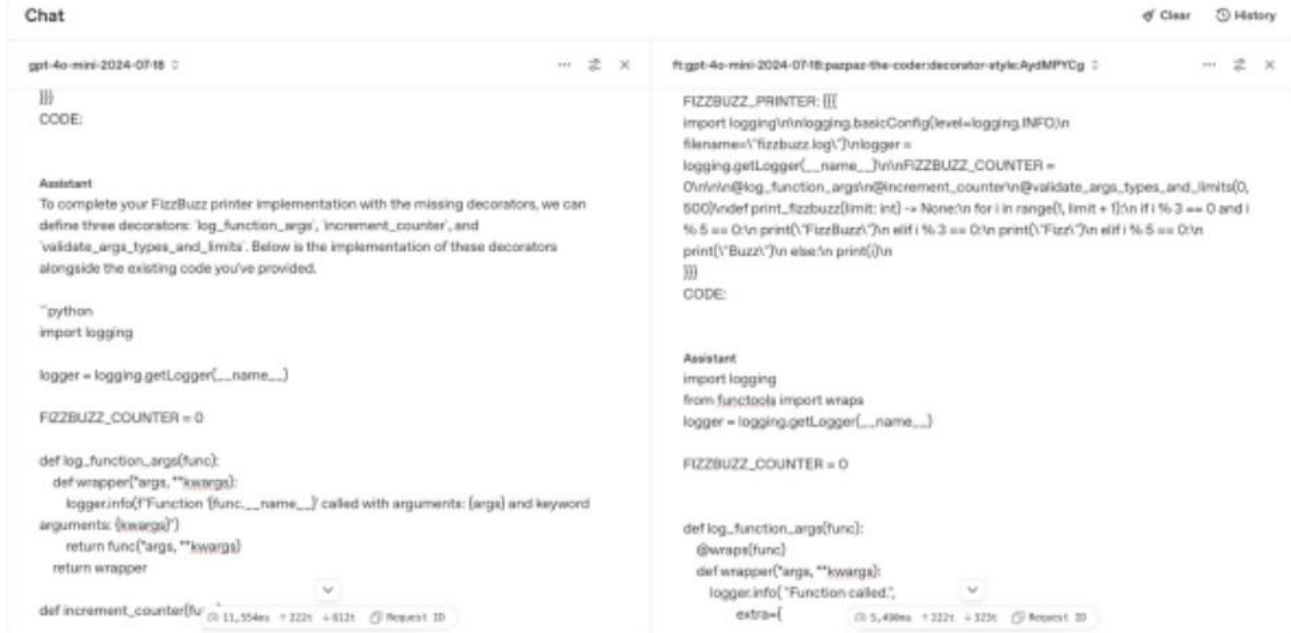
```
You are provided with a partial Python script in {{{FIZZBUZZ_PRINTER}}},  
where some of the code is missing. Your task is to implement the  
decorators with the missing implementation in the code.
```

User prompt:

```
FIZZBUZZ_PRINTER: {{{ ...omitted for brevity... }}}  
CODE:
```

Kết quả khi chạy prompt trên:

- Base model: Có thể sinh code nhưng chưa đúng style mong muốn, Output có thể dài dòng hoặc không nhất quán
- Fine-tuned model: Trả về chỉ code (không thêm giải thích) Áp dụng đúng logging style đã huấn luyện bao gồm @functools.wraps, format code nhất quán



Hình 58: Kết quả khi chạy prompt

Quan sát từ kết quả: Output của fine-tuned model bắt đầu trực tiếp bằng code, style logging được áp dụng đồng nhất phản ánh rõ ảnh hưởng của dữ liệu huấn luyện.

Điều này cho thấy fine-tuning giúp: kiểm soát format output, duy trì style code ổn định, giảm sự khác biệt giữa các lần sinh code.

15.5. Kết luận

Trong chương này, chúng ta đã tìm hiểu các tác vụ quan trọng khi đưa phần mềm vào production như logging, monitoring và xử lý lỗi. Tuy nhiên, khi sử dụng GenAI với các prompt đơn giản, những chức năng này thường bị chèn trực tiếp vào hàm, làm code trở nên phức tạp và vi phạm nguyên tắc single responsibility .

Giải pháp cho vấn đề này là sử dụng Inverse Chain-of-Thought (CoT) prompting với GitHub Copilot, kết hợp với decorator pattern nhằm tách biệt các chức năng như logging, đo lường và kiểm tra dữ liệu khỏi logic chính.

Ngoài ra, để kiểm soát style code, có thể áp dụng few-shot learning hoặc fine-tuning, giúp GenAI sinh code nhất quán và giảm công sức chỉnh sửa.

Tóm lại, hiệu quả của GenAI phụ thuộc vào cách thiết kế prompt và định hướng mô hình, từ đó giúp tạo ra code rõ ràng, có cấu trúc và phù hợp với môi trường production.

CHƯƠNG 16: Architecture, Design, and the Future

16.1. Giới thiệu

Trong bối cảnh công nghệ phát triển nhanh chóng, đặc biệt là sự bùng nổ của trí tuệ nhân tạo tạo sinh (Generative AI - GenAI), ngành công nghiệp phần mềm đang trải qua những thay đổi mang tính cách mạng. Chương 16 tập trung phân tích sự phát triển của GenAI, tác động của nó đến kinh tế phần mềm, vai trò của lập trình viên, cũng như dự đoán tương lai của ngành software engineering.

Mục tiêu của chương là giúp người đọc hiểu rõ cách mà GenAI đang định hình lại toàn bộ quy trình phát triển phần mềm, từ việc viết code đến thiết kế hệ thống và quản lý dự án.

16.2. Sự phát triển nhanh chóng của GenAI

Sự can thiệp của GenAI vào quy trình phát triển phần mềm không chỉ dừng lại ở mức độ kỹ thuật mà còn tạo ra những thay đổi sâu sắc về mặt kinh tế học. Trong mô hình kinh tế phần mềm truyền thống, chi phí chủ yếu tập trung vào **thời gian của con người** (Man-months) — yếu tố đắt đỏ và khó mở rộng nhất. GenAI đang phá vỡ cấu trúc này thông qua các tác động then chốt:

- **Tối ưu hóa chi phí sản xuất:** Nhờ khả năng sinh mã tự động, viết tài liệu và kiểm thử nhanh chóng, GenAI giúp giảm đáng kể số giờ công cần thiết cho các tác vụ lặp đi lặp lại. Điều này cho phép các doanh nghiệp phát triển các tính năng phức tạp với nguồn lực ít hơn, từ đó giảm giá thành sản phẩm hoặc tái đầu tư vào các giá trị sáng tạo cốt lõi.
- **Rút ngắn thời gian đưa sản phẩm ra thị trường (Time-to-Market):** Trong một thị trường cạnh tranh khốc liệt, tốc độ là yếu tố sống còn. GenAI giúp tăng tốc chu kỳ phát triển từ ý tưởng (Idea) đến sản phẩm khả thi tối thiểu (MVP). Việc có thể tạo ra các bản mẫu (Prototypes) chỉ trong vài giờ thay vì vài tuần giúp doanh nghiệp phản ứng linh hoạt với các phản hồi của thị trường.
- **Thay đổi cán cân bảo trì:** Chi phí bảo trì phần mềm thường chiếm tới 60-80% tổng chi phí vòng đời dự án. GenAI hỗ trợ phân tích mã nguồn cũ (Legacy code), dự đoán lỗi và đề xuất các phương án refactoring tự động, giúp giảm thiểu "nợ kỹ thuật" (Technical Debt) và chi phí vận hành dài hạn.
- **Dân chủ hóa phát triển phần mềm:** Với sự hỗ trợ của AI, rào cản gia nhập ngành lập trình đang dần thấp xuống. Những người có tư duy logic tốt nhưng chưa tinh thông mọi ngôn ngữ lập trình vẫn có thể tham gia vào quá trình xây dựng giải pháp, tạo ra một lực lượng lao động mới đa dạng và giàu tính thực tiễn hơn.

GenAI đang chuyển dịch ngành phần mềm từ mô hình "thâm dụng lao động" sang mô hình "thâm dụng tri thức và công nghệ". Đây là cơ hội vàng để các doanh nghiệp và cá nhân bứt phá về năng suất, tạo ra nhiều giá trị hơn trong cùng một khoảng thời gian.

16.3. Tác động đến kinh tế phần mềm

Sự can thiệp của GenAI đã và đang tái cấu trúc lại toàn bộ các biến số kinh tế cốt lõi của ngành công nghiệp phần mềm. Việc thay đổi mối quan hệ giữa chi phí, năng suất và quy mô đã tạo ra

một hiệu ứng dây chuyền, đưa phần mềm trở thành mạch máu không thể thiếu của mọi hoạt động kinh tế.

Dưới đây là bốn trụ cột thay đổi mô hình kinh tế trong kỷ nguyên mới:

- **Tăng trưởng năng suất đột phá:** Nhờ sự hỗ trợ của các mô hình ngôn ngữ lớn và các trợ lý mã nguồn, lập trình viên có thể bỏ qua các bước viết mã lặp lại (boilerplate code) để tập trung hoàn toàn vào logic nghiệp vụ và giải quyết các bài toán phức tạp. Hiệu suất cá nhân được nhân lên gấp nhiều lần, giúp rút ngắn chu kỳ phát triển sản phẩm từ hàng tháng xuống còn hàng tuần.
- **Tối ưu hóa chi phí đầu tư:** So với chi phí nhân sự trình độ cao — vốn là khoản chi lớn nhất trong phát triển phần mềm truyền thống — việc trang bị các công cụ GenAI có mức phí thấp hơn đáng kể nhưng lại mang lại hiệu quả thặng dư khổng lồ. Điều này cho phép các doanh nghiệp vận hành tinh gọn hơn mà vẫn đạt được mục tiêu công nghệ đề ra.
- **Sự bùng nổ về cung phần mềm:** Khi rào cản kỹ thuật được hạ thấp nhờ AI, việc hiện thực hóa ý tưởng thành ứng dụng trở nên dễ dàng và phổ biến hơn. Hệ quả là số lượng giải pháp phần mềm trên thị trường tăng vọt, đáp ứng được cả những nhu cầu ngách (niche markets) vốn trước đây bị bỏ qua vì chi phí phát triển quá cao.
- **Kích cầu thị trường công nghệ:** Theo quy luật kinh tế, khi giá thành giảm và giá trị sử dụng tăng, nhu cầu sẽ bùng nổ. Nhiều doanh nghiệp trước đây e dè với ngân sách chuyển đổi số hiện đã sẵn sàng đầu tư mạnh mẽ vào phần mềm để tối ưu hóa quy trình kinh doanh, vì họ nhận thấy tỷ lệ hoàn vốn (ROI) rõ ràng và nhanh chóng hơn.

Những thay đổi này đang biến phần mềm từ một sản phẩm "xa xỉ" hay "hỗ trợ" thành một yếu tố thiết yếu và phổ quát. Trong nền kinh tế số, phần mềm không còn chỉ là công cụ, mà đã trở thành năng lực cạnh tranh cốt lõi, quyết định sự sinh tồn của mọi tổ chức trên thị trường.

16.4. Mức độ chấp nhận của GenAI

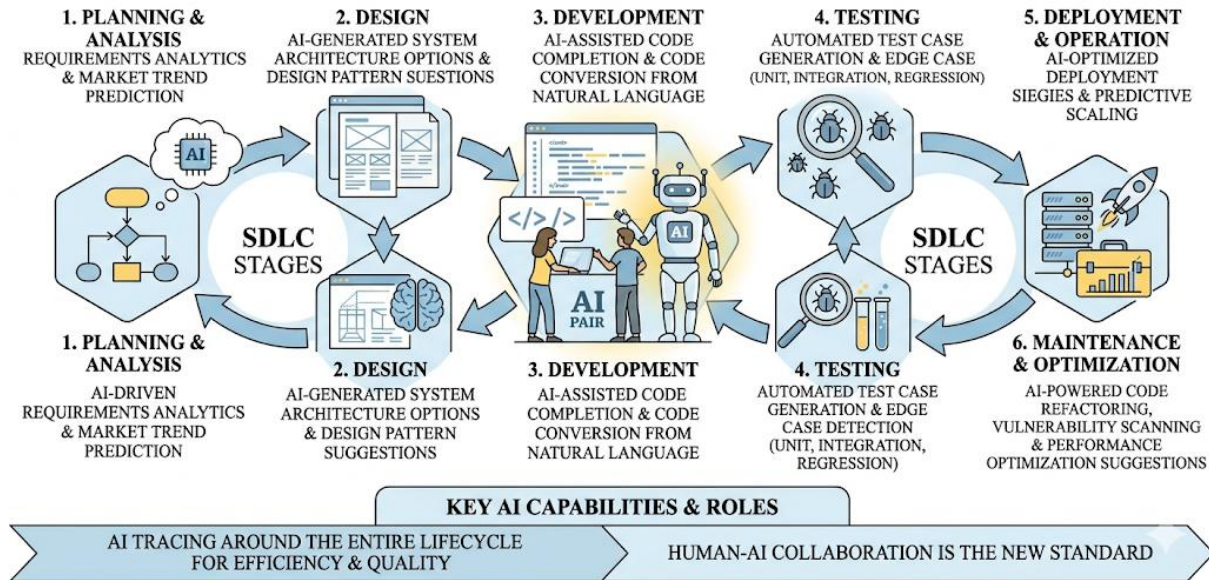
Theo các báo cáo dự đoán về thị trường lao động công nghệ, tỷ lệ lập trình viên tích hợp các công cụ AI vào quy trình làm việc hằng ngày sẽ tăng trưởng vượt bậc trong những năm tới. Từ một công nghệ sơ khởi với nhiều hạn chế về độ chính xác, GenAI đã nhanh chóng tiên hóa để trở thành một "trợ lý đắc lực" (AI Pair Programmer) không thể thay thế.

Sự chuyển dịch này được minh chứng qua việc GenAI đã tham gia sâu vào các công đoạn then chốt của SDLC:

- **Viết mã nguồn (Coding):** Khả năng tự động hoàn thiện đoạn mã (Auto-completion) và chuyển đổi ngôn ngữ tự nhiên thành mã nguồn giúp giảm bớt các tác vụ thủ công, lặp lại.
- **Tạo kịch bản kiểm thử (Test Case Generation):** AI có khả năng quét qua các hàm logic để tự động đề xuất các bộ kiểm thử đơn vị (Unit Tests), bao gồm cả các trường hợp biên (Edge cases) thường bị bỏ sót.

- **Sinh tài liệu kỹ thuật (Documentation):** Việc tóm tắt chức năng của các module phức tạp và viết tài liệu hướng dẫn (README, API Docs) giờ đây có thể thực hiện chỉ trong vài giây, đảm bảo tính nhất quán cho dự án.
- **Debug và tối ưu hóa:** GenAI hỗ trợ phát hiện các lỗ hổng bảo mật tiềm ẩn, giải thích các thông báo lỗi phức tạp và đề xuất các phương án tối ưu hóa hiệu năng dựa trên các best practices toàn cầu.

AI Integration Across the Software Development Lifecycle (SDLC) Stages



Hình 59: Mức độ chấp nhận của GenAI

Nhận định: Những dữ liệu thực nghiệm cho thấy GenAI không còn là một "xu hướng nhất thời" hay một món đồ chơi công nghệ. Nó đang xác lập vị thế là một thành phần cốt lõi và không thể thiếu trong quy trình phát triển phần mềm hiện đại. Những kỹ sư biết cách phối hợp với AI sẽ sở hữu lợi thế cạnh tranh không lồ so với những người chỉ làm việc theo phương thức truyền thống.

16.5. Thay đổi trong vai trò của lập trình viên

Sự tích hợp của GenAI vào quy trình phát triển phần mềm không tác động đồng nhất lên tất cả mọi người. Thay vào đó, nó tạo ra những thay đổi mang tính phân tầng, đòi hỏi mỗi cấp độ lập trình viên phải tái định nghĩa lại giá trị cốt lõi của mình để thích nghi với sự hiện diện của người trợ lý ảo này.

16.5.1. Lập trình viên mới (Junior Developer)

Đối với các lập trình viên mới vào nghề (Junior), GenAI vừa là một cơ hội lớn nhưng cũng đặt ra những thách thức về mặt kỹ năng nền tảng.

- **Tập trung vào các nhiệm vụ thực thi:** Các công việc truyền thống của một Junior thường xoay quanh việc viết các đoạn mã nhỏ (coding), xây dựng kịch bản kiểm thử đơn giản

(testing) và cập nhật tài liệu dự án (documentation). Đây chính là những mảng nhiệm vụ mà GenAI có khả năng hỗ trợ **tốt nhất và nhanh nhất**.

- **Hỗ trợ học tập và tăng tốc:** GenAI đóng vai trò như một người hướng dẫn trực tiếp, giúp Junior giải thích các cú pháp khó, sửa các lỗi cú pháp (syntax errors) ngay lập tức và gợi ý các cấu trúc mã chuẩn (boilerplate). Điều này giúp rút ngắn đáng kể thời gian "onboarding" vào một dự án mới.
- **Nguy cơ thiếu hụt chiều sâu:** Do AI có thể thực hiện quá tốt các nhiệm vụ thực thi, lập trình viên Junior có nguy cơ bị phụ thuộc, dẫn đến việc thiếu hiểu biết sâu sắc về "tại sao đoạn code lại chạy". Do đó, vai trò của họ đang dịch chuyển từ việc "**viết code**" sang "**hiểu và kiểm chứng code**" do AI sinh ra.

Trong kỷ nguyên GenAI, một Junior Developer giỏi không còn là người thuộc lòng cú pháp, mà là người biết sử dụng AI để hoàn thành các tác vụ thực thi một cách chính xác, đồng thời tận dụng thời gian dư thừa để học hỏi về tư duy hệ thống và kiến trúc phần mềm.

16.5.2. Lập trình viên cấp cao và chuyên gia (Senior Developer)

Đối với các lập trình viên dày dạn kinh nghiệm, GenAI không thay thế vai trò của họ mà đóng vai trò là một **công cụ đòn bẩy (Leverage)**. Sự tác động này thúc đẩy một cuộc dịch chuyển quan trọng từ kỹ năng thực thi sang kỹ năng tư duy chiến lược.

- **Tập trung vào thiết kế và kiến trúc:** Senior Developer giờ đây dành ít thời gian hơn cho việc gỡ từng dòng mã nguồn và dành nhiều thời gian hơn cho việc thiết kế kiến trúc hệ thống (System Design), lựa chọn các mẫu thiết kế (Design Patterns) và đảm bảo tính mở rộng (Scalability) của phần mềm. Đây là những tầng tư duy mà AI hiện tại vẫn chưa thể đạt tới sự toàn diện như con người.
- **Giải quyết các bài toán phức tạp và đặc thù:** Những vấn đề liên quan đến logic nghiệp vụ lắt léo, tối ưu hóa hệ thống ở mức độ hạ tầng hoặc xử lý các lỗi hệ thống mang tính dây chuyền vẫn đòi hỏi kinh nghiệm thực chiến và khả năng phán đoán nhạy bén của các chuyên gia.
- **Vai trò Thẩm định và Kiểm soát:** Thay vì trực tiếp viết code, Senior Developer đóng vai trò là "người gác cổng", thực hiện việc đánh giá mã nguồn (Code Review) do AI hoặc các Junior tạo ra để đảm bảo tính bảo mật và tuân thủ các tiêu chuẩn kỹ thuật của doanh nghiệp.

Vai trò của lập trình viên trong tương lai sẽ dịch chuyển mạnh mẽ từ "**Người viết mã**" (Coder) sang "**Người thiết kế và quản lý hệ thống**" (System Architect & Manager). Khả năng đặt câu hỏi đúng (Prompt Engineering) và hiểu rõ bức tranh tổng thể của dự án sẽ trở thành năng lực cạnh tranh cốt lõi.

16.6. GenAI và SWEBOK

Để đánh giá khách quan tầm ảnh hưởng của GenAI, chúng ta cần soi chiếu nó qua lăng kính của **SWEBOK (Software Engineering Body of Knowledge)** – bộ quy chuẩn định nghĩa các lĩnh vực

kiến thức cốt lõi của ngành kỹ thuật phần mềm. Thực tế cho thấy, mức độ tác động của GenAI là không đồng đều giữa các nhánh kiến thức này.

16.6.1. Các lĩnh vực chịu tác động mạnh (AI-Dominant)

GenAI hiện đang đóng vai trò "thay đổi cuộc chơi" trong những công đoạn mang tính thực thi cao và có tính lặp lại:

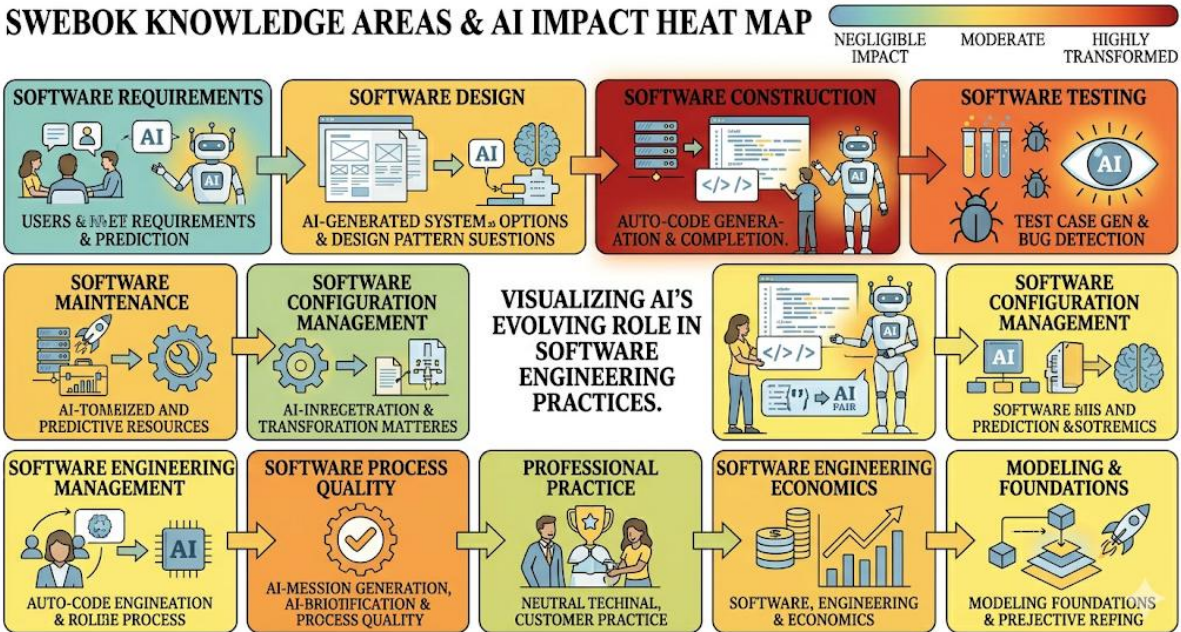
- **Xây dựng phần mềm (Software Construction):** Đây là nơi GenAI thể hiện sức mạnh rõ rệt nhất. Các mô hình AI có thể sinh mã nguồn, chuyển đổi ngôn ngữ lập trình và hoàn thiện các đoạn mã logic (boilerplate code) dựa trên yêu cầu từ lập trình viên.
- **Kiểm thử phần mềm (Software Testing):** AI hỗ trợ đắc lực trong việc tự động hóa tạo kịch bản kiểm thử (Test case generation), phát hiện lỗi (Bug detection) và mô phỏng các kịch bản biên để đảm bảo chất lượng phần mềm trước khi phát hành.

16.6.2. Các lĩnh vực vẫn do con người dẫn dắt (Human-Centric)

Ngược lại, các lĩnh vực đòi hỏi tư duy chiến lược, sự thấu hiểu bối cảnh kinh doanh và khả năng ra quyết định phức tạp vẫn phụ thuộc chủ yếu vào năng lực của các kỹ sư:

- **Kiến trúc phần mềm (Software Architecture):** Việc lựa chọn mô hình microservices hay monolith, thiết kế luồng dữ liệu giữa các thành phần và đảm bảo tính bền vững của hệ thống đòi hỏi một cái nhìn tổng thể mà AI chưa thể thay thế hoàn toàn.
- **Thiết kế phần mềm (Software Design):** Mặc dù AI có thể gợi ý các pattern, nhưng việc áp dụng chúng một cách linh hoạt để giải quyết các mâu thuẫn giữa hiệu năng và chi phí vẫn cần sự nhạy bén của con người.
- **Quản lý kỹ thuật phần mềm (Software Management):** Điều phối nguồn lực, quản trị rủi ro, thương lượng với khách hàng và thấu hiểu văn hóa đội ngũ là những kỹ năng mềm mà AI không thể sở hữu.

SWEBOK KNOWLEDGE AREAS & AI IMPACT HEAT MAP



Hình 60: Các lĩnh vực vẫn do con người dẫn dắt

Nhận định: GenAI đóng vai trò là một "công cụ tăng năng suất" cho các tác vụ kỹ thuật, nhưng con người vẫn giữ vai trò là "bộ não điều khiển" toàn bộ quy trình. Sự phân hóa này củng cố quan điểm rằng kỹ sư phần mềm tương lai cần tập trung vào các kỹ năng ở tầng cao hơn của SWEBOK để duy trì giá trị cốt lõi.

16.7. Dân chủ hóa lập trình (Democratization)

Sự trỗi dậy của GenAI đang xóa nhòa ranh giới giữa "lập trình viên" và "người dùng cuối". Hiện tượng này được gọi là Dân chủ hóa lập trình, nơi khả năng tạo ra các giải pháp phần mềm không còn bị giới hạn trong một nhóm nhỏ những người tinh thông mã máy.

Dân chủ hóa lập trình mang lại ba thay đổi mang tính bước ngoặt:

- **Xóa bỏ rào cản ngôn ngữ:** Trước đây, để viết code, bạn phải học thuộc hàng ngàn cú pháp phức tạp. Với GenAI, người dùng có thể sử dụng ngôn ngữ tự nhiên (Tiếng Việt, Tiếng Anh...) để mô tả yêu cầu. AI đóng vai trò là "thông dịch viên" chuyển đổi ý tưởng đó thành mã nguồn thực thi.
- **Sức mạnh cho chuyên gia đa ngành:** Các chuyên gia trong lĩnh vực tài chính, y tế, hay giáo dục giờ đây có thể tự xây dựng các công cụ phân tích dữ liệu hoặc tự động hóa công việc mà không cần đội ngũ IT. Điều này giúp các giải pháp chuyên biệt được ra đời nhanh hơn và sát với thực tế nghiệp vụ hơn.
- **Sự dịch chuyển bản chất kỹ năng:** Lập trình đang trải qua quá trình tương tự như kỹ năng soạn thảo văn bản hay sử dụng bảng tính (Excel). Nó đang dần chuyển dịch từ một kỹ năng chuyên sâu (Deep Skill) dành riêng cho một nghề nghiệp, trở thành một kỹ năng hỗ trợ (Utility Skill) cần thiết cho mọi nhân sự trong kỷ nguyên số.

Dân chủ hóa không làm mất đi giá trị của các kỹ sư phần mềm chuyên nghiệp. Ngược lại, nó đặt ra một tiêu chuẩn mới: Trong một thế giới mà ai cũng có thể viết code, các lập trình viên thực thụ phải là những người làm chủ kiến trúc, đảm bảo tính bảo mật và tối ưu hóa hệ thống ở mức độ mà AI và người dùng không chuyên chưa thể chạm tới.

16.8. Ảnh hưởng đến hệ thống legacy và tổ chức

GenAI không chỉ thay đổi cách chúng ta viết code mới mà còn mang lại những giải pháp đột phá cho một trong những thách thức lớn nhất của ngành phần mềm: Các hệ thống Legacy (hệ thống cũ). Đồng thời, nó cũng tái cấu trúc lại cách thức vận hành và quản lý nguồn nhân lực trong các tổ chức công nghệ.

16.8.1. Hồi sinh các hệ thống Legacy

Hệ thống legacy thường là "nỗi ám ảnh" của các doanh nghiệp vì mã nguồn lỗi thời, thiếu tài liệu và khó bảo trì. GenAI đóng vai trò như một "nhà khảo cổ học kỹ thuật số", giúp giải quyết các vấn đề này:

- **Hiểu và bảo trì hệ thống cũ:** AI có thể phân tích hàng triệu dòng code được viết bằng các ngôn ngữ lỗi thời (như COBOL, Fortran, hay các phiên bản Java/C# cũ), tự động sinh tài liệu giải thích logic nghiệp vụ, giúp các lập trình viên trẻ hiểu được hệ thống mà không cần mất nhiều năm nghiên cứu.
- **Tái cấu trúc và viết lại code (Refactoring & Modernization):** GenAI hỗ trợ đắc lực trong việc chuyển đổi mã nguồn cũ sang các ngôn ngữ hiện đại hơn (ví dụ: chuyển COBOL sang Java hoặc Python) mà vẫn đảm bảo giữ nguyên logic nghiệp vụ. Nó cũng giúp phát hiện các đoạn mã "rác", tối ưu hóa cấu trúc để hệ thống vận hành linh hoạt hơn.
- **Giảm sự phụ thuộc vào kỹ năng hiếm:** Các lập trình viên am hiểu ngôn ngữ cổ điển ngày càng ít đi và chi phí thuê họ rất cao. GenAI giúp lấp đầy khoảng trống này, cho phép các đội ngũ hiện đại tiếp quản và vận hành hệ thống mà không cần phải là chuyên gia về ngôn ngữ đó.

16.8.2. Tăng cường năng lực và tính linh hoạt của tổ chức

Bên cạnh việc xử lý kỹ thuật, GenAI còn tạo ra những thay đổi tích cực trong quản trị và vận hành đội ngũ:

- **Tăng tính linh hoạt của đội ngũ phát triển:** Nhờ sự hỗ trợ của AI trong việc sinh mã, kiểm thử và viết tài liệu, năng suất của mỗi cá nhân được nâng cao. Điều này cho phép đội ngũ phản ứng nhanh hơn trước các yêu cầu thay đổi của thị trường hoặc ưu tiên kinh doanh mới.
- **Cho phép chuyển đổi giữa các dự án nhanh hơn:** Khi một lập trình viên chuyển sang dự án mới, họ mất nhiều thời gian để hiểu code base hiện tại. GenAI giúp đẩy nhanh quá trình "onboarding" này bằng cách tóm tắt cấu trúc dự án, giải thích logic các module chính và

trả lời các câu hỏi về mã nguồn, giúp họ bắt đầu đóng góp cho dự án chỉ trong vài ngày thay vì vài tuần.

GenAI không chỉ là một công cụ viết code; nó là một giải pháp chiến lược giúp doanh nghiệp bảo vệ các khoản đầu tư công nghệ trong quá khứ, đồng thời xây dựng một tổ chức phát triển phần mềm tinh gọn, linh hoạt và không bị giới hạn bởi sự khan hiếm kỹ năng.

16.9. Tương lai của ngôn ngữ lập trình

Sự tiến hóa của ngôn ngữ lập trình luôn đi theo hướng tăng dần mức độ trừu tượng, từ mã máy (0 và 1) đến các ngôn ngữ bậc cao như Python hay Java. GenAI đang đẩy giới hạn này lên một tầm cao mới, nơi rào cản về cú pháp khắt khe đang dần được tháo gỡ.

Sự chuyển dịch này được dự báo qua ba giai đoạn quan trọng:

- **Ngôn ngữ tự nhiên là ngôn ngữ lập trình mới:** Trong tương lai gần, con người có thể mô tả các yêu cầu phần mềm phức tạp bằng ngôn ngữ tự nhiên (như Tiếng Việt hoặc Tiếng Anh). Thay vì phải lo lắng về việc thiếu một dấu chấm phẩy hay sai một tên biến, lập trình viên sẽ tập trung vào việc mô tả "**hệ thống cần làm gì**" thay vì "**làm điều đó như thế nào**".
- **AI đóng vai trò "Trình biên dịch thông minh":** Các mô hình ngôn ngữ lớn sẽ đóng vai trò là cầu nối, chuyển đổi các mô tả trừu tượng của con người thành mã nguồn thực thi chính xác. Quá trình này không chỉ dừng lại ở việc sinh mã mà còn tự động tối ưu hóa hiệu năng và bảo mật ngay trong quá trình biên dịch.
- **Pseudocode (Mã giả) - Giải pháp trung gian hoàn hảo:** Một xu hướng đang trỗi dậy là sử dụng mã giả có cấu trúc. Đây là sự kết hợp giữa tư duy logic chặt chẽ của con người và khả năng xử lý của máy tính. Mã giả giúp con người duy trì quyền kiểm soát đối với luồng xử lý (logic flow) mà không bị sa lầy vào các chi tiết cú pháp của từng ngôn ngữ cụ thể.

Ngôn ngữ lập trình sẽ không biến mất, nhưng cách chúng ta tương tác với chúng sẽ thay đổi hoàn toàn. Tư duy thuật toán và khả năng phân tích hệ thống sẽ trở nên quan trọng hơn nhiều so với việc ghi nhớ cú pháp của một ngôn ngữ cụ thể.

16.10. Vibe Coding

"Vibe coding" là một thuật ngữ mới trỗi dậy, phản ánh một phương thức tương tác hoàn toàn mới giữa con người và máy tính trong quá trình tạo ra phần mềm. Khác với lập trình truyền thống đòi hỏi sự chính xác tuyệt đối về cú pháp và cấu trúc, vibe coding tập trung vào việc **truyền tải "vibe" (cảm xúc, ý tưởng chủ đạo, luồng tư duy)** của nhà phát triển cho AI.

Trong mô hình này:

- **Con người là nhà đạo diễn:** Lập trình viên mô tả yêu cầu, chức năng, và cả giao diện mong muốn bằng ngôn ngữ tự nhiên phác thảo, mã giả (pseudocode), hoặc thậm chí là các câu lệnh mang tính cảm tính.

- **AI là người thực thi:** Dựa trên mô tả đó, AI tự động suy luận, lựa chọn công nghệ phù hợp và viết toàn bộ mã nguồn để hiện thực hóa ý tưởng.

Vibe coding mang lại những ưu điểm và thách thức rõ rệt:

Ưu điểm vượt trội:

- **Tốc độ phát triển cực nhanh:** Vibe coding loại bỏ rào cản về cú pháp và các tác vụ lặp lại, cho phép chuyển đổi ý tưởng thành sản phẩm chạy được chỉ trong vài phút.
- **Lý tưởng cho Prototyping (Tạo mẫu nhanh):** Đây là công cụ hoàn hảo để các startup, nhà thiết kế sản phẩm nhanh chóng hiện thực hóa ý tưởng để kiểm thử (test) thị trường mà không cần đầu tư quá nhiều nguồn lực ban đầu.

Nhược điểm và Rào cản:

- **Chất lượng code thấp và thiếu tối ưu:** Code do AI sinh ra dựa trên mô tả "vibe" thường thiếu sự tinh tế, có thể chứa các lỗi logic tiềm ẩn, code "rác" (redundant code) và không tuân thủ các best practices về bảo mật hoặc hiệu năng.
- **Cực kỳ khó bảo trì và debug:** Khi hệ thống trở nên phức tạp, việc hiểu và sửa đổi mã nguồn do AI tự động viết mà không có sự kiểm soát chặt chẽ của con người trở thành một thảm họa. Việc tìm ra nguyên nhân gốc rễ của lỗi (root cause) trong một "rừng" code AI là vô cùng khó khăn.

Vibe coding không thay thế lập trình chuyên nghiệp. Nó là một công cụ mạnh mẽ cho giai đoạn khởi tạo ý tưởng và tạo mẫu, nhưng để xây dựng các hệ thống cốt lõi (mission-critical), ổn định và có thể mở rộng, sự can thiệp và kiểm soát chặt chẽ của các kỹ sư phần mềm thực thụ vẫn là yếu tố không thể thay thế.

16.11. Tương lai của software engineering

Khi GenAI không còn là một công nghệ mới nổi mà trở thành một phần cốt lõi của ngành công nghiệp, chúng ta đang đứng trước ngưỡng cửa của những thay đổi cơ bản nhất trong lịch sử kỹ thuật phần mềm. Dựa trên các xu hướng hiện tại và dữ liệu thực nghiệm, chúng ta có thể phân tích tương lai của ngành theo hai giai đoạn: Ngắn hạn và Dài hạn.

16.11.1. Tầm nhìn Ngắn hạn (1-3 năm tới)

Trong giai đoạn này, sự tích hợp của GenAI sẽ mang tính chất **hỗ trợ và tối ưu hóa** các quy trình hiện có. AI không thay thế lập trình viên, nhưng nó sẽ làm cho họ trở nên mạnh mẽ và hiệu quả hơn.

AI tích hợp sâu vào các IDE (Môi trường phát triển tích hợp): Các trợ lý AI (như GitHub Copilot, Tabnine, codeium) không còn là các "add-on" tùy chọn mà sẽ trở thành một phần mặc định, không thể thiếu của các IDE như Visual Studio Code, IntelliJ, hay Eclipse. AI sẽ không chỉ tự động hoàn thiện mã nguồn (auto-completion) mà còn có khả năng:

Hiểu bối cảnh dự án toàn diện: AI có thể phân tích toàn bộ code base để đưa ra gợi ý không chỉ chính xác về cú pháp mà còn phù hợp với kiến trúc và các "design patterns" đã được thiết lập trong dự án.

Giải thích các lỗi phức tạp và lỗi tiềm ẩn: Thay vì chỉ hiển thị các thông báo lỗi (stack traces) khó hiểu, AI sẽ giải thích nguyên nhân gốc rễ và đề xuất cách sửa lỗi ngay lập tức, giúp giảm thiểu thời gian debug.

Tăng trưởng năng suất làm việc (Productivity Surge): Nhờ khả năng tự động hóa các tác vụ lặp lại (boilerplate code), sinh các hàm logic cơ bản, tự động tạo kịch bản kiểm thử (unit tests), và viết tài liệu hướng dẫn (README, API docs), lập trình viên có thể tập trung hoàn toàn vào việc giải quyết các bài toán logic nghiệp vụ phức tạp và tư duy hệ thống. Năng suất cá nhân và tập thể được dự báo sẽ tăng lên từ 30% đến 50%.

Cải thiện chất lượng code (Code Quality Improvement): AI đóng vai trò như một người "gác cổng" thông minh, thực hiện Code Review tự động ngay trong IDE. AI có thể phát hiện các lỗi logic phổ biến, các lỗ hổng bảo mật, và đưa ra gợi ý để tối ưu hóa mã nguồn dựa trên các "best practices" toàn cầu. Điều này giúp giảm thiểu lỗi kỹ thuật (technical debt) và đảm bảo tính nhất quán của mã nguồn trên toàn dự án.

Trong ngắn hạn, GenAI là một công cụ đòn bẩy. Lập trình viên không biết cách sử dụng AI sẽ gặp khó khăn lớn so với những người biết cách phối hợp với nó để đạt được hiệu suất và chất lượng vượt trội.

16.11.2. Tầm nhìn Dài hạn (4-10 năm tới)

Trong dài hạn, sự giao thoa giữa GenAI và Kỹ thuật phần mềm sẽ tiến tới mức độ tự động hóa cao cấp hơn, thay đổi hoàn toàn quy trình phát triển phần mềm truyền thống (SDLC).

Tự động chuyển đổi từ Requirement sang Code (End-to-End Automation): Khả năng hiểu ngôn ngữ tự nhiên của AI sẽ đạt đến độ chín muồi, cho phép nó tự động suy luận từ các tài liệu đặc tả yêu cầu (Software Requirements Specification - SRS) để tạo ra cấu trúc thư mục, logic nghiệp vụ và các API tương ứng. Lập trình viên sẽ chuyển sang vai trò "nguồn cấp dữ liệu yêu cầu" và "người kiểm chứng kết quả".

Hệ sinh thái tích hợp đa phương thức (Multimodal Integration): AI không chỉ đọc văn bản mà còn có khả năng hiểu các bản vẽ kỹ thuật.

Tích hợp UML: Tự động chuyển đổi các biểu đồ luồng dữ liệu (Data Flow), biểu đồ trình tự (Sequence Diagram) thành khung mã nguồn (Skeleton code).

Tích hợp Figma/Thiết kế: AI có thể "nhìn" các bản thiết kế UI/UX trên Figma và tự động sinh ra mã nguồn Front-end (React, Vue, Flutter) chính xác đến từng pixel, bao gồm cả các hiệu ứng chuyển động và xử lý sự kiện cơ bản.

Những "Pháo đài" cuối cùng của con người: Dù AI có tiến xa đến đâu, các khía cạnh mang tính hệ thống và vận hành vẫn cần sự điều phối của trí tuệ con người:

DevOps và Hạ tầng (Infrastructure): Việc quản lý các cụm máy chủ phức tạp, tối ưu hóa chi phí đám mây (Cloud cost), và xử lý các sự cố hạ tầng khẩn cấp đòi hỏi khả năng ứng biến và chịu trách nhiệm pháp lý mà AI chưa thể đảm đương.

Kiến trúc hệ thống (System Architecture): Quyết định đánh đổi (Trade-offs) giữa tính khả dụng, tính bảo mật và chi phí trong các hệ thống quy mô lớn vẫn là nghệ thuật của sự phán đoán dựa trên kinh nghiệm thực chiến.

Kỹ sư phần mềm trong tương lai dài hạn sẽ giống như một "**Tổng công trình sư**". Họ không trực tiếp cầm viên gạch (viết code) mà điều khiển các cánh tay robot (AI) để xây dựng nên những tòa tháp kỹ thuật số bền vững.

16.12. Liệu AI có thay thế lập trình viên?

Đây là một nỗi lo sợ phổ biến, nhưng dựa trên phân tích thực tế về khả năng của AI hiện tại và bản chất của ngành phần mềm, câu trả lời là **CHƯA** và khó có khả năng xảy ra trong tương lai gần.

16.12.1. Tại sao AI chưa thể thay thế hoàn toàn con người?

GenAI rất xuất sắc, nhưng nó vẫn là một mô hình thông kê học được từ dữ liệu quá khứ. Nó thiếu những yếu tố cốt lõi của một kỹ sư phần mềm thực thụ:

- **AI chỉ giỏi ở các nhiệm vụ nhỏ và cục bộ:** AI có thể viết một hàm logic, tạo một component UI, hoặc tối ưu hóa một đoạn code cụ thể chỉ trong vài giây. Tuy nhiên, nó gặp khó khăn cực lớn khi phải hiểu mối liên hệ giữa hàng ngàn module, quản lý trạng thái (state mangement) phức tạp của toàn bộ hệ thống, và đảm bảo tính nhất quán (consistency) của kiến trúc phần mềm trên quy mô lớn.
- **Sự phụ thuộc vào dữ liệu huấn luyện:** Các mô hình AI như GPT hay Claude cần một lượng lớn mã nguồn để học. Khi một công nghệ mới (như một ngôn ngữ, framework, hay thư viện vừa ra mắt), một kiến trúc máy tính đột phá, hoặc một bài toán kinh doanh hoàn toàn mới xuất hiện, AI sẽ **thiếu dữ liệu để huấn luyện** và không thể đưa ra gợi ý chính xác. Lập trình viên con người là những người tiên phong khám phá và tạo ra dữ liệu mới đó.
- **Tư duy phê phán và Đánh đổi (Trade-offs):** Xây dựng phần mềm không chỉ là viết code, mà là đưa ra hàng ngàn quyết định đánh đổi: Giữa tốc độ phát triển và hiệu năng, giữa chi phí hạ tầng và tính bảo mật, giữa tính linh hoạt và độ phức tạp. AI chưa có khả năng phán đoán ngữ cảnh kinh doanh (business context) và đạo đức kỹ thuật (engineering ethics) để đưa ra các quyết định này một cách độc lập.

16.12.2. Lập trình viên tương lai sẽ làm gì?

Vai trò của lập trình viên sẽ không biến mất, nhưng sẽ trải qua một cuộc dịch chuyển quan trọng:

- **Ít trực tiếp viết code hơn (Less Coding):** Các tác vụ viết code lặp lại, sinh boilerplate, hay unit test sẽ do AI đảm nhận. Lập trình viên sẽ sử dụng ngôn ngữ tự nhiên, mã giả, hoặc các công cụ thiết kế (như UML, Figma) để điều khiển AI sinh mã.

- **Tập trung vào Thiết kế và Kiểm soát Hệ thống (System Architecture & Manager):** Giá trị cốt lõi của họ sẽ nằm ở các kỹ năng tăng cao:
- **Kiến trúc phần mềm (System Design):** Phối hợp các thành phần AI-generated thành một hệ thống bền vững.
- **Kiểm soát chất lượng (Code Oversight):** Thăm định, đánh giá và sửa đổi code do AI tạo ra để đảm bảo tính logic và bảo mật.
- **Prompt Engineering:** Kỹ năng đặt câu hỏi và mô tả yêu cầu chính xác để AI hiểu và sinh ra mã nguồn tốt nhất.

AI sẽ không thay thế lập trình viên. Nhưng lập trình viên biết cách sử dụng AI hiệu quả chắc chắn sẽ thay thế những người không biết. Làn sóng AI không làm mất đi giá trị của con người, nó chỉ nâng cao tiêu chuẩn của một "kỹ sư phần mềm" thực thụ.

16.13. Rủi ro và quản trị

Sự bùng nổ của GenAI mang lại hiệu suất kinh ngạc, nhưng đồng thời cũng mở ra một "chiếc hộp Pandora" về các thách thức pháp lý và đạo đức. Việc tích hợp AI vào quy trình sản xuất phần mềm không chỉ là vấn đề kỹ thuật mà còn là bài toán về quản trị rủi ro doanh nghiệp.

16.13.1. Những thách thức cốt lõi

Trách nhiệm pháp lý (Liability): Khi một đoạn mã do AI tạo ra gây ra lỗi hệ thống nghiêm trọng hoặc làm rò rỉ dữ liệu người dùng, ai sẽ là người chịu trách nhiệm? Lập trình viên sử dụng AI, công ty phát triển phần mềm, hay đơn vị cung cấp mô hình AI? Đây là một vùng xám pháp lý đang gây tranh cãi gay gắt.

Vấn đề bản quyền (Intellectual Property - IP): Các mô hình ngôn ngữ lớn (LLM) được huấn luyện trên hàng tỷ dòng code công khai (Open Source). Điều này dẫn đến nguy cơ code do AI sinh ra vi phạm bản quyền hoặc vô tình sao chép các đoạn mã có giấy phép khắt khe (như GPL), gây rủi ro pháp lý cho các sản phẩm thương mại.

Tính minh bạch và khả năng giải thích (Explainability): Các mô hình AI thường hoạt động như một "hộp đen" (black box). Trong kỹ thuật phần mềm, việc hiểu *tại sao* một đoạn code được viết như vậy là cực kỳ quan trọng để bảo trì. Nếu AI sinh ra một logic phức tạp mà con người không thể giải thích, hệ thống đó sẽ trở nên cực kỳ nguy hiểm khi vận hành lâu dài.

16.13.2. Quản trị nghiêm ngặt trong các lĩnh vực nhạy cảm

Đặc biệt trong các lĩnh vực "High-stakes" như **Tài chính (FinTech)** và **Y tế (HealthTech)**, việc sử dụng GenAI sẽ bị giám sát chặt chẽ bởi các quy định nghiêm ngặt:

- **Tuân thủ quy chuẩn (Compliance):** Các hệ thống ngân hàng hoặc thiết bị y tế đòi hỏi độ tin cậy tuyệt đối. Mã nguồn phải trải qua các bước thăm định (Audit) nghiêm ngặt để đảm bảo không có "hallucination" (ảo tưởng AI) hay mã độc tiềm ẩn.

- **Kiểm soát dữ liệu:** Việc đưa mã nguồn nội bộ lên các công cụ AI đám mây có thể vi phạm các tiêu chuẩn bảo mật dữ liệu (như GDPR hay HIPAA). Điều này thúc đẩy xu hướng sử dụng các mô hình AI chạy cục bộ (Local LLMs) để đảm bảo an toàn thông tin.

GenAI không phải là chiếc "đũa thần" miễn phí. Quản trị rủi ro và tuân thủ đạo đức nghề nghiệp sẽ trở thành một phần không thể tách rời trong bộ kỹ năng của các kỹ sư phần mềm hiện đại.

16.14. Tác động đến giáo dục và tuyển dụng

Sự xuất hiện của GenAI không chỉ thay đổi cách chúng ta làm việc mà còn làm rung chuyển nền móng của hệ thống giáo dục và quy trình tuyển dụng nhân sự ngành phần mềm. Khi AI có thể viết code nhanh hơn bất kỳ con người nào, định nghĩa về một "lập trình viên giỏi" đang được viết lại.

16.14.1. Sự dịch chuyển trong Giáo dục Đào tạo

Kỹ năng viết code thủ công (Coding Syntax) giảm tầm quan trọng: Việc học thuộc lòng cú pháp, các hàm thư viện hay giải quyết các bài toán "vỡ lòng" bằng tay sẽ không còn là trọng tâm duy nhất. Sinh viên không còn cần dành quá nhiều thời gian để "vật lộn" với các lỗi cú pháp nhỏ nhất khi đã có AI hỗ trợ.

Trọng tâm mới - Tư duy hệ thống và Giải quyết vấn đề: Chương trình đào tạo sẽ tập trung vào các kỹ năng bậc cao:

- **Phân tích yêu cầu (Requirement Analysis):** Khả năng hiểu khách hàng thực sự muốn gì để truyền đạt lại cho AI.
- **Kiến trúc và Thiết kế (System Design):** Hiểu cách kết nối các thành phần phần mềm một cách bền vững.
- **Tư duy phản biện (Critical Thinking):** Biết đặt câu hỏi "Tại sao AI lại đưa ra giải pháp này?" thay vì chấp nhận nó một cách mù quáng.

16.14.2. Cách mạng trong Tuyển dụng

Các phương thức tuyển dụng truyền thống đang trở nên lỗi thời trước sức mạnh của AI:

- **Thay đổi các bài toán thuật toán (LeetCode-style):** Việc yêu cầu ứng viên giải các bài toán thuật toán kinh điển trong buổi phỏng vấn (như đảo ngược chuỗi hay tìm đường đi ngắn nhất) sẽ mất dần ý nghĩa vì AI có thể giải chúng trong nháy mắt. Các công ty sẽ chuyển sang đánh giá khả năng **vận dụng AI để giải quyết các bài toán thực tế phức tạp**.
- **Đánh giá khả năng "Hợp tác với AI":** Nhà tuyển dụng sẽ quan tâm đến việc ứng viên có biết cách đặt câu lệnh (Prompt Engineering), cách kiểm tra (Verify) và tối ưu hóa kết quả từ AI hay không.

- **Phỏng vấn dựa trên hệ thống (Architecture-based Interview):** Các buổi phỏng vấn sẽ tập trung vào việc yêu cầu ứng viên thiết kế một hệ thống lớn, giải thích các lựa chọn công nghệ và cách xử lý các tình huống rủi ro thực tế.

Giáo dục và tuyển dụng sẽ chuyển dịch từ việc tìm kiếm những "**Máy viết code**" (**Coding machines**) sang tìm kiếm những "**Người giải quyết vấn đề**" (**Problem solvers**) có khả năng làm chủ công nghệ AI.

16.15. Kết luận

Chương 16 cho thấy GenAI đang tạo ra một bước ngoặt lớn trong ngành phần mềm. Nó không chỉ giúp tăng năng suất mà còn thay đổi cách chúng ta phát triển, quản lý và hiệu phần mềm.

Trong tương lai, lập trình viên cần thích nghi bằng cách nâng cao kỹ năng về thiết kế hệ thống, tư duy logic và khả năng làm việc với AI. GenAI không thay thế con người mà đóng vai trò như một công cụ mạnh mẽ giúp con người làm việc hiệu quả hơn.

DANH MỤC TÀI LIỆU THAM KHẢO

[1] Packt Publishing, *Supercharged Coding with GenAI*. Birmingham, UK: Packt Publishing, 2025.

[2] Packt Publishing, "Supercharged Coding with Gen-AI Repository," 2025. [Online]. Available: <https://github.com/PacktPublishing/Supercharged-Coding-with-Gen-AI>.